

Tập luận văn đội tuyển quốc gia Trung Quốc năm 2020

Tập luận văn đội tuyển quốc gia Olympic Tin học

China Computer Federation, 2020

Nguồn gốc: Tập luận văn đội tuyển quốc gia Trung Quốc Olympic Tin học năm 2020

I0I China candidate team papers / National training team 2020 collection.pdf

Tóm tắt

PDF nguồn là tập hợp luận văn đội tuyển quốc gia Trung Quốc năm 2020. Tập được tạo bằng XeTeX và có lớp văn bản Unicode đọc được. Bản LaTeX này chuyển ngữ phần nhận diện tập sách, toàn bộ mục lục, và mười bài đầu tiên: bài của Jiang Mingrun về tối ưu các bài toán chia khối cổ điển bằng fractional cascading, bài của Chen Sunli về cây thống trị, báo cáo ra đề *Ghép số* của Jiang Xunchi, báo cáo ra đề *Khối liên thông nhỏ nhất* của Pan Junyue, bài của Chen Yu về nguyên lý chuyển vị, và bài của Li Baitian về duy trì động giá trị cực trị của hàm số, cùng báo cáo ra đề *Trò chơi trên đồ thị* của Zuo Junchi, bài của Zhou Yuyang về một loại DFT không bình thường, và bài của Zhang Zheyu về bài toán rừng hướng vào nhỏ nhất, và bài của Xu Yixuan về suffix automaton nén. Các hình nhỏ trong báo cáo thứ tư và thứ bảy được dựng lại bằng TikZ từ bản render PDF; các hình minh họa trong bài thứ chín và thứ mười được ghi lại bằng chú thích khung khi nội dung chính đã nằm trong lời văn và công thức.

1 Thông tin đầu sách

Tập sách có tiêu đề *Tập luận văn đội tuyển quốc gia Trung Quốc Olympic Tin học năm 2020*. Tập PDF gồm 196 trang và được tạo bằng XeTeX. Trang bìa không ghi riêng tên huấn luyện viên trong phần văn bản trích xuất được.

2 Mục lục đã dịch

Trang	Bài viết	Tác giả
3	Bàn về tối ưu hóa bài toán chia khối cổ điển bằng fractional cascading	Jiang Mingrun
12	Bàn về cây thống trị và ứng dụng	Chen Sunli
22	Báo cáo ra đề <i>Ghép số</i>	Jiang Xunchi
32	Báo cáo ra đề <i>Khối liên thông nhỏ nhất</i>	Pan Junyue
42	Giới thiệu đơn giản về nguyên lý chuyển vị	Chen Yu
52	Bàn về duy trì động giá trị cực trị của hàm số	Li Baitian
63	Báo cáo ra đề <i>Trò chơi trên đồ thị</i>	Zuo Junchi
74	Một loại DFT không bình thường	Zhou Yuyang
87	Bài toán rừng hướng vào nhỏ nhất	Zhang Zheyu
102	Bàn về suffix automaton nén	Xu Yixuan
117	Bàn về nimber và thuật toán đa thức	Luo Yuxiang
133	Các bài toán liên quan tới định thức trong thi tin học	Wang Zhanpeng
151	Thuật toán hiệu quả để tính diện tích vùng ghép trên mặt cầu	Zhou Renfei
173	Bàn về ứng dụng đơn giản của lý thuyết nhóm trong thi tin học	Yu Haoxiang

3 Bàn về tối ưu hóa bài toán chia khối cổ điển bằng fractional cascading

Jiang Mingrun, Trường Trung học số 7 Thành Đô

Tóm tắt

Fractional cascading là một kỹ thuật tăng tốc tìm kiếm nhị phân một giá trị trong nhiều dãy số được tổ chức theo một cấu trúc nhất định. Bài viết này giới thiệu cách dùng thuật toán đó để tối ưu một bài toán chia khối cổ điển, đồng thời trình bày một vài ứng dụng của nó.

Lời nói đầu

Thuật toán fractional cascading đã ra đời hơn 20 năm, nhưng vẫn chưa được dùng rộng rãi trong cộng đồng thi thuật toán. Bài viết này thử dùng thuật toán đó để tối ưu bài toán chia khối cổ điển, với hy vọng đưa fractional cascading vào thi tin học. Mục 1 giới thiệu fractional cascading và ý nghĩa của nó; mục 2 giới thiệu bài toán chia khối cổ điển và các cách giải thường dùng; mục 3 dùng fractional cascading để tối ưu bài toán chia khối cổ điển, so sánh với các thuật toán truyền thống, đồng thời mở rộng bài toán cổ điển.

3.1 Fractional cascading

Mục này giới thiệu thuật toán fractional cascading. Ta sẽ dùng một ví dụ cụ thể để giải thích ý nghĩa của thuật toán.

3.1.1 Giới thiệu

Trong khoa học máy tính, fractional cascading là một kỹ thuật tăng tốc tìm kiếm nhị phân một số trong nhiều dãy số được tổ chức theo một cấu trúc nhất định: lần tìm kiếm đầu tiên là tìm kiếm nhị phân thông thường với độ phức tạp $O(\log n)$, còn các lần tìm kiếm nhị phân phía sau sẽ nhanh hơn lần đầu. Fractional

cascading được Chazelle và Guibas đề xuất lần đầu trong hai bài báo năm 1986 [1, 2]. Đúng như tên gọi, “fractional” mang nghĩa rời rạc, nhỏ lẻ; “cascade” là một ẩn dụ chỉ sự xếp tầng như các thác nước nhỏ. Trong bài gốc, Chazelle và Guibas đưa ra một hình minh họa, ở đây gọi là Hình 1, mô tả rất trực quan ý nghĩa sâu hơn của tên gọi này: giống như thác nước từ cao xuống thấp, dần tách thành các nhánh nhỏ hơn rồi phủ chồng lên nhau theo từng tầng [3, 4].

Hình 1: hình minh họa do Chazelle và Guibas đưa ra trong bài gốc.

3.1.2 Minh họa bằng ví dụ

Ta sẽ mô tả thuật toán này dưới dạng một bài toán ví dụ.

Xét bài toán sau: cho k dãy có thứ tự với tổng độ dài là n , dãy thứ i ký hiệu là L_i . Cần xây dựng một cấu trúc dữ liệu hỗ trợ tìm kiếm nhị phân cùng một số q trong từng dãy trong k dãy, tức tìm vị trí của phần tử đầu tiên lớn hơn hoặc bằng q . Ví dụ dưới đây có $k = 4, n = 17$:

$$\begin{aligned} L_1 &= 24, 64, 65, 80, 93, \\ L_2 &= 23, 25, 26, \\ L_3 &= 13, 44, 62, 66, \\ L_4 &= 11, 35, 46, 79, 81. \end{aligned}$$

Cách đơn giản nhất là lưu riêng từng dãy. Khi đó ta chỉ cần độ phức tạp không gian $O(n)$. Nhưng khi truy vấn, ta phải tìm kiếm nhị phân riêng trên từng dãy; trong trường hợp xấu nhất, khi k dãy có độ dài bằng nhau, độ phức tạp thời gian là $O(k \log(n/k))$.

Cách thứ hai hỗ trợ truy vấn nhanh hơn nhưng tốn nhiều không gian hơn: ta gộp k dãy này thành một dãy lớn L , và với mỗi phần tử trong L lưu kết quả tìm kiếm nhị phân của nó trong k dãy ban đầu. Nếu mỗi phần tử trong dãy sau khi gộp được viết là $x[a, b, c, d]$, trong đó x là giá trị, còn a, b, c, d là vị trí của phần tử kế tiếp tương ứng của x trong các dãy ban đầu, đánh số từ 0, thì:

$$\begin{aligned} L &= 11[0, 0, 0, 0], 13[0, 0, 0, 1], 23[0, 0, 1, 1], 24[0, 1, 1, 1], 25[1, 1, 1, 1], \\ &26[1, 2, 1, 1], 35[1, 3, 1, 1], 44[1, 3, 1, 2], 46[1, 3, 2, 2], 62[1, 3, 2, 3], \\ &64[1, 3, 3, 3], 65[2, 3, 3, 3], 66[3, 3, 3, 3], 79[3, 3, 4, 3], 80[3, 3, 4, 4], \\ &81[4, 3, 4, 4], 93[4, 3, 4, 5]. \end{aligned}$$

Cách này đạt độ phức tạp truy vấn $O(k + \log n)$: chỉ cần tìm kiếm nhị phân trực tiếp trong L , rồi trả về thông tin đã lưu trong phần tử x . Tuy nhiên, nó cần không gian $O(nk)$.

Fractional cascading đồng thời đạt độ phức tạp thời gian và không gian tối ưu cho cùng bài toán: truy vấn $O(k + \log n)$, không gian $O(n)$. Ta xây dựng k dãy có thứ tự mới, dãy thứ i ký hiệu là M_i . Trong đó, dãy cuối cùng M_k giống L_k . Với mỗi dãy phía trước, M_i được tạo bằng cách trộn L_i với dãy con lấy mẫu cách một phần tử của M_{i+1} , bắt đầu từ phần tử thứ hai; với mỗi phần tử trong M_i , ta lưu kết quả tìm kiếm nhị phân của nó trong L_i và M_{i+1} . Với 4 dãy ở trên, ta có:

$$\begin{aligned} M_1 &= 24[0, 1], 25[1, 1], 35[1, 3], 64[1, 5], 65[2, 5], 79[3, 5], 80[3, 6], 93[4, 6], \\ M_2 &= 23[0, 1], 25[1, 1], 26[2, 1], 35[3, 1], 62[3, 3], 79[3, 5], \\ M_3 &= 13[0, 1], 35[1, 1], 44[1, 2], 62[2, 3], 66[3, 3], 79[4, 3], \\ M_4 &= 11[0, 0], 35[1, 0], 46[2, 0], 79[3, 0], 81[4, 0]. \end{aligned}$$

Nếu muốn truy vấn $q = 50$, trước hết ta tìm kiếm nhị phân trong M_1 và nhận được $64[1, 5]$. Số 1 nghĩa là kết quả tìm kiếm nhị phân trong L_1 là $L_1[1] = 64$; số 5 nghĩa là kết quả tìm kiếm nhị phân trong

M_2 nằm xấp xỉ ở chỉ số 5. Chính xác hơn, nó là phần tử 79[3, 5] ở chỉ số 5 hoặc phần tử 62[3, 3] ngay trước đó. So sánh q với 62, ta biết kết quả đúng là 62[3, 3]. Bằng cùng phương pháp, ta có thể nhận được kết quả tìm kiếm nhị phân của q trong L_2, M_3, L_3 , và M_4 .

Nói tổng quát hơn, với bất kỳ cấu trúc như vậy, ta có thể tìm kiếm nhị phân trước trong M_1 . Sau đó với mọi i , từ vị trí của q trong M_i ta định vị được vị trí của q trong L_i và trong M_{i+1} . Vị trí trong M_{i+1} mà kết quả truy vấn ở M_i trở tới hoặc chính là kết quả đúng của truy vấn trong M_{i+1} , hoặc chỉ lệch một vị trí, nên một phép so sánh là đủ để xác định. Tổng độ phức tạp thời gian là $O(k + \log n)$.

Trong ví dụ trên, tổng độ dài của bốn dãy mới là 25, không vượt quá $2n$. Nói chung, độ dài của M_i không vượt quá

$$|L_i| + \frac{1}{2}|L_{i+1}| + \frac{1}{4}|L_{i+2}| + \dots.$$

Tổng độ dài của mọi M_i không vượt quá

$$\sum |M_i| \leq \sum |L_i| \left(1 + \frac{1}{2} + \frac{1}{4} + \frac{1}{8} + \dots\right) = 2n = O(n).$$

3.1.3 Trường hợp tổng quát

Xét một đồ thị có hướng không chu trình. Bậc vào và bậc ra của mỗi đỉnh đều không vượt quá một hằng số d , và trên mỗi đỉnh có lưu một dãy có thứ tự L_u . Một truy vấn cần tìm kiếm nhị phân một số q trong các dãy L_u được lưu trên mọi đỉnh u của một đường đi. Trong ví dụ phía trên, đồ thị có hướng đã cho là một dãy chuyển độ dài 4.

Với mỗi đỉnh u , ta xây dựng một dãy mới M_u . Dãy M_u được tạo bằng cách trộn L_u với các dãy thu được khi lấy mẫu đều theo một tỉ lệ nhất định từ M_v của mọi hậu duệ trực tiếp v của u ; nếu muốn đạt không gian $O(n)$, tỉ lệ lấy mẫu phải nhỏ hơn $1/d$. Với mỗi phần tử, ta lưu kết quả tìm kiếm nhị phân của nó trong L_u và trong từng M_v . Trong ví dụ phía trên, $d = 1$ và tỉ lệ lấy mẫu là $1/2$.

Với một truy vấn, trước hết ta thực hiện một lần tìm kiếm nhị phân $O(\log n)$ trong dãy M_s được xây dựng ở điểm đầu s của đường đi. Giả sử ta đã biết kết quả tìm kiếm nhị phân của q trong M_u , và cần tìm kết quả tìm kiếm nhị phân của q trong M_v với một hậu duệ trực tiếp v của u . Nếu tỉ lệ lấy mẫu khi xây dựng cấu trúc dữ liệu là $1/r$, thì khoảng cách giữa vị trí trong M_v mà kết quả tìm kiếm nhị phân trong M_u trở tới và vị trí đúng trong M_v sẽ không vượt quá r . Vì r là hằng số, ta có thể tìm kết quả đúng trong M_v với độ phức tạp thời gian $O(1)$.

Nếu độ dài đường đi cần truy vấn là k , độ phức tạp thời gian của một truy vấn là $O(k + \log n)$.

Ngoài ra, fractional cascading có thể hỗ trợ chèn một phần tử vào một dãy trong $O(\log n)$, nhưng độ phức tạp truy vấn sẽ trở thành $O(\log n + k \log \log n)$. Điều này không liên quan nhiều tới bài viết này nên được lược bỏ.

3.2 Bài toán chia khối cổ điển

Mục này giới thiệu một bài toán chia khối kinh điển trong OI và cách giải truyền thống của nó. Ở mục sau ta sẽ thảo luận cách dùng fractional cascading để tối ưu bài toán này.

3.2.1 Mô tả bài toán

Cho một dãy độ dài n và n thao tác. Có hai loại thao tác:

1. Thao tác sửa: cho l, r, x ; với mọi $l \leq i \leq r$, tăng a_i thêm x .
2. Thao tác hỏi: cho l, r, x ; hỏi trong mọi $l \leq i \leq r$, có bao nhiêu chỉ số i thỏa $a_i \leq x$.

3.2.2 Cách giải thường dùng, thuật toán 1

Chia dãy thành các khối, mỗi khối gồm B phần tử liên tiếp, tổng cộng $O(n/B)$ khối. Trong mỗi khối, lưu kết quả sắp xếp của mọi phần tử trong khối. Như vậy với một đoạn $[l, r]$, nhiều nhất chỉ có 2 khối bị phủ không hoàn toàn; các khối còn lại hoặc được phủ hoàn toàn, hoặc không bị phủ.

Với thao tác sửa, các khối bị phủ không hoàn toàn cần được xây dựng lại trực tiếp. Lúc này phải tính lại kết quả sắp xếp. Nếu sắp xếp thô thì độ phức tạp là $O(B \log B)$; nhưng chú ý rằng với hai số đều không bị sửa hoặc hai số đều bị sửa, quan hệ lớn nhỏ giữa chúng không đổi. Vì vậy, nếu khi sắp xếp ta ghi lại vị trí của phần tử trước khi sắp xếp, ta có thể sắp riêng các số không bị sửa và các số bị sửa trong $O(B)$, rồi trộn lại, nên độ phức tạp thời gian là $O(B)$. Với khối được phủ hoàn toàn, ta có thể ghi một thể lười biểu thị rằng mọi phần tử trong khối đều được cộng cùng một số. Độ phức tạp cho mỗi khối là $O(1)$, tổng cộng có $O(n/B)$ khối, nên tổng độ phức tạp thời gian là $O(n/B)$.

Với thao tác hỏi, các khối bị phủ không hoàn toàn có thể được duyệt trực tiếp qua mọi phần tử, độ phức tạp $O(B)$. Với khối được phủ hoàn toàn, ta cần tìm kiếm nhị phân trên mảng đã sắp xếp, độ phức tạp cho mỗi khối là $O(\log B)$, tổng độ phức tạp thời gian là $O(n \log B/B)$.

Tóm lại, độ phức tạp thời gian cho một thao tác là $O(B + n \log B/B)$; tổng độ phức tạp thời gian là $O(nB + n^2 \log B/B)$. Khi lấy $B = \sqrt{n \log n}$, độ phức tạp thời gian là tối ưu và bằng $O(n\sqrt{n \log n})$.

3.2.3 Thuật toán offline, thuật toán 2

Mục này thảo luận thuật toán offline.

Trong bài này, thuật toán ở mục này chỉ dùng được với phạm vi đầu vào sau: mọi số trong đầu vào đều là số nguyên có trị tuyệt đối không vượt quá 2^ω , và $\omega = O(\log n)$.

Chú ý rằng nút thắt của thuật toán ở mục trước nằm ở tìm kiếm nhị phân, nên mục này xét cách tối ưu phần đó.

Tuy nhiên, tối ưu một lần tìm kiếm nhị phân đơn lẻ là việc khó; có thể dùng các cấu trúc dữ liệu như y-fast tree, nhưng không thực dụng. Do đó, để tối ưu thuật toán, ta cần tối ưu tổng độ phức tạp thời gian của nhiều lần tìm kiếm nhị phân. Có hai hướng: hướng thứ nhất là tối ưu nhiều truy vấn tìm kiếm nhị phân trong một khối; hướng thứ hai là tối ưu một truy vấn tìm kiếm nhị phân trên nhiều khối. Mục này thảo luận hướng thứ nhất, còn hướng thứ hai sẽ được thảo luận ở mục sau.

Xét việc lưu trước mọi truy vấn và xử lý từng khối. Khi xây dựng lại một khối, ta tính kết quả của mọi lần tìm kiếm nhị phân trước đó trên khối này. Khi đó bài toán chuyển thành thực hiện Q lần tìm kiếm nhị phân trong một dãy độ dài B .

Chú ý rằng nếu các số được tìm kiếm nhị phân tăng đơn điệu, ta có thể tận dụng tính đơn điệu để đạt độ phức tạp $O(B + Q)$. Vì vậy nếu có thể sắp xếp nhanh Q số, ta có thể nhanh chóng tính được kết quả của Q lần tìm kiếm nhị phân.

Khi $\omega = O(\log B)$, có thể dùng sắp xếp cơ số với cơ số B , độ phức tạp thời gian là

$$O\left(\frac{(B + Q)\omega}{\log B}\right) = O(B + Q).$$

Như vậy, độ phức tạp thời gian của bài toán chia khối ban đầu có thể được tối ưu thành $O(nB + n^2/B)$; lấy $B = \sqrt{n}$ là tối ưu, cho $O(n\sqrt{n})$.

3.3 Thuật toán dựa trên fractional cascading

Mục trước đã giới thiệu bài toán chia khối cổ điển trong OI và các cách giải truyền thống. Mục này thảo luận cách dùng fractional cascading để tối ưu bài toán đó.

Cách dùng sắp xếp cơ sở có thể đạt độ phức tạp thời gian $O(n\sqrt{n})$, nhưng nó không còn dùng được nếu miền giá trị không phải số nguyên, hoặc nếu bài toán bị buộc phải online, nghĩa là trước khi đọc một thao tác mới thì phải tính xong kết quả của mọi truy vấn trước đó, nên không thể lưu trước toàn bộ truy vấn.

Một lần tìm kiếm nhị phân riêng lẻ rất khó tối ưu. Nhưng cần chú ý rằng thứ ta cần không phải một lần tìm kiếm nhị phân riêng lẻ, mà là tìm kiếm nhị phân cùng một số trên $O(n/B)$ dãy. Vì vậy, có thể xét dùng fractional cascading. Tuy nhiên, fractional cascading không hỗ trợ sửa trực tiếp, nên cần xử lý thêm.

3.3.1 Cách làm $O(n\sqrt{n} \log \log n)$, thuật toán 3

Cai Chengze của Đại học Thanh Hoa đã nghĩ ra thuật toán có độ phức tạp thời gian $O(n\sqrt{n} \log \log n)$ trước tác giả, nhưng chưa công bố chi tiết. Cách làm được mô tả trong mục này là một cách khả dĩ mà tác giả suy đoán từ một số trao đổi liên quan; không rõ có trùng với cách làm của Cai hay không. Dù thế nào, xin cảm ơn Cai Chengze.

Trong thuật toán này, cách xây dựng fractional cascading là: trong mỗi khối, lấy mẫu đều $1/D$ số phần tử để xây dựng fractional cascading. Khi tìm kiếm nhị phân, nhờ fractional cascading ta định vị được một vị trí lệch không quá D , do đó trong mỗi khối riêng lẻ đạt độ phức tạp thời gian $O(\log D)$. Dùng cách này, cứ mỗi D khối liên tiếp xây dựng một nhóm fractional cascading; tổng cộng có $O(n/(BD))$ nhóm.

Khi thực hiện thao tác sửa, nhiều nhất chỉ có 2 nhóm fractional cascading bị phá vỡ cấu trúc; các nhóm khác nhiều nhất chỉ bị cộng cùng một số vào mọi phần tử, điều này không thay đổi cấu trúc fractional cascading. Vì mỗi khối chỉ lấy $O(B/D)$ phần tử để xây dựng fractional cascading, và mỗi nhóm fractional cascading chỉ được xây dựng giữa D khối, nên độ phức tạp thời gian để xây dựng lại một nhóm là $O(B)$.

Với thao tác hỏi, cần thực hiện một truy vấn trong mỗi nhóm fractional cascading. Độ phức tạp truy vấn trong một nhóm là $O(D + \log B)$; có tổng cộng $O(n/(BD))$ nhóm, nên tổng độ phức tạp là $O(n/B + n \log B/(BD))$. Ngoài ra, với mỗi khối còn cần một lần tìm kiếm nhị phân $O(\log D)$, tổng độ phức tạp là $O(n \log D/B)$. Ta cũng phải duyệt $O(B)$ phần tử trong các khối bị phủ không hoàn toàn.

Tóm lại, độ phức tạp thời gian của một thao tác có thể được tối ưu thành

$$O\left(B + \frac{n \log D}{B} + \frac{n \log B}{BD}\right),$$

tổng độ phức tạp thời gian là

$$O\left(nB + \frac{n^2 \log D}{B} + \frac{n^2 \log B}{BD}\right).$$

Lấy $B = \sqrt{n \log \log n}$, $D = \log n$, độ phức tạp thời gian là $O(n\sqrt{n} \log \log n)$.

3.3.2 Cách làm $O(n\sqrt{n})$, thuật toán 4

Nếu dùng cách gộp ở mục 1.2 thì đã khó tối ưu tiếp. Tiếp theo, mục này bắt đầu từ một cách gộp khác để tối ưu bài toán thêm một bước.

Xây dựng một cây đoạn có n/B lá; các lá của cây đoạn lần lượt lưu dãy đã sắp xếp được duy trì trong mỗi khối. Dãy lưu ở đỉnh trong được tạo bằng cách trộn các dãy thu được khi lấy mẫu đều theo một tỉ lệ nhất định từ dãy lưu ở hai con, và ghi lại với mỗi phần tử kết quả tìm kiếm nhị phân của nó trong hai dãy lưu ở hai con. Nói cách khác, trong mục 1.3, cho mỗi đỉnh trong của cây đoạn nối tới hai con của nó để tạo đồ thị có hướng không chu trình; L_u ở lá là dãy được duy trì trong mỗi khối, còn L_u ở đỉnh trong là dãy rỗng, từ đó nhận được cấu trúc fractional cascading.

Khi thực hiện thao tác sửa, theo tính chất của cây đoạn, chỉ có $O(\log(n/B))$ đỉnh có đoạn tương ứng bị phủ không hoàn toàn. Nếu đoạn ứng với một đỉnh được phủ hoàn toàn, mọi phần tử trong cây con của

đỉnh đó chỉ bị cộng thêm cùng một số, còn cấu trúc không đổi. Vì vậy chỉ có $O(\log(n/B))$ đỉnh cần tính lại dây lưu trữ. Khi tỉ lệ lấy mẫu là $1/2$, độ dài dây lưu ở mỗi đỉnh đều là $O(B)$, tổng độ phức tạp thời gian là $O(B \log(n/B))$, chưa đủ tốt. Nhưng nếu dùng tỉ lệ nhỏ hơn, chẳng hạn $1/3$, thì ở mỗi tầng của cây đoạn, độ dài dây lưu ở mỗi đỉnh bằng $2/3$ so với tầng dưới; theo tính chất của cây đoạn, ở mỗi tầng chỉ có $O(1)$ đỉnh bị phủ không hoàn toàn. Vì vậy tổng độ phức tạp thời gian không vượt quá

$$O\left(B\left(1 + \frac{2}{3} + \frac{4}{9} + \frac{8}{27} + \dots\right)\right) = O(B).$$

Với thao tác hỏi, tương tự mục 1.3, ta có thể tìm kiếm nhị phân trước trong gốc, rồi từ kết quả tìm kiếm nhị phân trong một đỉnh xác định trong $O(1)$ kết quả tìm kiếm nhị phân ở hai con. Tổng độ phức tạp thời gian là $O(n/B + \log n)$. Ngoài ra, ta vẫn cần duyệt $O(B)$ phần tử trong các khối bị phủ không hoàn toàn.

Tóm lại, độ phức tạp thời gian của một thao tác là $O(B + n/B + \log n)$; tổng độ phức tạp thời gian là $O(nB + n^2/B + n \log n)$. Lấy $B = \sqrt{n}$, độ phức tạp thời gian là tối ưu và bằng $O(n\sqrt{n})$.

3.3.3 Tổng kết thuật toán

Tổng hợp lại, độ phức tạp thời gian của các thuật toán được nêu trong Bảng 1, trong đó thuật toán 1 là thuật toán truyền thống. Thuật toán 2 cũng đạt mục tiêu tối ưu, nhưng chỉ phù hợp với một phần phạm vi đầu vào. Thuật toán 3 và thuật toán 4 đều là các thuật toán tối ưu bằng fractional cascading, không bị hạn chế khi sử dụng; trong các thuật toán đã biết, thuật toán 4 là tối ưu nhất.

Thuật toán	Độ phức tạp thời gian	Ghi chú
Thuật toán 1	$O(n\sqrt{n \log n})$	
Thuật toán 2	$O(n\sqrt{n})$	Chỉ phù hợp với một phần phạm vi đầu vào
Thuật toán 3	$O(n\sqrt{n \log \log n})$	
Thuật toán 4	$O(n\sqrt{n})$	

Bảng 1: So sánh các thuật toán

3.3.4 Mở rộng ứng dụng thuật toán

Fractional cascading không chỉ có thể áp dụng cho bài toán cổ điển này, mà còn có thể mở rộng sang một số bài toán khác.

Bài toán 1. Cho một dãy độ dài n và n thao tác. Có hai loại thao tác:

1. Thao tác sửa: cho l, r, x ; với mọi $l \leq i \leq r$, tăng a_i thêm x .
2. Thao tác hỏi: cho l, r, x ; hỏi đoạn $[l, r]$ có bao nhiêu đoạn con $[l', r']$ thỏa mọi số trong $[l', r']$ đều không vượt quá x .¹

Chú ý rằng với hai đoạn $[l, mid]$ và $[mid + 1, r]$, nếu ta biết đáp án của chúng, ở đây “đáp án” bao gồm đáp án của thao tác hỏi, vị trí của số đầu tiên lớn hơn x trong đoạn, và vị trí của số cuối cùng lớn hơn x trong đoạn, thì ta có thể tính đáp án của đoạn $[l, r]$.

¹Bài toán này được chỉnh sửa từ Comet OJ Contest #7 F: https://cometoj.com/contest/52/problem/F?problem_id=2426.

Chia mỗi B số liên tiếp thành một khối, tổng cộng $O(n/B)$ khối. Với một truy vấn, ta có thể tách đoạn $[l, r]$ thành $O(n/B)$ đoạn có độ dài không vượt quá B , trong đó chỉ có $O(1)$ đoạn không phải là nguyên một khối. Với đoạn không phải nguyên một khối, có thể tách tiếp thành $O(B)$ đoạn độ dài 1 để xử lý. Với một khối nguyên vẹn, ta tính dãy đã sắp xếp của mọi phần tử trong khối, và tính đáp án của khối khi từng phần tử trong khối được dùng làm x . Phần sau có thể được tính bằng cách sau:

Xét việc xử lý theo thứ tự tăng dần. Đưa mọi số lớn hơn x vào một danh sách liên kết theo thứ tự vị trí xuất hiện; để tiện xử lý, có thể thêm nút đặc biệt ở hai đầu danh sách. Khi x tăng dần, các phần tử trong danh sách sẽ dần bị xóa. Khi tính đáp án của khối, vị trí số đầu tiên lớn hơn x và vị trí số cuối cùng lớn hơn x trong đoạn có thể được lấy trực tiếp qua con trỏ của các nút đặc biệt. Khi tính đáp án thao tác hỏi, cần xét ảnh hưởng của việc xóa một nút lên đáp án: rõ ràng mọi đoạn chứa nút này nhưng không chứa tiền nhiệm và hậu nhiệm của nút này sẽ được cộng vào đáp án, và không có ảnh hưởng nào khác. Bước này có độ phức tạp thời gian $O(B)$, nên không làm tăng độ phức tạp thời gian xây dựng lại khối.

Sau khi tính được phần này, phần còn lại cơ bản giống cách giải bài toán chia khối cổ điển ở mục 2, và có thể được tối ưu bằng phương pháp ở mục 3.2.

Bài toán 2. Cho một dãy độ dài n và n thao tác. Có hai loại thao tác:

1. Thao tác sửa: cho l, r, x ; với mọi $l \leq i \leq r$, tăng a_i thêm x .
2. Thao tác hỏi: cho l, r, x ; hỏi trong mọi đoạn con của đoạn $[l, r]$, tổng các phần tử trong đoạn con lớn nhất là bao nhiêu.

Trước hết trình bày cách làm của lời giải chính thức.

Tương tự bài toán trước, với hai đoạn $[l, mid]$ và $[mid + 1, r]$, nếu ta biết đáp án của chúng, ở đây “đáp án” bao gồm tổng đoạn con lớn nhất trong đoạn, tức đáp án của thao tác hỏi, tổng tiền tố lớn nhất trong đoạn, và tổng hậu tố lớn nhất trong đoạn, thì ta có thể tính đáp án của đoạn $[l, r]$.

Vẫn xét chia mỗi B số liên tiếp thành một khối, tổng cộng $O(n/B)$ khối. Điểm mấu chốt là xử lý trường hợp một khối nguyên vẹn.

Để xử lý một khối nguyên vẹn, ta cần duy trì đáp án khi mọi số trong cả khối được cộng cùng một số. Trước hết xử lý tình huống này.

Xét dùng cây đoạn để duy trì. Với một đỉnh, cần duy trì đáp án của đoạn mà đỉnh đó đại diện khi mọi số trong đoạn được cộng cùng một số x . Không khó chứng minh đây là một hàm tuyến tính từng đoạn theo x , và số đoạn cùng bậc với độ dài của đoạn. Thông tin duy trì ở một đỉnh có thể được gộp từ hai con của nó; phép gộp chỉ cần cộng hai hàm từng đoạn hoặc lấy max, có thể hoàn thành trong $O(len)$, trong đó len là số đoạn của hàm từng đoạn, cùng bậc với độ dài đoạn. Như vậy, độ phức tạp thời gian xây dựng cây đoạn là tổng độ dài các đoạn tương ứng với mọi đỉnh trong cây đoạn, tức $O(B \log B)$.

Nhưng với bài toán ban đầu, làm như vậy sẽ gặp vài vấn đề. Vấn đề thứ nhất là khi thực hiện thao tác sửa, cần xây dựng lại các khối bị phủ không hoàn toàn. Nếu xây thô lại toàn bộ cây đoạn thì độ phức tạp thời gian là $O(B \log B)$, chưa đủ tốt.

Chú ý rằng khi xây dựng lại một khối, thao tác sửa trên khối đó chỉ là cộng đoạn. Theo tính chất của cây đoạn, ở mỗi tầng chỉ có $O(1)$ đỉnh bị phủ không hoàn toàn, còn sửa trên đỉnh được phủ hoàn toàn chỉ là tịnh tiến hàm từng đoạn, có thể dùng thẻ lười để duy trì. Vì vậy, ở mỗi tầng chỉ có $O(1)$ đỉnh cần tính lại thông tin, độ phức tạp thời gian là

$$O\left(B + \frac{B}{2} + \frac{B}{4} + \frac{B}{8} + \dots\right) = O(B).$$

Vấn đề thứ hai là khi hỏi đáp án của một khối, ta cần tìm kiếm nhị phân để biết đáp án nằm ở đoạn nào của hàm từng đoạn. Nếu tìm kiếm nhị phân trực tiếp, độ phức tạp của một thao tác là $O(B + n \log B/B)$,

chưa đủ tốt. Cách trong lời giải chính thức là tối ưu tương tự mục 2.3, tức dùng sắp xếp cơ số rồi tận dụng tính đơn điệu. Độ phức tạp thời gian là $O(n\sqrt{n})$. Do xử lý từng khối, ta có thể chỉ xây dựng cây đoạn của khối đang được xử lý, nên độ phức tạp không gian là $O(B \log B + n) = O(n)$, chứ không phải $O(n \log n)$ như khi xây tất cả cây đoạn của mọi khối.

Thực ra ta có thể dùng phương pháp fractional cascading của thuật toán 4 ở mục 3.2, trong đó dãy được lưu ở các lá là dãy gồm các điểm phân chia của hàm từng đoạn. Với khối được phủ hoàn toàn, thao tác sửa chỉ tịnh tiến hàm từng đoạn, tức cộng cùng một số vào mọi điểm phân chia. Vì vậy nếu đoạn tương ứng với một đỉnh trong cây đoạn được phủ hoàn toàn, mọi dãy mà các đỉnh trong cây con của nó duy trì cũng chỉ bị cộng cùng một số vào mọi phần tử, không làm thay đổi cấu trúc. Do đó cách làm ở mục 3.2 vẫn áp dụng được, với độ phức tạp thời gian $O(n\sqrt{n})$ và độ phức tạp không gian $O(n \log n)$.

3.4 Kết luận

Fractional cascading tuy đã có hơn 20 năm lịch sử và là một thuật toán phổ biến, nhưng hiện vẫn chưa được ứng dụng rộng rãi trong thi tin học. Bài viết này giới thiệu fractional cascading, so sánh thuật toán này với các thuật toán khác, và áp dụng nó vào một số bài toán trong thi tin học, đặc biệt là tối ưu thuật toán khi giải dạng bài chia khối cổ điển này. Hy vọng bài viết có ích cho mọi người.

Tuy nhiên, chưa có nhiều người nghiên cứu lĩnh vực này, và do trình độ tác giả còn hạn chế, vẫn còn nhiều vấn đề chưa được giải quyết. Mong bài viết này có thể gợi mở thêm thảo luận và thu hút nhiều đồng nghiệp hơn nghiên cứu thuật toán này.

Lời cảm ơn

Xin cảm ơn China Computer Federation đã cung cấp nền tảng học tập và trao đổi. Xin cảm ơn huấn luyện viên Gao Wenyuan của đội tuyển quốc gia đã hướng dẫn. Xin cảm ơn các thầy Lin Yang, Zhang Junliang, Hu Fan, Li Zhiwu, và Lin Hong đã bồi dưỡng và chỉ dạy tôi. Xin cảm ơn các anh Cai Chengze, Li Xinlong, v.v. đã cho tôi cảm hứng trong các cuộc thảo luận. Xin cảm ơn các anh Wang Xiuhuan, Wang Siqi, Yuan Fangzhou, v.v. đã chỉ dẫn. Xin cảm ơn toàn thể các bạn lớp tin học khóa 2018 của Trường Trung học số 7 Thành Đô đã đồng hành trong hai năm. Xin cảm ơn cha mẹ đã thấu hiểu và ủng hộ tôi.

Tài liệu tham khảo

1. Chazelle, Bernard; Guibas, Leonidas J. *Fractional cascading: I. A data structuring technique*. 1986.
2. Chazelle, Bernard; Guibas, Leonidas J. *Fractional cascading: II. Applications*. 1986.
3. Wikipedia contributors. *Fractional cascading*. Wikipedia. https://en.wikipedia.org/wiki/Fractional_cascading.
4. Blog CSDN của Baimafujinji: blog.csdn.net/baimafujinji/.../52956605.

4 Bàn về cây thống trị và ứng dụng

Chen Sunli, Trường Ngoại ngữ Nam Kinh

Tóm tắt

Bài viết này chủ yếu giới thiệu thuật toán tìm cây thống trị và các ứng dụng của nó.

Mục 2 giới thiệu một số ký hiệu và định nghĩa. Mục 3 tập trung trình bày thuật toán tìm cây thống trị. Mục 4 giới thiệu một số ứng dụng của cây thống trị. Mục 5 là phần tổng kết.

4.1 Mở đầu

Khái niệm cây thống trị thường xuất hiện trong lý thuyết đồ thị và lý thuyết thiết kế máy tính; với tư cách một thuật toán, nó cũng thỉnh thoảng xuất hiện trong các kỳ thi thuật toán. Tuy nhiên, tài liệu tiếng Trung giới thiệu có hệ thống về nó còn tương đối ít. Tác giả hy vọng bài viết này có thể giúp nhiều bạn học sinh hiểu cây thống trị hơn, cũng như hiểu một vài ứng dụng của nó.

4.2 Định nghĩa

Trong bài viết này ta xét một đồ thị có hướng $G = (V, E)$ gồm n đỉnh và m cạnh, trong đó V là tập đỉnh, E là tập cạnh, và $|V| = n, |E| = m$. Bài viết giả sử người đọc đã quen với các khái niệm cơ bản trong lý thuyết đồ thị như đường đi, chu trình, tìm kiếm theo chiều sâu (DFS), thứ tự topo, thứ tự DFS, v.v.

Khi đã cho một nguồn $s \in V$, nếu với hai đỉnh x, y , mọi đường đi bắt đầu từ s và kết thúc ở y đều đi qua x , thì ta nói x thống trị (*dominates*) y . Vì khi không tồn tại đường đi từ s đến x , các quan hệ thống trị liên quan tới x đều không có ý nghĩa, nên trong bài viết này ta luôn giả sử từ s có thể tới được mọi đỉnh khác. Với đồ thị tổng quát, chỉ cần duyệt hết duyệt sâu từ s và chỉ giữ lại các đỉnh được thăm.

Trước hết quan sát rằng ta chỉ cần xét các đường đi đơn. Lý do là nếu một đường đi xuất hiện đỉnh lặp lại, ta có thể xóa các đỉnh giữa hai lần xuất hiện lặp đó; phép sửa này không ảnh hưởng tới điều kiện thống trị. Một tính chất hiển nhiên là s thống trị mọi đỉnh. Ngoài ra, quan hệ thống trị còn thỏa các tính chất dưới đây, nhờ đó có thể dùng một cấu trúc cây để mô tả đầy đủ chúng.

Tính chất 1. Nếu x thống trị y và y thống trị z , thì x thống trị z . Nói cách khác, quan hệ thống trị có tính bắc cầu.

Tính chất này khá hiển nhiên.

Tính chất 2. Nếu x thống trị y và y thống trị x , thì $x = y$.

Chứng minh. Giả sử $x \neq y$. Khi đó mọi đường đi đơn từ s đến x đều đi qua y , và trước khi đi qua y lại bắt buộc phải đi qua x . Vì vậy đường đi có dạng $s, \dots, x, \dots, y, \dots, x$, mâu thuẫn với tính đơn. Suy ra bắt buộc có $x = y$. $\square$

Tính chất 3. Nếu x, y, z đôi một khác nhau, và cả x lẫn y đều thống trị z , thì hoặc x thống trị y , hoặc y thống trị x .

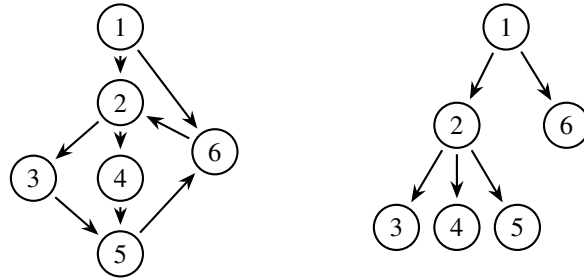
Chứng minh. Xét một đường đi đơn bất kỳ từ s đến z . Đường đi này chắc chắn chứa cả x lẫn y . Không mất tính tổng quát, giả sử đường đi có dạng $s, \dots, x, \dots, y, \dots, z$. Nếu x không thống trị y , thì tồn tại một đường đi từ s đến y không qua x . Khi đó thay phần từ s đến y của đường đi nói trên bằng đường đi không qua x này, ta thu được một đường đi từ s đến z không qua x , mâu thuẫn với việc x thống trị z . $\square$

Các tính chất trên cho thấy: với một đỉnh $x \neq s$, xét tập

$$S = \{y \mid y \text{ thống trị } x\}.$$

Các đỉnh trong S tạo thành một tập được sắp toàn phần theo quan hệ thống trị, và S có ít nhất hai phần tử. Do đó chắc chắn tồn tại một đỉnh z thỏa: nếu $y \neq x$ thống trị x , thì y cũng thống trị z . Ta gọi z là điểm thống trị trực tiếp (*immediate dominator*) của x , ký hiệu $z = \text{idom}(x)$.

Bây giờ, với mọi đỉnh x khác s , nối một cạnh $\text{idom}(x) \rightarrow x$. Đồ thị thu được có bậc vào của mỗi đỉnh không quá 1. Kết hợp Tính chất 1 và Tính chất 2, đồ thị này không có chu trình, vì thế nó chắc chắn là một cây có hướng gốc s , gọi là cây thống trị (*Dominator Tree*), ký hiệu $DT(G, s)$. Nó thỏa: với hai đỉnh x, y , x thống trị y khi và chỉ khi trên cây thống trị x là tổ tiên của y (trong bài viết này, tổ tiên bao gồm chính đỉnh đó). Vì vậy, chỉ cần tìm được cây thống trị, ta tìm được toàn bộ các quan hệ thống trị.



Hình 2: hình bên phải là cây thống trị của hình bên trái khi $s = 1$.

4.3 Thuật toán tìm cây thống trị

4.3.1 Trường hợp đặc biệt: đồ thị có hướng không chu trình

Với đồ thị có hướng không chu trình $G = (V, E)$, do các đỉnh đứng sau trong thứ tự topo không ảnh hưởng tới quan hệ thống trị của các đỉnh đứng trước, ta có thể lần lượt tìm điểm thống trị trực tiếp của từng đỉnh x theo thứ tự topo. Giả sử điểm thống trị trực tiếp của mọi đỉnh đứng trước x trong thứ tự topo đã được tìm. Khi đó ta có một cây thống trị chưa hoàn chỉnh, chắc chắn là một phần của cây thống trị cuối cùng; tạm gọi nó là “cây thống trị hiện tại”. Giả sử các cạnh kết thúc ở x là $(c_1, x), \dots, (c_k, x)$, khi đó $\text{idom}(c_i)$ đều đã được tìm.

Bổ đề 1. Đỉnh y thống trị x khi và chỉ khi $y = x$, hoặc y thống trị tất cả $c_1, \dots, c_k$.

Chứng minh. Khi $y = x$, kết luận hiển nhiên. Sau đây giả sử $y \neq x$. Trước hết chứng minh nếu y thống trị tất cả $c_1, \dots, c_k$ thì y chắc chắn thống trị x : xét một đường đi bất kỳ từ s đến x , đỉnh ngay trước x chắc chắn là một trong $c_1, \dots, c_k$, nên đường đi này bắt buộc đi qua y .

Tiếp theo chứng minh mệnh đề phản đảo của chiều trên: không mất tính tổng quát, giả sử y không thống trị c_1 . Khi đó tồn tại đường đi từ s đến c_1 không qua y ; đi tiếp theo cạnh (c_1, x) , ta thu được một đường đi từ s đến x không qua y . Vì vậy y không thống trị x . $\square$

Các đỉnh thỏa tính chất trên chính là tổ tiên chung của $c_1, \dots, c_k$ trên “cây thống trị hiện tại”. Vì vậy

$$\text{idom}(x) = \text{LCA}(\text{idom}(c_1), \dots, \text{idom}(c_k)),$$

trong đó LCA là tổ tiên chung gần nhất. Điều này tương đương với việc thêm một lá và một cạnh vào “cây thống trị hiện tại”. Nếu dùng thuật toán LCA bằng nhảy bậc hai, chỉ cần sửa nhẹ là có thể hỗ trợ mỗi lần thêm một lá. Độ phức tạp thời gian và không gian của cách làm này đều là $O((n + m) \log n)$.

4.3.2 Định nghĩa và tính chất của điểm bán thống trị

Trong thuật toán sau đây, ta cần xét cây DFS của đồ thị tùy ý G xuất phát từ s , tức cấu trúc cây do các cạnh được đi qua trong quá trình duyệt sâu tạo thành; đồng thời gán thứ tự cho các đỉnh theo thứ tự duyệt

(thứ tự DFS). Cụ thể, với hai đỉnh x, y , nếu x được thăm sớm hơn y trong quá trình duyệt, ta viết $x < y$; tương tự có thể định nghĩa $x > y$, giá trị lớn nhất và nhỏ nhất của một tập đỉnh, v.v. Cây DFS của đồ thị có hướng không tồn tại cạnh (x, y) thỏa $x < y$ và x không phải tổ tiên của y . Một phát biểu hữu ích hơn là:

Định lý 1. Nếu $x \leq y$, thì mọi đường đi từ x đến y bắt buộc đi qua một tổ tiên chung nào đó của x và y trên cây DFS.

Chứng minh. Nếu x là tổ tiên của y , kết luận hiển nhiên, vì vậy có thể coi x không phải tổ tiên của y . Giả sử tồn tại một đường đi

$$x = v_0, v_1, \dots, v_k = y$$

trên đó không có tổ tiên chung nào của x và y trong cây DFS. Gọi z là tổ tiên chung gần nhất của x và y trên cây DFS; trong các con của z , chắc chắn có đúng một đỉnh y' là tổ tiên của y .

Xét đường đi bất kỳ $x = v_0, v_1, \dots, v_k = y$. Đặt

$$p = \max\{i \mid v_i < y'\},$$

và giả sử v_p không phải tổ tiên của z . Khi đó v_p cũng chắc chắn không phải tổ tiên của v_{p+1} . Điều này suy ra từ tính chất của thứ tự DFS: nếu x là tổ tiên của y , thì x cũng là tổ tiên của mọi z thỏa $x \leq z \leq y$. Từ đó có cạnh (v_p, v_{p+1}) , $v_p < v_{p+1}$, và v_p không phải tổ tiên của v_{p+1} . Nhưng cạnh này không thể tồn tại trong cây DFS, nếu không khi duyệt tới v_p ta sẽ đi trực tiếp theo cạnh đó để thăm v_{p+1} . Mâu thuẫn này cho thấy v_p chắc chắn là tổ tiên của z . $\square$

Sau đây là định nghĩa điểm bán thống trị (*semi-dominator*).

Định nghĩa. Điểm bán thống trị của đỉnh x là

$$\text{sdom}(x) = \min\{y \mid \text{tồn tại đường đi } y = v_0, v_1, \dots, v_k = x \text{ sao cho với mọi } 1 \leq i \leq k-1, v_i > x\}.$$

Ta gọi sự tồn tại của đường đi trong định nghĩa này là “điều kiện điểm bán thống trị”. Việc tìm điểm bán thống trị là bước trung gian để tìm điểm thống trị trực tiếp. Dưới đây là một số tính chất quan trọng về điểm thống trị trực tiếp và điểm bán thống trị.

Bổ đề 2. $\text{idom}(x)$ và $\text{sdom}(x)$ đều là tổ tiên của x trên cây DFS và đều khác x . Hơn nữa, trên cây DFS, $\text{idom}(x)$ là tổ tiên của $\text{sdom}(x)$.

Chứng minh. Việc $\text{idom}(x)$ là tổ tiên của x trên cây DFS là hiển nhiên, vì trên cây DFS có một đường đi từ s đến x , và $\text{idom}(x)$ bắt buộc nằm trên đường đi này.

Do cha của x trên cây DFS đã thỏa yêu cầu của điểm bán thống trị, nên $\text{sdom}(x) < x$. Mặt khác, nếu $\text{sdom}(x)$ không phải tổ tiên của x , thì không khó thấy cha của $\text{sdom}(x)$ trên cây DFS cũng thỏa yêu cầu của điểm bán thống trị, và còn nhỏ hơn $\text{sdom}(x)$, mâu thuẫn với định nghĩa điểm bán thống trị.

Theo định nghĩa điểm bán thống trị, ta có thể đi theo cây DFS từ s tới $\text{sdom}(x)$, rồi đi theo đường đi trong định nghĩa tới x . Vì vậy các đỉnh trên đường từ $\text{sdom}(x)$ tới x trong cây DFS (không bao gồm $\text{sdom}(x)$) đều không thống trị x . Do đó $\text{idom}(x)$ chắc chắn là tổ tiên của $\text{sdom}(x)$. $\square$

Bổ đề 3. Cho v, w thỏa v là tổ tiên của w trên cây DFS. Khi đó hoặc v là tổ tiên của $\text{idom}(w)$, hoặc $\text{idom}(w)$ là tổ tiên của $\text{idom}(v)$.

Chứng minh. Nếu $v = w$, kết luận hiển nhiên, nên giả sử $v \neq w$. Chứng minh bằng phản chứng. Vì $v, w, \text{idom}(v), \text{idom}(w)$ đều là tổ tiên của w , nếu kết luận trong bổ đề không đúng, thì theo thứ tự độ sâu chúng lần lượt là

$$\text{idom}(v), \text{idom}(w), v, w$$

và đôi một khác nhau. Điều này có nghĩa là tồn tại một đường đi từ s đến v không qua $\text{idom}(w)$. Đi tiếp theo cây DFS tới w , ta thu được một đường đi từ s đến w không qua $\text{idom}(w)$, mâu thuẫn. $\square$

Với các tính chất trên, ta sẽ chỉ ra mối liên hệ chặt chẽ giữa điểm bán thống trị và điểm thống trị trực tiếp, từ đó có thể dùng cái trước để tính cái sau.

Định lý 2. Lấy một đỉnh $w \neq s$. Xét mọi đỉnh x trên đường từ $\text{sdom}(w)$ tới w trong cây DFS, không bao gồm $\text{sdom}(w)$. Nếu chúng đều thỏa $\text{sdom}(x) \geq \text{sdom}(w)$, thì

$$\text{idom}(w) = \text{sdom}(w).$$

Chứng minh. Theo Bổ đề 2, $\text{idom}(w)$ là tổ tiên của $\text{sdom}(w)$, nên chỉ cần chứng minh $\text{sdom}(w)$ thống trị w . Xét một đường đi P bất kỳ từ s đến w ; ta sẽ chứng minh $\text{sdom}(w)$ chắc chắn nằm trên P . Gọi x là đỉnh cuối cùng trên P thỏa $x < \text{sdom}(w)$. Nếu không tồn tại đỉnh như vậy, thì chắc chắn $\text{sdom}(w) = \text{idom}(w) = s$, kết luận vẫn đúng. Đồng thời, gọi y là đỉnh đầu tiên sau x trên P nằm trên đường từ $\text{sdom}(w)$ tới w trong cây DFS. Lấy đoạn con từ x tới y trên P , được $v_0 = x, v_1, \dots, v_k = y$.

Bây giờ xét đường $v_0, \dots, v_k$. Ta khẳng định với mọi $1 \leq i < k$ đều có $v_i > y$, tức $\text{sdom}(y) \leq x < \text{sdom}(x)$. Nếu không, tồn tại một $v_i < y$. Khi đó theo Định lý 1, tồn tại $i \leq j \leq k-1$ sao cho v_j là tổ tiên của y . Theo cách chọn x , có $\text{sdom}(w) \leq v_j$, vì vậy v_j cũng nằm trên đường từ $\text{sdom}(w)$ tới w trong cây DFS, mâu thuẫn với cách chọn y . Mâu thuẫn trên suy ra $\text{sdom}(y) \leq x < \text{sdom}(x)$. Kết hợp với điều kiện của định lý, bắt buộc có $y = \text{sdom}(w)$, tức đường đi P chứa $\text{sdom}(w)$. $\square$

Định lý 3. Lấy một đỉnh $w \neq s$. Xét mọi đỉnh trên đường từ $\text{sdom}(w)$ tới w trong cây DFS, không bao gồm $\text{sdom}(w)$. Gọi u là đỉnh có giá trị sdom nhỏ nhất trong số đó. Khi đó $\text{sdom}(u) \leq \text{sdom}(w)$ và

$$\text{idom}(u) = \text{idom}(w).$$

Chứng minh. Vì w cũng là một ứng viên của u , nên $\text{sdom}(u) \leq \text{sdom}(w)$ là hiển nhiên. Chú ý rằng trên cây DFS, $\text{idom}(w)$ là tổ tiên của u . Theo Bổ đề 3, $\text{idom}(w)$ cũng là tổ tiên của $\text{idom}(u)$, nên chỉ cần chứng minh $\text{idom}(u)$ thống trị w .

Xét một đường đi P bất kỳ từ s đến w ; ta sẽ chứng minh $\text{idom}(u)$ chắc chắn nằm trên P . Gọi x là đỉnh cuối cùng trên P thỏa $x < \text{idom}(u)$. Nếu không tồn tại đỉnh như vậy, thì chắc chắn $\text{idom}(u) = \text{idom}(w) = s$, kết luận vẫn đúng. Đồng thời, gọi y là đỉnh đầu tiên sau x trên P nằm trên đường từ $\text{idom}(u)$ tới w trong cây DFS. Lấy đoạn con từ x tới y trên P , được $v_0 = x, v_1, \dots, v_k = y$.

Tương tự chứng minh Định lý 2, ta có $\text{sdom}(y) \leq x$. Theo Bổ đề 2, $\text{idom}(u) \leq \text{sdom}(u)$. Kết hợp lại:

$$\text{sdom}(y) \leq x < \text{idom}(u) \leq \text{sdom}(u).$$

Đến đây, từ cách chọn u , ta biết y không thể là hậu duệ thật sự của $\text{sdom}(w)$. Mặt khác, y cũng không thể vừa là hậu duệ thật sự của $\text{idom}(u)$, vừa là tổ tiên của u ; nếu không, đi theo cây DFS từ s tới $\text{sdom}(y)$, rồi đi theo P tới y , cuối cùng đi theo cây DFS tới u , ta thu được một đường đi không qua $\text{idom}(u)$, mâu thuẫn với định nghĩa điểm thống trị.

Theo $\text{idom}(u) \leq \text{sdom}(w) < u \leq w$ và $\text{idom}(u) < y$, khả năng duy nhất còn lại là $y = \text{idom}(u)$. Vậy đường đi P chứa $\text{idom}(u)$. $\square$

Hai định lý trên giải thích quan hệ giữa điểm bán thống trị và điểm thống trị trực tiếp, đồng thời cho một cách tính điểm thống trị trực tiếp từ điểm bán thống trị. Với đỉnh $w \neq s$, xét mọi đỉnh trên đường từ $\text{sdom}(w)$ tới w trong cây DFS, không bao gồm $\text{sdom}(w)$. Gọi u là đỉnh có sdom nhỏ nhất trong số đó. Nếu $\text{sdom}(u) = \text{sdom}(w)$, dùng Định lý 2 ta có $\text{idom}(w) = \text{sdom}(w)$; ngược lại, $u \neq w$, dùng Định lý 3 ta có $\text{idom}(w) = \text{idom}(u)$.

4.3.3 Tính điểm bán thống trị và điểm thống trị trực tiếp

Điểm bán thống trị. Để tìm điểm bán thống trị, ta cần định lý dưới đây.

Định lý 4. $\text{sdom}(w)$ bằng giá trị nhỏ nhất trong các ứng viên sinh ra từ hai trường hợp sau:

1. Có cạnh (v, w) ; khi đó v là một ứng viên của $\text{sdom}(w)$.
2. u là tổ tiên của v trên cây DFS, $u > w$, và có cạnh (v, w) ; khi đó $\text{sdom}(u)$ là một ứng viên của $\text{sdom}(w)$.

Chứng minh. Gọi x là giá trị nhỏ nhất trong mọi ứng viên. Trước hết ta chứng minh mỗi ứng viên đều thỏa điều kiện điểm bán thống trị, từ đó suy ra $\text{sdom}(w) \leq x$. Nếu x đến từ trường hợp 1, rõ ràng (x, w) chính là đường đi thỏa điều kiện điểm bán thống trị. Ngược lại, $x = \text{sdom}(u)$. Khi đó lấy đường đi từ $\text{sdom}(u)$ đến u được nêu trong điều kiện điểm bán thống trị, nối tiếp đường đi từ u đến v trong cây DFS, rồi cuối cùng đi từ v đến w , ta được một đường đi thỏa điều kiện điểm bán thống trị của w .

Còn cần chứng minh $\text{sdom}(w) \geq x$. Xét đường đi ứng với điều kiện điểm bán thống trị từ $\text{sdom}(w)$ đến w :

$$v_0 = \text{sdom}(w), v_1, \dots, v_k = w,$$

trong đó với mọi $1 \leq i < k$ đều có $v_i \geq w$. Nếu $k = 1$, nó đã được xét trong trường hợp 1; sau đây giả sử $k > 1$. Lấy j là chỉ số nhỏ nhất thỏa $j \geq 1$ và v_j là tổ tiên của v_{k-1} trên cây DFS. Chỉ số như vậy chắc chắn tồn tại, vì k là một chỉ số hợp lệ.

Bây giờ xét đường $v_0, \dots, v_j$. Ta khẳng định nó là đường thỏa điều kiện điểm bán thống trị của v_j , tức với mọi $1 \leq i < j$ đều có $v_i > v_j$. Nếu không, xét mọi chỉ số i thỏa $v_i \leq v_j$, lấy chỉ số có v_i nhỏ nhất. Theo Định lý 1, v_i chắc chắn là tổ tiên của v_j , mâu thuẫn với cách chọn j . Vì vậy $\text{sdom}(v_j) \leq \text{sdom}(w)$; mà $\text{sdom}(v_j)$ chắc chắn được xét trong trường hợp 2, nên $x \leq \text{sdom}(w)$. $\square$

Với Định lý 4, có thể thiết kế thuật toán tìm điểm bán thống trị như sau:

1. Tìm một cây DFS của G và thứ tự DFS.
2. Nếu điểm sdom của mọi đỉnh đều đã được tìm, kết thúc thuật toán; ngược lại, tìm đỉnh lớn nhất w trong các đỉnh chưa có $\text{sdom}(w)$.
3. Xét trường hợp 1, có thể tính trực tiếp mọi ứng viên.
4. Xét trường hợp 2: duyệt các cạnh (v, w) thỏa $v > w$. Khi đó v chắc chắn đã được đánh dấu. Bắt đầu từ v , liên tục đi lên cha đã được đánh dấu trên cây DFS để tạo thành một đường đi, rồi lấy giá trị sdom nhỏ nhất trên đường này làm ứng viên.
5. Gộp các ứng viên thu được từ trường hợp 1 và 2 để nhận giá trị $\text{sdom}(w)$, đánh dấu w là đã tìm được sdom , rồi quay lại bước 2.

Có thể thấy việc xét mọi đỉnh theo thứ tự từ lớn xuống nhỏ chính là để các giá trị sdom cần dùng trong bước 4 đều đã được tìm. Dễ thấy các phần ngoài bước 4 có tổng độ phức tạp không vượt quá $O(n + m)$.

Để tối ưu độ phức tạp, ở bước 4 ta dùng cấu trúc dữ liệu DSU có trọng số để duy trì giá trị $sdom$ nhỏ nhất trên đường đi và vị trí đạt giá trị nhỏ nhất đó. Khi đánh dấu x , nối mọi con của x trên cây DFS tới x và duy trì lại thông tin nhỏ nhất. Nếu cài đặt DSU với nén đường đi một cách mộc mạc, độ phức tạp thời gian đạt $O((n + m) \log n)$, độ phức tạp không gian là $O(n + m)$. Với bài toán này còn có các cách cài đặt DSU nhanh hơn, đạt độ phức tạp thấp hơn là $O((n + m)\alpha(n))$, thậm chí tuyến tính; bạn đọc quan tâm có thể tham khảo tài liệu liên quan.

Điểm thống trị trực tiếp. Để tìm mọi idom, định nghĩa $pivot(w)$ là đỉnh có giá trị $sdom$ nhỏ nhất trên đường từ $sdom(w)$ tới w trong cây DFS, không bao gồm $sdom(w)$. Nếu đã tìm được mọi $sdom(w)$ và $pivot(w)$, ta có thể lần lượt xác định idom của từng đỉnh theo thứ tự từ nhỏ tới lớn dựa trên Bổ đề 2 và Bổ đề 3. Cụ thể: nếu

$$sdom(pivot(w)) = sdom(w),$$

thì $idom(w) = sdom(w)$; ngược lại

$$idom(w) = idom(pivot(w)).$$

Ở đây, do $pivot(w) \leq w$, trước khi xét tới w thì $idom(pivot(w))$ chắc chắn đã được tìm.

Cuối cùng, chỉ cần sửa nhẹ thuật toán tìm điểm bán thống trị ở trên là có thể tìm được $pivot$ của mỗi đỉnh. Với mỗi đỉnh x , mở một $bucket(x)$ để lưu mọi đỉnh w thỏa $sdom(w) = x$. Nếu hiện đang tìm $sdom$ của đỉnh w , thì trước khi thực hiện thao tác đánh dấu ở bước 5, tức thao tác *link* của DSU, xét mọi đỉnh x trong $bucket(w)$ và truy vấn trực tiếp trong DSU để lấy giá trị $pivot(x)$. Sau khi tìm được $sdom(w)$, cũng đừng quên thêm w vào $bucket(sdom(w))$.

Kết hợp những điều trên, ta đã có thuật toán cây thống trị hoàn chỉnh.

4.3.4 Tổng kết thuật toán

Dưới đây là phần tổng hợp và tóm lược thuật toán:

1. Tìm một cây DFS của G và thứ tự DFS.
2. Nếu điểm $sdom$ của mọi đỉnh đều đã được tìm, chuyển tới bước 8; ngược lại, tìm đỉnh lớn nhất w trong các đỉnh chưa có $sdom(w)$.
3. Xét trường hợp 1, có thể tính trực tiếp mọi ứng viên.
4. Xét trường hợp 2: duyệt các cạnh (v, w) thỏa $v > w$. Khi đó v chắc chắn đã được đánh dấu. Bắt đầu từ v , liên tục đi lên cha đã được đánh dấu trên cây DFS để tạo thành một đường đi, rồi lấy giá trị $sdom$ nhỏ nhất trên đường này làm ứng viên. Đường này chính là đường từ đỉnh v tới gốc của v trong DSU hiện tại, nên có thể truy vấn trực tiếp trong DSU.
5. Gộp các ứng viên thu được từ trường hợp 1 và 2 để nhận giá trị $sdom(w)$, rồi thêm w vào $bucket(sdom(w))$.
6. Với mỗi đỉnh x trong $bucket(w)$, tìm $pivot(x)$. Giá trị này cũng có thể lấy trực tiếp bằng cách truy vấn giá trị nhỏ nhất trên đường từ x tới gốc của x trong DSU.
7. Với mỗi con c của w trên cây DFS, đặt cha của c trong DSU là w , đồng thời duy trì thông tin giá trị nhỏ nhất trên đường.
8. Xét lần lượt từng đỉnh w theo thứ tự từ nhỏ tới lớn. Nếu $sdom(pivot(w)) = sdom(w)$, đặt $idom(w) = sdom(w)$; ngược lại đặt $idom(w) = idom(pivot(w))$.

Thuật toán này có độ phức tạp không gian $O(n + m)$. Nút thắt thời gian nằm ở việc duy trì DSU; nếu dùng nén đường đi mộc mạc, độ phức tạp là $O((n + m) \log n)$. Trong phạm vi thi tin học, độ phức tạp này đã đủ tốt.

4.4 Ứng dụng của cây thống trị

4.4.1 Điểm thống trị của một tập đỉnh

Điểm thống trị của tập đỉnh S được định nghĩa là đỉnh mà mọi đường đi từ s tới bất kỳ $x \in S$ nào cũng bắt buộc đi qua. Không khó thấy các đỉnh như vậy chính là giao của các tập điểm thống trị của mọi đỉnh trong S . Mà điểm thống trị của đỉnh x chính là mọi tổ tiên của x trên cây thống trị, nên điểm thống trị của tập S chính là tổ tiên chung gần nhất (LCA) của mọi $x \in S$ trên cây thống trị.

4.4.2 Cạnh thống trị

Tương tự định nghĩa điểm thống trị, nếu mọi đường đi từ s tới x đều đi qua cạnh (u, v) , thì ta nói (u, v) thống trị x . Đồng thời cũng có thể định nghĩa cạnh thống trị trực tiếp một cách tương tự: nếu (u, v) thống trị x , và mọi cạnh thống trị của x cũng thống trị u , thì gọi (u, v) là cạnh thống trị trực tiếp của x .

Để tìm cạnh thống trị của mỗi đỉnh, một cách đơn giản là với mỗi cạnh (u, v) trong đồ thị gốc, tạo một đỉnh phụ w , nối (u, w) và (w, v) , rồi tìm cây thống trị của đồ thị mới. Với một đỉnh x của đồ thị gốc, các cạnh thống trị của nó chính là các tổ tiên là đỉnh phụ của x trong cây thống trị của đồ thị mới; cạnh thống trị trực tiếp là đỉnh có độ sâu lớn nhất trong số đó, có thể tìm đơn giản bằng DFS. Tuy nhiên, đồ thị mới có $n + m$ đỉnh và $2m$ cạnh, có thể gây hăng số khá lớn. Xem kỹ cách làm này, ta thấy các đỉnh phụ có nhiều tính chất trong quá trình chạy thuật toán, nhờ đó không nhất thiết phải chạy trọn vẹn thuật toán trên đồ thị mới mới suy ra được cạnh thống trị.

Bổ đề 4. Cạnh (u, v) là cạnh thống trị của đỉnh x khi và chỉ khi u và v đều là điểm thống trị của x , đồng thời chỉ có đúng một đường đi đơn từ u tới v , tức cạnh (u, v) là cạnh thống trị của v .

Chứng minh. Nếu cạnh (u, v) là cạnh thống trị của x , thì mọi đường đi từ s tới x đều đi qua u và v , tức u và v thống trị x . Hơn nữa, nếu từ u tới v còn một đường đi khác ngoài cạnh (u, v) , ta có thể đi theo một đường đi bất kỳ từ s tới u , rồi tới v , rồi tới x mà không qua (u, v) . Vì vậy từ u tới v bắt buộc có đúng một đường đi đơn.

Ngược lại, nếu các điều kiện của bổ đề đúng, thì hiển nhiên cạnh (u, v) sẽ thống trị đỉnh x . $\square$

Từ Bổ đề 4, với một cạnh (u, v) trên cây DFS, nếu u thống trị v và từ u tới v chỉ có đúng một đường đi đơn, thì (u, v) thống trị mọi đỉnh mà v thống trị. Nhìn lại thuật toán cây thống trị, nếu cạnh (u, v) nằm trên cây DFS và $\text{idom}(v) = u$, thì chắc chắn có $\text{sdom}(v) = u$.

Bây giờ tập trung vào điều kiện “từ u tới v chỉ có đúng một đường đi đơn”. Có thể hiểu nó qua cách tạo đỉnh phụ ở trên. Giả sử chỉ tạo đỉnh phụ w cho riêng cạnh này, khi đó $\text{sdom}(w) = u$ và $\text{sdom}(v) \geq u$. Lúc này $\text{idom}(v) = w$ khi và chỉ khi $\text{sdom}(v) = w$. Quay lại đồ thị gốc, điều này tương đương với: khi tính $\text{sdom}(v)$, không xét cạnh (u, v) , tức khi chỉ xét trường hợp 2 thì không có ứng viên nào. Như vậy, chỉ cần thêm một kiểm tra trong quá trình tìm sdom , ta có thể xác định liệu từ u tới v có chỉ một đường đi đơn hay không.

Cuối cùng, ta tìm được mọi cạnh (u, v) thỏa nó thống trị v , rồi duyệt một lần trên cây DFS là xác định được cạnh thống trị trực tiếp của mỗi đỉnh. Cách làm này không cần tạo đỉnh phụ, hằng số nhỏ hơn và độ khó cài đặt không tăng rõ rệt.

4.4.3 Điểm khớp và cầu trong đồ thị có hướng

Trong đồ thị có hướng G , u và v liên thông mạnh nếu u tới được v và v cũng tới được u . Quan hệ liên thông mạnh là quan hệ tương đương; các lớp tương đương mà nó tạo ra được gọi là thành phần liên thông mạnh, tương ứng với thành phần liên thông trong đồ thị vô hướng. Với đồ thị G , ký hiệu $G \setminus \{u\}$ là đồ thị mới thu được sau khi xóa đỉnh u và các cạnh liên quan; $G \setminus \{(u, v)\}$ là đồ thị mới thu được sau khi xóa cạnh (u, v) .

Điểm khớp và cầu của đồ thị vô hướng là các đỉnh và cạnh mà sau khi xóa, số thành phần liên thông tăng lên. Tương ứng với chúng, điểm khớp mạnh và cầu mạnh trong đồ thị có hướng là các đỉnh và cạnh mà sau khi xóa, số thành phần liên thông mạnh tăng lên. Một câu hỏi tự nhiên là làm thế nào để tìm tất cả điểm khớp mạnh và cầu mạnh trong một đồ thị G . Giống như đồ thị vô hướng, khi tìm điểm khớp mạnh và cầu mạnh, có thể xét riêng từng thành phần liên thông mạnh, nên phần thảo luận dưới đây đều dành cho đồ thị liên thông mạnh.

Bổ đề 5.1. Đỉnh v là điểm khớp mạnh khi và chỉ khi với mọi $x \neq v$, đều tồn tại một đỉnh $y \neq v$ sao cho một trong hai điều sau đúng:

1. Mọi đường đi từ x tới y đều đi qua v .
2. Mọi đường đi từ y tới x đều đi qua v .

Bổ đề 5.2. Cạnh (u, v) là cầu mạnh khi và chỉ khi với mọi x , đều tồn tại một đỉnh y sao cho một trong hai điều sau đúng:

1. Mọi đường đi từ x tới y đều đi qua cạnh (u, v) .
2. Mọi đường đi từ y tới x đều đi qua cạnh (u, v) .

Chứng minh. Chứng minh của hai bổ đề gần như giống nhau; ở đây chỉ chứng minh Bổ đề 5.1. Nếu v là điểm khớp mạnh, thì $G \setminus \{v\}$ không phải đồ thị liên thông mạnh. Khi đó xét một đỉnh y không cùng thành phần liên thông mạnh với x : hoặc không tồn tại đường đi từ x tới y , hoặc không tồn tại đường đi từ y tới x . Do G là liên thông mạnh, suy ra điều kiện của bổ đề đúng. Mặt khác, nếu điều kiện của bổ đề đúng, thì x và y không còn liên thông mạnh sau khi xóa v . $\square$

Nếu dùng Bổ đề 5.1, chọn tùy ý một đỉnh v là có thể tìm mọi điểm khớp mạnh khác v . Với điều kiện 1, chỉ cần tìm $DT(G, v)$; mọi đỉnh không phải lá, trừ v , trong cây thống trị đều là điểm khớp mạnh. Với điều kiện 2, chỉ cần xây đồ thị đảo G^R của G rồi tìm $DT(G^R, v)$. Cuối cùng, dùng thuật toán tìm thành phần liên thông mạnh thông thường là có thể xác định liệu đỉnh v có phải điểm khớp mạnh hay không.

Với cầu mạnh cũng có kết luận tương tự, có thể áp dụng thuật toán cạnh thống trị ở mục trước để giải quyết. Các thuật toán tìm điểm khớp mạnh và cầu mạnh nói trên có độ phức tạp thời gian bằng với độ phức tạp tìm cây thống trị.

4.4.4 Ứng dụng trong công nghiệp

Cây thống trị có nhiều công dụng trong các cấu trúc cần dựa trên đồ thị có hướng.

Mạng nơ-ron có thể được trừu tượng hóa thành một đồ thị có hướng: nhiệm vụ tính toán ở mỗi đỉnh phụ thuộc vào kết quả của các đỉnh có cạnh trỏ tới nó. Để tăng tốc việc chạy mạng nơ-ron, người ta thường dùng tính toán song song để tối ưu thứ tự tính các đỉnh. Khi đó cây thống trị có thể giúp tìm một phần các đỉnh được sử dụng với tần suất cao và ưu tiên tính chúng.

Ngoài ra, cây thống trị còn có thể phát huy tác dụng trong nguyên lý biên dịch và phát hiện rò rỉ bộ nhớ. Nhìn chung, khái niệm “thống trị” có các cách nói tương tự trong nhiều lĩnh vực, nên cây thống trị, với tư cách công cụ tìm quan hệ thống trị, có phạm vi ứng dụng rộng.

4.5 Tổng kết

Bài viết này giới thiệu thuật toán tìm cây thống trị và một số ứng dụng. Dù bài toán cây thống trị đã được giải quyết, từ góc độ ứng dụng có thể thấy cây thống trị có nhiều biến thể và vẫn còn nhiều điều đáng nghiên cứu.

Hy vọng bài viết này giúp nhiều bạn học sinh hiểu thuật toán và công cụ cây thống trị hơn, đồng thời đi sâu khám phá các vấn đề liên quan.

Lời cảm ơn

Xin cảm ơn China Computer Federation đã cung cấp nền tảng học tập và trao đổi. Xin cảm ơn huấn luyện viên Gao Wenyan của đội tuyển quốc gia đã hướng dẫn. Xin cảm ơn thầy Li Shu của Trường Ngoại ngữ Nam Kinh đã hướng dẫn và ủng hộ. Xin cảm ơn tất cả thầy cô và bạn học từng giúp đỡ tôi. Xin cảm ơn cha mẹ đã luôn hết lòng ủng hộ và khích lệ tôi.

Tài liệu tham khảo

1. Thomas Lengauer, Robert Endre Tarjan. *A Fast Algorithm for Finding Dominators in a Flowgraph*.
2. Adam L. Buchsbaum, Haim Kaplan, Anne Rogers, Jeffery R. Westbrook. *A New, Simpler Linear-Time Dominators Algorithm*.
3. Giuseppe F. Italiano, Luigi Laura, Federico Santaroni. *Finding Strong Bridges and Strong Articulation Points in Linear Time*.
4. Robert E. Tarjan. *Depth-First Search and Linear Graph Algorithms*.

5 Báo cáo ra đề Ghép số

Jiang Xunchi, Trường Trung học Xuejun Hàng Châu

Tóm tắt

Bài viết này giới thiệu bài *Ghép số* do tác giả ra cho một buổi luyện tập nội bộ của trường. Quá trình giải bắt đầu từ một thuật toán đơn giản, sau đó liên tục quan sát tính chất và tối ưu độ phức tạp, cuối cùng thu được một cách giải khéo léo.

5.1 Mô tả bài toán

5.1.1 Ý chính bài toán

Định nghĩa $f(i)$ là xâu thu được khi viết dạng nhị phân của i ; chẳng hạn $f(3) = "11"$, $f(6) = "110"$.

Cần chọn một hoán vị P của các số từ 1 tới n sao cho xâu

$$S = f(P_1) + f(P_2) + \dots + f(P_n)$$

có thứ tự từ điển nhỏ nhất có thể.

Hãy xuất số lượng ký tự 1 trong k vị trí đầu của S .

Vì một vài lý do, dữ liệu vào và ra đều dùng dạng nhị phân.

5.1.2 Quy mô dữ liệu

Với mọi dữ liệu, $1 \leq n < 2^{2000}$, $1 \leq k \leq |S|$.

- Subtask 1 (7 điểm): $1 \leq n \leq 8$;
- Subtask 2 (16 điểm): $1 \leq n \leq 10^5$;
- Subtask 3 (26 điểm): $1 \leq n \leq 2^{60}$;
- Subtask 4 (23 điểm): $1 \leq n \leq 2^{300}$;
- Subtask 5 (28 điểm): không có ràng buộc đặc biệt.

Giới hạn thời gian: 1s.

Giới hạn bộ nhớ: 512MB.

5.1.3 Định dạng đầu vào

Đọc dữ liệu từ đầu vào chuẩn.

Dòng đầu tiên là một xâu 01 không có số 0 đứng đầu, cho n trong dạng nhị phân.

Dòng thứ hai là một xâu 01 không có số 0 đứng đầu, cho k trong dạng nhị phân.

5.1.4 Định dạng đầu ra

Ghi ra đầu ra chuẩn.

Gồm một dòng, là một xâu 01 không có số 0 đứng đầu, cho đáp án của bài toán trong dạng nhị phân.

5.1.5 Ví dụ đầu vào

1011
10010

5.1.6 Ví dụ đầu ra

1000

5.1.7 Giải thích ví dụ

Theo đề bài, một hoán vị có thể là (mọi số dưới đây đều được viết trong dạng nhị phân):

1000, 100, 1001, 10, 1010, 101, 1011, 110, 1, 11, 111.

Ghép các biểu diễn nhị phân này nối tiếp nhau thành xâu S , dễ thấy đáp án là 8.

5.2 Ký hiệu và quy ước

Với một chuỗi S , định nghĩa $S[i]$ là ký tự thứ i của S .

Định nghĩa $S[l \dots r]$ là chuỗi tạo bởi các ký tự từ vị trí thứ l tới vị trí thứ r của S . Đặc biệt, nếu $l > r$, thì $S[l \dots r] = \emptyset$.

Định nghĩa $|S|$ là độ dài của S .

Định nghĩa $h(S)$ là số nhị phân thu được khi viết chuỗi $01 S$ từ trái sang phải.

Định nghĩa $|f(n)| = N$.

Định nghĩa $x \times S$ là chuỗi thu được sau khi lặp chuỗi S đúng x lần.

Định nghĩa S^∞ là chuỗi vô hạn thu được bằng cách lặp vô hạn chuỗi S .

Định nghĩa thứ tự từ điển như sau: nếu $|X| < |Y|$ và $X = Y[1 \dots |X|]$, hoặc tồn tại một $i \leq |X|$ thỏa

$$X[1 \dots i - 1] = Y[1 \dots i - 1] \quad \text{và} \quad X[i] < Y[i],$$

thì thứ tự từ điển của X nhỏ hơn Y .

5.3 Phân tích ban đầu

5.3.1 Thuật toán một

Liệt kê mọi hoán vị hợp lệ P , rồi tìm chuỗi có thứ tự từ điển nhỏ nhất trong các chuỗi được tạo ra.

Độ phức tạp là $O(n! \times n \log n)$.

Điểm kỳ vọng: 7 điểm.

5.3.2 Thuật toán hai

Bổ đề 1. Với hai chuỗi X và Y , nếu $X^\infty = Y^\infty$, thì $X + Y = Y + X$.

Chứng minh. Tìm chu kỳ nhỏ nhất s của X^∞ , khi đó có

$$X = \frac{|X|}{|s|} \times s, \quad Y = \frac{|Y|}{|s|} \times s.$$

Do đó

$$X + Y = Y + X = \frac{|X| + |Y|}{|s|} \times s.$$

□

Bổ đề 2. Với hai chuỗi X và Y , nếu chọn một $1 \leq k \leq |X| + |Y|$ thỏa

$$X^\infty[1 \dots k - 1] = Y^\infty[1 \dots k - 1],$$

thì

$$(X + Y)[k] = X^\infty[k],$$

$$(Y + X)[k] = Y^\infty[k].$$

Chứng minh. Không mất tính tổng quát, giả sử $|X| < |Y|$.

Nếu $k \leq |X|$, thì

$$(X + Y)[k] = X[k] = (X^\infty)[k],$$

$$(Y + X)[k] = Y[k] = (Y^\infty)[k].$$

Nếu $|X| < k \leq |Y|$, thì

$$(X + Y)[k] = Y[k - |X|] = (X^\infty)[k - |X|] = (X^\infty)[k],$$

$$(Y + X)[k] = Y[k] = (Y^\infty)[k].$$

Nếu $|Y| < k$, thì

$$(X + Y)[k] = Y[k - |X|] = (X^\infty)[k - |X|] = (X^\infty)[k],$$

$$(Y + X)[k] = X[k - |Y|] = (Y^\infty)[k - |Y|] = (Y^\infty)[k].$$

□

Bổ đề 3. Với hai xâu X và Y , nếu $X^\infty < Y^\infty$, thì

$$X + Y < Y + X.$$

Chứng minh. Giả sử vị trí khác nhau đầu tiên của X^∞ và Y^∞ là k . Theo Bổ đề 2, ta có

$$(X + Y)[1 \dots k] = (X^\infty)[1 \dots k],$$

$$(Y + X)[1 \dots k] = (Y^\infty)[1 \dots k].$$

Suy ra

$$(X + Y)[1 \dots k] < (Y + X)[1 \dots k],$$

tức $X + Y < Y + X$. □

Định lý 1. Cho n xâu $A_1, A_2, \dots, A_n$. Mọi hoán vị P làm cho

$$A_{P_1} + A_{P_2} + \dots + A_{P_n}$$

có thứ tự từ điển nhỏ nhất đều thỏa điều kiện sau: với mọi $1 \leq i < n$,

$$(A_{P_i})^\infty \leq (A_{P_{i+1}})^\infty.$$

Chứng minh. Nếu tồn tại một i thỏa

$$(A_{P_i})^\infty > (A_{P_{i+1}})^\infty,$$

thì theo Bổ đề 3,

$$A_{P_i} + A_{P_{i+1}} > A_{P_{i+1}} + A_{P_i}.$$

Vậy đổi chỗ P_i và P_{i+1} sẽ tốt hơn.

Còn với một cặp thỏa

$$(A_{P_i})^\infty = (A_{P_{i+1}})^\infty,$$

theo Bổ đề 1 ta có

$$A_{P_i} + A_{P_{i+1}} = A_{P_{i+1}} + A_{P_i}.$$

Do đó đổi chỗ hai xâu có $(A_{P_i})^\infty$ và $(A_{P_{i+1}})^\infty$ bằng nhau không ảnh hưởng tới xâu thu được; tức các xâu có giá trị A^∞ bằng nhau có thể được xếp theo thứ tự bất kỳ mà vẫn tối ưu. $\square$

Với $f(1), f(2), \dots, f(n)$, chỉ cần sắp xếp chúng theo thứ tự tăng dần của $f(i)^\infty$, rồi ghép lại, là thu được xâu S .

Độ phức tạp là $O(n \log^2 n)$.

Điểm kỳ vọng: 23 điểm.

5.4 Phân tích lời giải

Sắp xếp mọi số từ 1 tới n theo thứ tự tăng dần của $f(i)^\infty$; khi $f(i)^\infty$ bằng nhau thì sắp theo i tăng dần. Như vậy có thể xác định duy nhất một hoán vị P .

Định nghĩa $Q = P^{-1}$, tức $Q_{P_i} = i$.

Định nghĩa c là số nguyên duy nhất thỏa

$$\sum_{i=1}^{Q_c-1} |f(P_i)| \leq k,$$

$$\sum_{i=1}^{Q_c} |f(P_i)| > k.$$

Dễ thấy $f(c)$ chỉ giữ lại một tiền tố trong $S[1 \dots k]$, còn $f(P_1), f(P_2), \dots, f(P_{Q_c-1})$ xuất hiện đầy đủ trong $S[1 \dots k]$.

Lời giải tham khảo được chia thành hai phần. Phần thứ nhất là tìm giá trị c ; phần thứ hai là từ giá trị c tính ra đáp án. Vì phần thứ hai đơn giản hơn, dưới đây sẽ giới thiệu phần thứ hai trước.

5.5 Phần thứ hai

5.5.1 Tiền xử lý

Bổ đề 4. Nếu $i < j$ và $|f(i)| = |f(j)|$, thì $Q_i < Q_j$.

Chứng minh. Nếu $i < j$, $|f(i)| = |f(j)|$, thì $f(i)^\infty < f(j)^\infty$, tức vị trí của i đứng trước vị trí của j . Suy ra $Q_i < Q_j$. $\square$

Từ Bổ đề 4, mọi số có cùng độ dài đều xuất hiện trong P theo thứ tự tăng dần.

Định nghĩa

$$g(i, j) = \max\{k \mid |f(k)| = i, Q_k < Q_j\},$$

tức số có độ dài i đứng muộn nhất trong các số đứng trước j . Sau đây sẽ trình bày thuật toán tìm

$$g(1, c), g(2, c), \dots, g(N, c).$$

Định lý 2. Với hai xâu X và Y , nếu $X^\infty \neq Y^\infty$, gọi k là vị trí khác nhau đầu tiên của X^∞ và Y^∞ , thì

$$k \leq 2 \max(|X|, |Y|).$$

Chứng minh. Dưới đây sẽ chứng minh rằng nếu $2 \max(|X|, |Y|)$ ký tự đầu của X^∞ và Y^∞ bằng nhau, thì $X^\infty = Y^\infty$.

Nếu $\gcd(|X|, |Y|) = 1$, xét $|Y|$ ký tự đầu của X^∞ và Y^∞ , ta có

$$X[(i-1) \bmod |X| + 1] = Y[i].$$

Xét từ vị trí $|Y| + 1$ tới $2|Y|$ của X^∞ và Y^∞ , ta có

$$X[(i + |Y| - 1) \bmod |X| + 1] = Y[i].$$

Suy ra

$$X[i] = X[(i + |Y| - 1) \bmod |X| + 1].$$

Vì $\gcd(|X|, |Y|) = 1$, mọi ký tự của X đều bằng nhau. Từ $X[(i-1) \bmod |X| + 1] = Y[i]$ lại suy ra mọi ký tự của Y cũng đều bằng nhau, nên $X^\infty = Y^\infty$.

Nếu $\gcd(|X|, |Y|) = d$, lấy riêng các vị trí đồng dư modulo d trong X và Y để xét; dùng cách tương tự như trên có thể chứng minh chúng đều bằng nhau. Như vậy cũng suy ra $X^\infty = Y^\infty$. $\square$

Định nghĩa $T = f(c)^\infty[1 \dots 2N]$. Khi đó $g(i, c)$ có thể là $h(T[1 \dots i])$ hoặc $h(T[1 \dots i]) - 1$. Chỉ cần so sánh thứ tự từ điển của

$$T[1 \dots i]^\infty[1 \dots 2N]$$

hoặc

$$f(h(T[1 \dots i]) - 1)^\infty[1 \dots 2N]$$

với T , là có thể nhận được đáp án trong độ phức tạp $O(N)$.

Chú ý rằng với $i = N$, nếu giá trị $g(i, c)$ tìm được lớn hơn n , cần gán $g(i, c) = n$.

Độ phức tạp để tìm mọi $g(i, c)$ là $O(N^2)$.

5.5.2 Thống kê đóng góp của $f(c)$

Độ dài tiền tố bị lấy của $f(c)$ là

$$k - \sum_{i=1}^N i \times (g(i, c) - 2^{i-1} + 1).$$

Dùng tính toán số lớn cho biểu thức trên sẽ thu được độ dài l của phần bị lấy từ $f(c)$; sau đó chỉ cần tìm trong $f(c)[1 \dots l]$ có bao nhiêu ký tự 1.

5.5.3 Thống kê đóng góp của $f(P_1), f(P_2), \dots, f(P_{Q_c-1})$

Với mỗi $1 \leq i \leq N$, cần cộng vào đáp án số lượng ký tự 1 trong mọi số có độ dài i thỏa điều kiện, tức các số

$$2^{i-1}, 2^{i-1} + 1, \dots, g(i, c).$$

Giả sử $g(i, c) = a_1 a_2 \dots a_i$. Liệt kê mọi $2 \leq j \leq i$; nếu $a_j = 1$, thống kê đóng góp của mọi số trong khoảng

$$[\overline{a_1 a_2 \dots a_{j-1} 0} \times 2^{i-j}, \overline{a_1 a_2 \dots a_{j-1} 1} \times 2^{i-j}).$$

Như vậy tổng cộng đã thống kê đóng góp của các số trong khoảng $[2^{i-1}, g(i, c))$.

Đối với mọi số trong khoảng

$$[\overline{a_1 a_2 \dots a_{j-1} 0} \times 2^{i-j}, \overline{a_1 a_2 \dots a_{j-1} 1} \times 2^{i-j}),$$

đóng góp có thể được xử lý thành hai phần.

Với đóng góp của j bit cao nhất, vì j bit cao nhất trong khoảng này đều giống nhau, giả sử j bit cao nhất có tổng cộng x bit 0, thì đáp án cần cộng thêm $x \times 2^{i-j}$.

Với đóng góp của $i - j$ bit còn lại, có thể xem là số lượng bit 1 trong $[0, 2^{i-j})$. Với mọi $1 \leq k \leq i - j$, số lượng số có bit thứ k bằng 1 là 2^{i-j-1} . Vì vậy đáp án cần cộng thêm

$$(i - j) \times 2^{i-j-1}.$$

Dùng các bước trên để thống kê đóng góp cho một i cần $O(N)$ lần cộng $y \times 2^i$ vào đáp án, trong đó y là một số nguyên dương khá nhỏ. Có thể dùng số lớn với cơ số là lũy thừa của 2 để thực hiện việc này trong độ phức tạp xấp xỉ $O(N)$.

Vì cần thống kê đóng góp cho mọi $1 \leq i \leq N$, phần này có thể giải quyết trong thời gian xấp xỉ $O(N^2)$.

Quy trình trên cho ta cách giải hiệu quả sau khi đã tìm được c .

5.6 Phần thứ nhất

Thuật toán mô tả dưới đây chỉ cung cấp cách giải cho phần thứ nhất; cách giải cho phần thứ hai đã được xử lý trong phần trên.

5.6.1 Thuật toán ba

Liệt kê độ dài của c , sau đó tìm kiếm nhị phân giá trị của c . Với một giá trị mid khi tìm kiếm nhị phân, dùng phương pháp trên để tìm

$$g(1, mid), g(2, mid), \dots, g(N, mid),$$

rồi tính tổng độ dài

$$len = \sum_{i=1}^N i \times (g(i, mid) - 2^{i-1} + 1).$$

So sánh len với k để quyết định giữ nửa nào của khoảng tìm kiếm. Cuối cùng, nếu thỏa $0 \leq k - len < |f(c)|$, thì điều này cho thấy độ dài đang được liệt kê đúng là độ dài của c .

Độ phức tạp là $O(N^4)$.

Điểm kỳ vọng: 49 điểm.

5.6.2 Thuật toán bốn

Bổ đề 5. Với mọi $2^{N-2} \leq i < 2^{N-1} - 1$, có

$$Q_{i+1} - Q_i \leq 2N + 2.$$

Chứng minh. Với mọi $1 \leq j < N$, các số có độ dài j và nằm giữa i và $i + 1$ chỉ có thể là hai số:

$$f(i)[1 \dots j] \quad \text{và} \quad f(i+1)[1 \dots j].$$

Các số có độ dài N và nằm giữa i và $i + 1$ chỉ có thể có bốn số:

$$f(i) + "0", \quad f(i) + "1", \quad f(i+1) + "0", \quad f(i+1) + "1".$$

Vậy số lượng số nằm giữa i và $i + 1$ chỉ có thể là

$$2(N - 1) + 4 = 2N + 2.$$

□

Bổ đề 6.

$$n - Q_{2^{N-1}-1} \leq 1.$$

Chứng minh. Các số đứng sau $2^{N-1} - 1$ nhất định chỉ gồm toàn ký tự 1, và có độ dài lớn hơn $N - 1$.

Nếu $n = 2^N - 1$, thì $n - Q_{2^{N-1}-1} = 1$; ngược lại $n - Q_{2^{N-1}-1} = 0$. □

Bổ đề 7. Nếu $g(N - 1, c)$ tồn tại, thì

$$Q_c - Q_{g(N-1,c)} \leq 2N + 2.$$

Chứng minh. Từ Bổ đề 5 và Bổ đề 6, khoảng cách giữa mọi số có độ dài $N - 1$ không vượt quá $2N + 2$, và sau số cuối cùng có độ dài $N - 1$ nhiều nhất chỉ còn một số. Vì vậy với một số bất kỳ, nếu tìm được số có độ dài $N - 1$ lớn nhất đứng trước nó, thì khoảng cách giữa hai số đó không vượt quá $2N + 2$. □

Bổ đề 8.

$$Q_{2^{N-2}} = 1.$$

Chứng minh. Dễ thấy số duy nhất đứng trước 2^{N-2} là 2^{N-1} . □

Theo kết luận của Bổ đề 7 và Bổ đề 8, nếu trước c (không tính c) không có số nào có độ dài $N - 1$, thì c chỉ có thể là 2^{N-1} hoặc 2^{N-2} .

Ngược lại, có thể dùng phương pháp tìm kiếm nhị phân của thuật toán ba để tìm $g(N - 1, c)$. Giả sử $g(N - 1, c) = x$; tiếp theo cần tìm $2N + 2$ số đứng sau x , đồng thời duy trì

$$len = \sum_{i=1}^N i \times (g(i, x) - 2^{i-1} + 1).$$

Để tìm $2N + 2$ số đứng sau x , có thể duy trì $g(i, x)$ với mọi $1 \leq i \leq N$, rồi chèn

$$f(g(i, x) + 1)^\infty [1 \dots 2N]$$

vào một trie. Mỗi thao tác có thể tìm trong trie số có thứ tự từ điển nhỏ nhất trong $O(N)$ và xem nó là số tiếp theo; đồng thời cần cộng độ dài của số này vào len . Nếu sau một thao tác phát hiện $len > k$, thì số hiện tại chính là c .

Phần này có độ phức tạp $O(N^2)$.

Do đã bỏ được bước liệt kê độ dài, độ phức tạp của phần tìm kiếm nhị phân giảm xuống $O(N^3)$.

Điểm kỳ vọng: 72 điểm.

5.6.3 Thuật toán năm

Nhận thấy nút thắt của thuật toán bốn là tìm kiếm nhị phân, còn bước sau khi tìm kiếm nhị phân có vẻ khó tối ưu. Vì vậy có thể thử thay tìm kiếm nhị phân bằng cách xác định từng bit, duy trì một số

$$x = b_1 b_2 \dots b_{N-1}$$

và

$$len = \sum_{i=1}^N i \times (g(i, x) - 2^{i-1} + 1).$$

Ban đầu

$$b_1 = b_2 = \dots = b_{N-1} = 0.$$

Sau đó liệt kê theo thứ tự $b_1, b_2, \dots, b_{N-1}$ từ bit cao tới bit thấp; mỗi lần thử đặt b_i thành 1. Nếu sau khi đặt thành 1 phát hiện $len > k$, đặt lại b_i thành 0. Cuối cùng, x thu được chính là $g(N-1, c)$.

Ở đây cần hỗ trợ sửa một bit của x , đồng thời nhanh chóng duy trì giá trị len .

Đóng góp của $g(N, x)$ có thể được xử lý riêng trong thời gian $O(N)$.

Với $1 \leq i < N$, ta có

$$g(i, x) = h(f(x)[1 \dots i])$$

hoặc

$$g(i, x) = h(f(x)[1 \dots i]) - 1.$$

Định nghĩa dãy

$$d_1, d_2, \dots, d_{N-1} \quad (0 \leq d_i \leq 1)$$

thỏa

$$g(i, x) = h(f(x)[1 \dots i]) - d_i.$$

Khi đó ảnh hưởng của mỗi lần sửa là: $h(f(x)[1 \dots i])$ có thể bị sửa một bit, và d_i bị sửa. Ứng với len , thay đổi này làm len biến động một lượng

$$i \times (y \times 2^j + z) \quad (-1 \leq y, z \leq 1).$$

Nếu có cách duy trì nhanh các biến động của $h(f(x)[1 \dots i])$ và d_i , thì với mọi $1 \leq i < N$, có thể dùng số lớn với cơ sở là lũy thừa của 2 để cộng tất cả các giá trị $i \times (y \times 2^j + z)$ trong độ phức tạp xấp xỉ $O(N)$.

Với việc sửa $h(f(x)[1 \dots i])$, chỉ cần quan sát xem thao tác sửa lần này có ảnh hưởng tới i bit đầu của x hay không. Nếu có, thực hiện sửa đổi tương ứng.

Với việc sửa d_i , định nghĩa

$$r_j = [f(x)^\infty[j] = f(x)^\infty[j+i]].$$

Nếu r_j toàn là 1, thì

$$d_i = [i = N-1].$$

Ngược lại, giả sử vị trí đầu tiên trong r_j bằng 0 là l , khi đó

$$d_i = [f(x)^\infty[l] > f(x)^\infty[l+i]].$$

Theo Định lý 2, chỉ cần duy trì $2(N-1)$ phần tử đầu của r . Khi sửa một bit của x , trong $f(x)^\infty$ chỉ cần sửa 2 vị trí tương ứng, và trong r chỉ cần sửa 4 vị trí tương ứng. Sau đó bài toán trở thành sửa một phần tử của r , và nhanh chóng trả lời vị trí 0 đầu tiên của r .

Ở đây có thể dùng thuật toán chia khối. Trong phạm vi dữ liệu của bài này, $2(N - 1) \leq 4000$, nên đặt kích thước khối là 64 và dùng một unsigned long long để duy trì thông tin của mỗi khối. Số lượng khối cũng không vượt quá 64, nên có thể dùng một unsigned long long để duy trì khối nào còn tồn tại 0. Khi truy vấn, trước hết dùng phép toán bit để tìm khối đầu tiên còn tồn tại 0, rồi trong khối này tìm vị trí 0 đầu tiên.

Như vậy có thể duy trì từng d_i trong độ phức tạp xấp xỉ $O(1)$, và việc sửa một bit của x cũng đạt độ phức tạp xấp xỉ $O(N)$.

Trong quá trình xác định từng bit, cần thực hiện $O(N)$ lần sửa. Vì vậy có thể tìm $g(N - 1, c)$ trong độ phức tạp xấp xỉ $O(N^2)$, và giải quyết bài toán.

Điểm kỳ vọng: 100 điểm.

5.7 Tổng kết

Thuật toán cuối cùng của bài này không dùng bất kỳ kiến thức phức tạp nào. Nhìn bề ngoài đây là một bài số lớn đơn giản, nhưng nó kiểm tra năng lực tư duy, năng lực phân tích vấn đề, và năng lực viết mã của thí sinh. Thuật toán cuối cùng thay tìm kiếm nhị phân bằng xác định từng bit, kết hợp với nội dung riêng của bài để tối ưu; điều này có thể rèn luyện rất tốt tính mở rộng và tính sáng tạo trong tư duy của thí sinh. Độ khó cài đặt của bài này cũng khá cao, có thể kiểm tra nền tảng lập trình cơ bản của thí sinh.

Người ra đề hy vọng dùng bài này để gợi mở cho mọi người, mong rằng mọi người có thể tạo ra những bài hay chỉ dùng kiến thức đơn giản nhưng vẫn đạt độ khó tương đối cao và độ phân hóa tương đối tốt.

Tài liệu tham khảo

1. Liu Rujia. *Sách kinh điển nhập môn thi thuật toán*. Nhà xuất bản Đại học Thanh Hoa, 2009.
2. Cormen T. H., Leiserson C. E., Rivest R. L., và các tác giả khác. *Introduction to Algorithms*. MIT Press, 2009.

6 Báo cáo ra đề *Khối liên thông nhỏ nhất*

Pan Junyue, Trường Trung học số 2 Hàng Châu, tỉnh Chiết Giang

Tóm tắt

Bài viết này giới thiệu một bài tương tác do tác giả ra trong một kỳ thi thử liên trường nội bộ. Bài có nhiều cách giải, đòi hỏi thí sinh hiểu khá sâu cấu trúc của cây và biết vận dụng linh hoạt các tính chất của cây. Đây là một bài tốt để kiểm tra mức độ hiểu biết của thí sinh về cấu trúc dữ liệu cây.

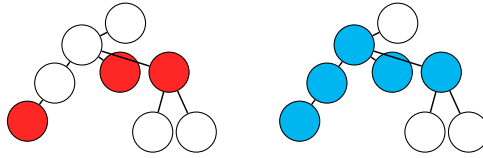
6.1 Ý chính bài toán

6.1.1 Mô tả bài toán

Đây là một bài tương tác.

Với một cây T , ta định nghĩa khối liên thông nhỏ nhất của một tập đỉnh trên cây là khối liên thông trên cây nhỏ nhất chứa tất cả các đỉnh trong tập đó.

Chẳng hạn trong hình dưới, tập đỉnh màu xanh là khối liên thông nhỏ nhất của tập đỉnh màu đỏ.



Biết kích thước n của một cây nào đó, bạn có thể thực hiện một số lần truy vấn. Mỗi lần truy vấn, bạn đưa ra một tập đỉnh S và một đỉnh x trên cây; thư viện tương tác sẽ trả về một giá trị boolean cho biết x có nằm trên khối liên thông nhỏ nhất của tập đỉnh S hay không.

Bạn cần xác định hình dạng của cây.

Bài đảm bảo cây được dùng đã hoàn toàn cố định trước khi bắt đầu tương tác, không được xây dựng động theo quá trình tương tác với chương trình của bạn.

6.1.2 Chi tiết cài đặt

Bạn không cần, và cũng không nên, cài đặt hàm chính. Bạn chỉ cần cài đặt hàm `work(n)`, trong đó n là số đỉnh của cây cần tìm. Bạn có thể gọi bốn hàm sau để tương tác với thư viện:

- `clear()`, biểu thị xóa rỗng tập đỉnh hiện tại S .
- `add(x)`, biểu thị thêm đỉnh số x vào tập đỉnh S .
- `query(x)`, biểu thị hỏi đỉnh x có nằm trên khối liên thông nhỏ nhất của tập đỉnh S hay không.
- `report(x, y)`, biểu thị xác nhận trong cây cần tìm có một cạnh (x, y) .

Khi chấm, thư viện tương tác sẽ gọi `work(n)` đúng một lần.

Ta chỉ giới hạn số lần thực hiện thao tác `query`.

6.1.3 Cách chấm điểm

Bài này có một gói kiểm thử, gồm nhiều điểm kiểm thử.

Với mỗi điểm kiểm thử, nếu chương trình của bạn có truy vấn hoặc giá trị trả về không hợp lệ, hoặc trả về hình dạng cây không đúng, thì bạn được 0 điểm ở điểm kiểm thử đó. Ngược lại, đặt `step` là số lần truy vấn của chương trình, điểm của bạn tại điểm kiểm thử đó được tính là

$$\min \left(\left\lfloor \frac{2.2 \times 10^6}{\text{step}} \right\rfloor, 100 \right).$$

Điểm bạn nhận được cho bài này là giá trị nhỏ nhất trong điểm của tất cả các điểm kiểm thử. Có thể thấy, để đạt điểm tối đa, số lần truy vấn không được vượt quá 22000.

6.1.4 Giới hạn và quy ước

Với mọi dữ liệu, $n = 1000$.

Giới hạn thời gian: 5 giây.

Giới hạn bộ nhớ: 1024 MB.

6.2 Phân tích lời giải

6.2.1 Thuật toán một

Trong phần sau, ta gọi một thao tác “hỏi x có nằm trên khối liên thông nhỏ nhất của tập đỉnh S hay không” là $\text{query}(S, x)$.

Trước hết, ta chuyển đổi cách truy vấn đã cho. Hỏi x có nằm trên khối liên thông nhỏ nhất của tập đỉnh S hay không tương đương với hỏi trong S có tồn tại hai đỉnh u, v sao cho x nằm trên đường đi từ u đến v hay không.

Vì vậy, nếu tập được hỏi S có kích thước 2, ta có thể trực tiếp hỏi đỉnh x được truy vấn có nằm trên đường đi giữa hai đỉnh trong S hay không.

Ta có ngay một cách làm đa thức: với ba đỉnh bất kỳ u, v, w , dùng $\text{query}(\{u, v\}, w)$ để thu được quan hệ giữa chúng, từ đó xác định hình dạng của toàn bộ cây.

Nói cách khác, ta có:

Thuật toán một. Với mỗi cặp đỉnh (u, v) , ta thực hiện một lần $\text{query}(\{u, v\}, w)$ cho mỗi đỉnh còn lại w . Nếu đáp án của cả $n - 2$ truy vấn này đều là `false`, thì u và v nối trực tiếp với nhau. Nhờ đó có thể tìm tất cả các cặp đỉnh nối trực tiếp, rồi xác định hình dạng của toàn bộ cây.

Độ phức tạp thuật toán: $O(n^3)$.

6.2.2 Thuật toán hai

Có thể nhận thấy ta không cần lấy kết quả $\text{query}(\{u, v\}, w)$ cho mọi bộ ba đỉnh u, v, w . Trên thực tế, sau khi xem cây này như một cây có gốc, nếu ta biết được quan hệ “tổ tiên–hậu duệ” giữa mọi cặp đỉnh, thì cũng có thể xác định hình dạng cây. Đây cũng là một cách làm thường gặp để giải loại bài này.

Trong phần sau, ta đều xem cây là cây có gốc, đồng thời lấy đỉnh số 1 làm gốc.

Để biết giữa một cặp đỉnh u, v có quan hệ “tổ tiên–hậu duệ” hay không, chỉ cần hỏi $\text{query}(\{1, u\}, v)$.

Nói cách khác, ta có:

Thuật toán hai. Với mỗi cặp đỉnh (u, v) , ta dùng $\text{query}(\{1, u\}, v)$ để phán đoán v có phải tổ tiên của u hay không. Từ đó ta có thể thu được tập tổ tiên của mỗi đỉnh. Gọi tập tổ tiên của đỉnh i là Anc_i (đặc biệt, ta xem chính đỉnh i cũng là tổ tiên của i). Với một đỉnh x , cha f của nó chính là đỉnh thỏa $\text{Anc}_f \subset \text{Anc}_x$ và có $|\text{Anc}_f|$ lớn nhất. Xác định cha của mỗi đỉnh là xác định được hình dạng của cây.

Độ phức tạp thuật toán: $O(n^2)$.

6.2.3 Thuật toán ba

Chú ý rằng đến đây ta vẫn chỉ dùng các truy vấn với tập đỉnh có kích thước 2. Tiếp theo ta khai thác thêm cách dùng của dạng truy vấn được cho.

Vì có thể thu được quan hệ “tổ tiên–hậu duệ” giữa hai đỉnh, ta cũng có thể dùng cách tương tự để chỉ bằng một truy vấn biết giữa nhiều đỉnh có tồn tại quan hệ “tổ tiên–hậu duệ” hay không.

Với một tập đỉnh S và một đỉnh u , ta thực hiện $\text{query}(S \cup \{1\}, u)$. Nếu kết quả là `true`, điều đó nói rằng trong S có hậu duệ của u ; nếu không thì không có. Lý do là nếu trong S đồng thời có hai đỉnh, một đỉnh nằm trong cây con của u và một đỉnh nằm ngoài cây con của u , thì đường đi giữa chúng sẽ đi qua u , mà ở đây ta đã cung cấp sẵn một đỉnh ngoài cây con là đỉnh số 1. Tương tự, muốn tồn tại hai đỉnh sao cho đường đi tương ứng của chúng đi qua u , thì ít nhất một trong hai đỉnh đó phải nằm trong cây con của u . Vì vậy, “trong S có hậu duệ của u ” và “giá trị trả về của truy vấn là `true`” là điều kiện cần và đủ của nhau.

Khai thác thêm một bước, sau khi xác nhận trong S có hậu duệ của u , ta có thể dùng cách “tìm kiếm nhị phân” để trực tiếp tìm một hậu duệ nào đó của u trong S . Cách làm cụ thể như sau:

- Chia tập đỉnh S thành hai tập đỉnh S_L, S_R , sao cho $S_L \cup S_R = S$, $S_L \cap S_R = \emptyset$, và $-1 \leq |S_L| - |S_R| \leq 1$.
- Thực hiện một truy vấn $\text{query}(S_L \cup \{1\}, u)$. Nếu giá trị trả về là true , sửa S thành S_L ; nếu không, sửa S thành S_R .
- Lặp lại hai bước trên cho tới khi $|S| = 1$. Khi đó đỉnh trong S là hậu duệ cần tìm.

Vì sau mỗi truy vấn, kích thước của S nhiều nhất trở thành $\lceil |S|/2 \rceil$, nên độ phức tạp của một thao tác “tìm một hậu duệ nào đó” là $O(\log |S|)$.

Khai thác xa hơn, ta có thể tìm tất cả các hậu duệ của u trong S bằng các bước sau:

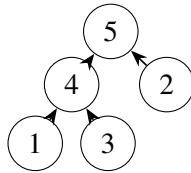
- Phán đoán trong S có hậu duệ của u hay không; nếu không thì thoát trực tiếp.
- Tìm một hậu duệ của u trong S .
- Loại hậu duệ vừa tìm khỏi S , rồi quay lại bước đầu tiên.

Giả sử số hậu duệ của u trong S là m , độ phức tạp của cách làm này là $O(m \log |S|)$.

Sau khi có các công cụ trên, ta thấy có thể chuyển hóa bài toán.

Nếu xem mỗi cạnh của cây có gốc là một cạnh có hướng từ con trở về cha, ta có thể xem cây có gốc như một đồ thị có hướng không chu trình, và định nghĩa một thứ tự topo của cây có gốc.

Ví dụ trong hình dưới, với một thứ tự topo nào đó của cây có gốc này, con số trên mỗi đỉnh thể hiện vị trí của nó trong thứ tự topo đó:



Ta nhận thấy nếu biết được một thứ tự topo của cây cần tìm, ta có thể trực tiếp xác định cây đó. Các bước cụ thể như sau:

- Gọi $a_1, \dots, a_n$ là số hiệu của đỉnh thứ i trong thứ tự topo của cây. Giá trị ban đầu của tập V là tập đỉnh của cây.
- Cho i quét từ 1 đến n . Ở mỗi bước, tìm tất cả hậu duệ của a_i trong V , rồi loại các hậu duệ này khỏi V .

Vì các hậu duệ không phải con của một đỉnh đã bị loại khỏi V tại cha tương ứng của chúng, nên các hậu duệ mà một đỉnh tìm được chính là tất cả các con của nó trong cây ban đầu.

Có thể chứng minh độ phức tạp của phần này là $O(n \log n)$, vì mỗi đỉnh chỉ bị tìm ra với vai trò hậu duệ đúng một lần, và mỗi lần tìm một hậu duệ đều tốn $O(\log n)$.

Từ đây, ta chuyển bài toán thành tìm một thứ tự topo của cây ban đầu. Tất cả các cách làm tiếp theo đều nhằm mục tiêu “tìm ra thứ tự topo”.

Xét thuật toán topo thường dùng: mỗi lần ta tìm một đỉnh có bậc vào bằng 0, đưa nó vào cuối thứ tự topo hiện tại, rồi xóa nó khỏi đồ thị ban đầu. Chuyển sang cây, tức là mỗi lần tìm một lá của cây ban đầu, rồi loại lá đó khỏi cây.

Ta gọi cách làm này là “bóc lá”.

Việc phán đoán một đỉnh u có phải lá hay không rất đơn giản. Gọi tập đỉnh của cây là V , chỉ cần hỏi $\text{query}(V - \{u\}, u)$.

Nói cách khác, ta có:

Thuật toán ba. Mỗi vòng, ta quét qua từng đỉnh của cây ban đầu và phán đoán nó có phải lá hay không. Nếu phải, đưa nó vào cuối thứ tự topo. Sau khi vòng đó kết thúc, loại tất cả các lá tìm được khỏi tập đỉnh của cây ban đầu. Cách phán đoán một đỉnh có phải lá hay không và cách làm sau khi tìm được thứ tự topo sẽ không được nhắc lại trong phần này và các phần sau.

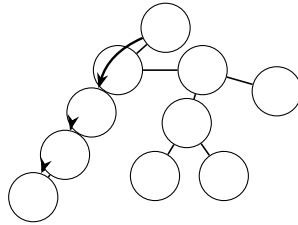
Độ phức tạp thuật toán: $O(n^2)$.

6.2.4 Thuật toán bốn

Trên thực tế, độ phức tạp của việc cố tìm một lá ở mỗi lần là quá cao; ta có thể thử tối ưu ở đây.

Ta lại dùng công cụ trước đó: mỗi lần liên tục tìm ngẫu nhiên một hậu duệ của đỉnh hiện tại trong tập đỉnh của cây rồi “nhảy” tới đó, cho tới khi tìm được một lá.

Hình dưới là một ví dụ cho quá trình “nhảy” của thuật toán:



Gọi siz_x là kích thước cây con của đỉnh x . Với mỗi đỉnh u , trong các hậu duệ của nó, số đỉnh có kích thước cây con vượt quá $\lceil \text{siz}_u/2 \rceil$ nhiều nhất chiếm một nửa. Do đó kỳ vọng nhiều nhất 2 lần là có thể khiến kích thước cây con của đỉnh hiện tại giảm còn một nửa. Từ đây có thể chứng minh, nếu “nhảy” xuống dưới theo cách này, số lần “nhảy” kỳ vọng là $O(\log n)$. Mỗi lần tìm một hậu duệ cũng có độ phức tạp $O(\log n)$, nên độ phức tạp để tìm một lá là $O(\log^2 n)$.

Nói cách khác, ta có:

Thuật toán bốn. Đây là một thuật toán ngẫu nhiên. Ta bốc n lá để xác định thứ tự topo. Mỗi lần bắt đầu, đặt $u = 1$, rồi tìm ngẫu nhiên một hậu duệ v của đỉnh u , gán v cho u , cho tới khi u trở thành một lá.

Độ phức tạp thuật toán: $O(n \log^2 n)$.

6.2.5 Thuật toán năm

Thuật toán ba còn có một hướng tối ưu khác: tối ưu số vòng “bóc lá”.

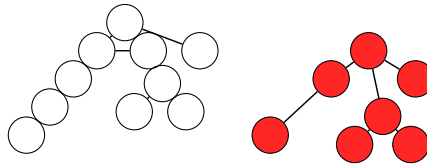
Có thể nhận thấy nếu mỗi vòng đều bóc tất cả các lá của cây ban đầu, trong trường hợp xấu nhất (khi cây ban đầu là một đường thẳng), cần bóc n vòng.

Ta phát hiện cây có một tính chất: nếu trong cây không có đỉnh bậc hai, thì số lá của cây ít nhất bằng một nửa tổng số đỉnh. Từ đây có thể nghĩ tự nhiên tới định nghĩa sau.

Định nghĩa cây ảo của một cây là cây thu được sau khi thực hiện các thao tác sau:

- Tìm một đỉnh bậc hai w ngoài gốc trong cây hiện tại, giả sử hai đỉnh kề với nó là u, v . Nếu không tìm được thì thoát.
- Xóa w và các cạnh nhận w làm đầu mút khỏi cây, rồi thêm cạnh (u, v) .
- Quay lại bước đầu tiên.

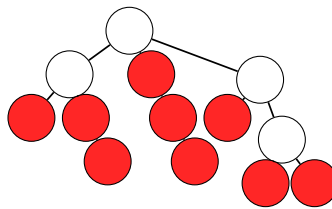
Ví dụ trong hình dưới, cây bên phải chính là cây ảo của cây bên trái.



Có thể nhận thấy, nếu ta khiến các lá của cây ảo của cây cần tìm đều bị bóc ở mỗi vòng, thì ta có thể bóc hết mọi đỉnh trong $O(\log n)$ vòng.

Muốn các lá của cây ảo ở một vòng nào đó đều bị bóc, những đỉnh ta cần bóc trong cây ban đầu là: các lá của cây ban đầu và chuỗi các đỉnh bậc hai nằm phía trên lá. Nói cách khác, đó là những đỉnh mà trong các hậu duệ chỉ có một hậu duệ là lá (đặc biệt, ta xem một đỉnh cũng là hậu duệ của chính nó).

Trong hình dưới, các đỉnh đỏ là những đỉnh ta cần bóc trong cây:



Đồng thời, việc tìm các đỉnh này cũng dễ. Chỉ cần trước hết tìm tập lá L , rồi với mỗi đỉnh, phán đoán nó có ít nhất 2 hậu duệ trong L hay không. Cách phán đoán là: trước hết tìm một hậu duệ v của đỉnh đó trong L , rồi xem trong $L - \{v\}$ có tồn tại hậu duệ của đỉnh đó hay không.

Sau khi tìm các đỉnh này, ta chia nhóm chúng theo hậu duệ lá tương ứng; trong mỗi nhóm còn phải sắp xếp một lần theo quan hệ “tổ tiên–hậu duệ”.

Như vậy ta hoàn thành công việc của một vòng với độ phức tạp $O(n \log n)$. Ta cần thực hiện tổng cộng $O(\log n)$ vòng, nên tổng độ phức tạp là $O(n \log^2 n)$. Cần đặc biệt chú ý rằng ở mỗi vòng, ta không làm số đỉnh của cây ban đầu giảm còn một nửa, nên không thể cho rằng độ phức tạp là $O(n \log n)$.

Nói cách khác, ta có:

Thuật toán năm. Mỗi vòng, tìm tất cả lá trong cây hiện tại. Sau đó, với mỗi đỉnh khác trên cây, tìm một hậu duệ lá của nó và phán đoán nó có ít nhất 2 hậu duệ lá hay không. Những đỉnh chỉ có một hậu duệ lá được chia nhóm theo hậu duệ lá đó; trong mỗi nhóm, sắp xếp các đỉnh, trong đó hai đỉnh u, v được so sánh bằng $\text{query}(\{1, u\}, v)$. Cuối cùng, bóc toàn bộ các đỉnh tìm được trong vòng này.

Độ phức tạp thuật toán: $O(n \log^2 n)$.

6.2.6 Thuật toán sáu

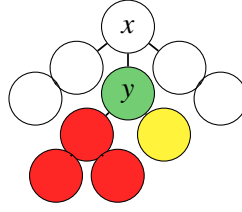
Không tiếp cận từ góc độ “bóc lá” nữa, ta xét cách làm thường gặp của loại bài này: chia để trị.

Ta đặt hàm $\text{solve}(x, S)$ biểu thị rằng đã xác định tập đỉnh trong cây con của x là S , và bây giờ cần sắp xếp các đỉnh trong cây con của x theo thứ tự topo.

Tìm ngẫu nhiên một hậu duệ y của x trong $S - \{x\}$. Ta cố xác định tập đỉnh S' tạo bởi các đỉnh trong cây con của y , để cây con của x có thể được chia thành hai phần, thuận tiện cho chia để trị.

Thực ra điều này không giống lắm với chia để trị theo trọng tâm truyền thống, nhưng ta vẫn có thể giải quyết bằng một thuật toán sắp xếp có độ phức tạp tốt.

Để hiện thực ý tưởng này với độ phức tạp tốt hơn, trước hết ta giả sử các đỉnh này trong cây ban đầu được tô các màu khác nhau theo thành phần liên thông chứa chúng sau khi xóa đỉnh y , như hình dưới:



Có thể thấy mục tiêu của ta là phán đoán mỗi đỉnh có phải màu trắng hay không. Trước hết, ta lấy các đỉnh này trừ y rồi sắp thành một hàng theo số hiệu đỉnh, có dạng như sau:



Gọi một đoạn liên tiếp cực đại gồm các đỉnh cùng màu là “một đoạn”. Bây giờ ta thử tìm vị trí các đường phân chia giữa các đoạn trong hàng đỉnh này.

Ta biết $\text{query}(A, y)$ trả về true khi và chỉ khi trong tập đỉnh A có hai đỉnh khác màu. Vì vậy ta có thể dùng một truy vấn để phán đoán một chuỗi đỉnh liên tiếp có cùng màu hay không. Khi đã xác định đầu trái của một đoạn, ta có thể dùng thao tác này để tìm kiếm nhị phân đầu phải của đoạn đó; quét từ trái sang phải là có thể xác định cách chia giữa các đoạn.

Cuối cùng, có thể dùng $\text{query}(\{1, u\}, y)$ để phán đoán đỉnh u có phải màu trắng hay không. Nhờ đó ta biết mỗi đoạn có phải màu trắng hay không, rồi tiếp tục chia để trị.

Xét chứng minh độ phức tạp của cách làm này.

Giả sử có m đỉnh được tô màu xuất hiện nhiều nhất. Độ phức tạp để xác định cách chia giữa các đoạn là $O((|S| - m) \log |S|)$. Nguyên nhân là mỗi lần xác định một đoạn đều tốn $O(\log |S|)$, còn số đoạn là $O(|S| - m)$: màu xuất hiện nhiều nhất bị chia thành nhiều nhất $|S| - m + 1$ đoạn, trong khi số đỉnh còn lại chỉ là $|S| - m$.

Nói cách khác, trong mỗi lần chia để trị, ta có thể xem các đỉnh có màu không phải màu xuất hiện nhiều nhất là những đỉnh đóng góp $O(\log |S|)$ vào độ phức tạp. Xét đóng góp của mỗi đỉnh:

- Nếu đỉnh đó là một đỉnh màu trắng, vì màu trắng không phải màu xuất hiện nhiều nhất, thành phần liên thông còn lại được tách ra lớn hơn thành phần liên thông màu trắng; kích thước của thành phần liên thông màu trắng không vượt quá một nửa kích thước thành phần liên thông ban đầu, nên tình huống này xảy ra không quá $O(\log n)$ lần.
- Nếu đỉnh đó là một đỉnh có màu, thì nếu thành phần liên thông màu trắng là lớn nhất, ta quay về tình huống thứ nhất. Ngược lại, gốc của thành phần liên thông chứa đỉnh đó là một con nhẹ của y . Lại vì mỗi đỉnh được chọn làm y nhiều nhất một lần, tình huống này cũng xảy ra không quá $O(\log n)$ lần.

Từ đó, ta chứng minh được tổng độ phức tạp là $O(n \log^2 n)$.

Nói cách khác, ta có:

Thuật toán sáu. Khi xử lý cây con của x , giả sử các đỉnh trong cây con lần lượt là $a_1, \dots, a_{\text{cnt}}$ (ở đây $\text{cnt} = |S|$). Tìm ngẫu nhiên một hậu duệ y của x , và giả định cây được tô màu theo đó. Ban đầu đặt $l = 1$; mỗi lần tìm kiếm nhị phân để tìm r lớn nhất thỏa $l \leq r \leq \text{cnt}$ và $a_l, \dots, a_r$ cùng màu, rồi đặt $l = r + 1$, cho tới khi $l > \text{cnt}$. Sau khi xác định cách chia giữa các đoạn, với mỗi giá trị l từng xuất hiện trước đó, hỏi $\text{query}(\{1, a_l\}, y)$ để phán đoán màu của đoạn. Cuối cùng chia để trị theo màu.

Độ phức tạp thuật toán: $O(n \log^2 n)$.

6.2.7 Thuật toán bảy

Thực ra, dùng tư tưởng tương tự chia để trị có thể thu được độ phức tạp tốt hơn.

Đặt hàm $\text{solve}(x, S)$ biểu thị yêu cầu tìm những đỉnh nào trong S nằm trong cây con của x , rồi sắp xếp chúng theo thứ tự topo. Nói cách khác, trước khi gọi $\text{solve}(x, S)$, ta chưa xác định những đỉnh nào nằm trong cây con của x .

Quy trình thuật toán như sau:

- Tìm một hậu duệ y của x trong $S - \{x\}$. Nếu không tồn tại hậu duệ như vậy thì thoát.
- Thực hiện hàm $\text{solve}(y, S)$, sau đó xóa khỏi S các đỉnh nằm trong cây con của y , và thêm các đỉnh trong cây con của y vào cuối thứ tự topo theo thứ tự topo của chúng.
- Quay lại bước đầu tiên.

Ban đầu chỉ cần gọi $\text{solve}(1, \{1, \dots, n\})$. Vì trong thuật toán này mỗi đỉnh chỉ bị tìm ra với vai trò y đúng một lần, tổng độ phức tạp là $O(n \log n)$.

Tư tưởng cốt lõi của thuật toán này là vứt bỏ những thông tin thực ra vô dụng, bóc cây con của x từng khối một cho tới khi chỉ còn x . Tuy quy trình rất đơn giản, độ khó tư duy để nghĩ ra nó lại không thấp.

Nói cách khác, ta có:

Thuật toán bảy. Khi xử lý cây con của x , liên tục tìm một hậu duệ y của x và chia để trị xuống y , rồi bóc phần cây con của y khỏi cây con của x ; lặp lại cho tới khi chỉ còn x .

Độ phức tạp thuật toán: $O(n \log n)$.

6.2.8 Thuật toán tám

Thực ra mọi cách làm ở trên đều xuất phát từ cấu trúc của cây, nhưng ta cũng có thể xét trực tiếp trên thứ tự topo.

Ta biết một điều kiện cần và đủ của thứ tự topo của một cây có gốc như vậy: mọi tổ tiên của mỗi đỉnh đều phải xuất hiện sau đỉnh đó. Nếu có thể lần lượt thêm các đỉnh vào thứ tự topo và luôn duy trì tính chất này, ta có thể giải quyết toàn bộ bài toán.

Ta đã có cách phán đoán trong tập đỉnh S có tồn tại hậu duệ của x hay không. Vì vậy chỉ cần tìm kiếm nhị phân một vị trí trong thứ tự topo hiện tại, sao cho sau vị trí đó không tồn tại hậu duệ của x và vị trí đó càng sớm càng tốt, thì ta có thể bảo đảm sau khi thêm đỉnh này, tất cả tổ tiên của nó vẫn nằm sau nó, và tất cả hậu duệ của nó vẫn nằm trước nó.

Vì mỗi lần thêm một đỉnh ta chỉ cần tìm kiếm nhị phân, tổng độ phức tạp là $O(n \log n)$.

Nói cách khác, ta có:

Thuật toán tám. Duyệt i từ 1 đến n . Giả sử trước khi thêm đỉnh số i vào thứ tự topo, thứ tự topo là $a_1, \dots, a_{i-1}$. Tìm kiếm nhị phân một vị trí p sao cho giá trị trả về của $\text{query}(\{1\} \cup \{a_p, \dots, a_{i-1}\}, i)$ là false và p nhỏ nhất có thể. Chèn i vào trước vị trí thứ p , tức là sửa thứ tự topo thành

$$a_1, \dots, a_{p-1}, i, a_p, \dots, a_{i-1}.$$

Cuối cùng dùng lại cách làm sau khi đã có thứ tự topo.

Độ phức tạp thuật toán: $O(n \log n)$.

6.3 Tổng kết

Đề bài này ngắn gọn và tinh tế; kiến thức cần dùng chỉ là một số tính chất của cây, nhưng bài kiểm tra tổng hợp năng lực chế tạo công cụ và chuyển hóa vấn đề của thí sinh, đồng thời lời giải dùng nhiều kỹ thuật chia để trị. Bài không yêu cầu cao về năng lực viết mã, nhưng có yêu cầu nhất định về tư duy.

Để giải bài này, cần chuyển bài toán thành “tìm một thứ tự topo của cây ban đầu”. Trong quá trình chuyển hóa, bài nhấn mạnh tư tưởng “đóng gói dạng truy vấn được cho thành nhiều loại công cụ”. Sau khi chuyển thành “tìm thứ tự topo”, bài có thể được tiếp cận từ nhiều góc độ như “chia để trị theo đỉnh”, “bóc lá”, “duy trì trực tiếp thứ tự topo”, và đều có thể thu được các thuật toán khá tốt và khá thú vị. Chẳng hạn, ta khéo léo dùng tư tưởng “tô màu đỉnh và chia thành các đoạn” để thu được thuật toán sáu, đồng thời dùng nhiều tính chất của cây để chứng minh độ phức tạp của thuật toán sáu; một ví dụ khác là ta dùng tư tưởng “cây không có đỉnh bậc hai thì ít nhất một nửa số đỉnh là lá” để thu được thuật toán năm.

Người ra đề cho rằng một bài không cần kiến thức cao sâu nhưng vẫn có thể kiểm tra sâu mức độ hiểu biết của thí sinh về tính chất của cây như thế này xứng đáng là một bài hay, và cũng hy vọng bài này có thể gợi mở, để thấy thêm nhiều bài thú vị hơn.

6.4 Lời cảm ơn

Cảm ơn China Computer Federation đã cung cấp nền tảng học tập và trao đổi.

Cảm ơn huấn luyện viên đội tuyển quốc gia Gao Wenyuan vì sự hướng dẫn.

Cảm ơn thầy Li Jian của Trường Trung học số 2 Hàng Châu vì sự quan tâm và chỉ dẫn.

Cảm ơn các bạn Zhou Xin, Fang Youle, Fang Tangqi và những người khác đã thảo luận thuật toán của bài này với tôi, giúp kiểm tra đề.

Cảm ơn cha mẹ vì sự quan tâm và ủng hộ.

Tài liệu tham khảo

1. Codeforces 1129E, *Legendary Tree*.

7 Giới thiệu đơn giản về nguyên lý chuyển vị

Chen Yu, Trường Trung học trực thuộc Đại học Sư phạm An Huy

Tóm tắt

Bài viết này giới thiệu nguyên lý chuyển vị (transposition principle), còn gọi là nguyên lý Tellegen, một nhóm quy tắc để viết lại các thuật toán tuyến tính (linear algorithm). Bài viết sẽ giới thiệu chuyển vị của một số thuật toán tuyến tính thường dùng, chỉ ra rằng phương pháp chuyển vị có thể giúp giải một số bài toán thuận tiện hơn và tối ưu hiệu suất, đồng thời giới thiệu một lớp đặc biệt của bài toán nhân ma trận với vector được tối ưu bằng nguyên lý chuyển vị cùng các ứng dụng của nó.

7.1 Khái quát

Ta giả sử mọi thao tác trên số dưới đây đều được thực hiện trên một trường số F .

Định nghĩa 1. Một thuật toán nhận vào một vector $n \times 1$ là a , lấy một ma trận hằng $n \times n$ là A nhân bên trái vector đó, rồi xuất kết quả $b = Aa$ (trong đó b cũng là một vector $n \times 1$), được gọi là thuật toán tuyến tính. Ta xem vector đầu vào là biến, còn ma trận là hằng. Để tiện trình bày, ở đây chỉ xét trường hợp A là ma trận vuông; nếu không vuông thì có thể bổ sung các số 0 thích hợp.

Định nghĩa 2. Với thuật toán tuyến tính có dạng $b = Aa$, ta gọi thuật toán tuyến tính có dạng $b' = A^T a'$ là chuyển vị của thuật toán đó.

Nguyên lý chuyển vị khẳng định rằng ta có thể viết lại một thuật toán tuyến tính thành thuật toán sau chuyển vị, đồng thời giữ nguyên độ phức tạp thời gian và không gian. Như vậy ta có thể chuyển vị một thuật toán, tối ưu thuật toán sau chuyển vị, rồi viết lại thuật toán sau chuyển vị đó thành thuật toán giải bài toán ban đầu. Trong nhiều tình huống, cách này có thể đơn giản hóa bài toán.

7.2 Viết lại

Mục này chỉ ra rằng việc viết lại một thuật toán tuyến tính thành chuyển vị của nó là một quy trình khá cơ học.

Trước hết ta thảo luận ảnh hưởng của phép chuyển vị ma trận lên quá trình thuật toán. Để tiện, ta tính mọi bộ nhớ phụ liên quan tới biến mà chương trình dùng vào vector đầu vào; khi đó chỉ cần thay đổi đầu vào và đầu ra thích hợp là có thể đạt hiệu quả tương đương.

Định nghĩa 3. Ta sửa nhẹ ma trận đơn vị I để nhận được ba loại ma trận sau:

1. Đổi chỗ hàng thứ i và hàng thứ j của I . Ma trận này nhân bên trái một vector có tác dụng đổi chỗ phần tử thứ i và phần tử thứ j của vector đó.
2. Đổi phần tử thứ i trên hàng thứ i của I từ 1 thành số a . Ma trận này nhân bên trái một vector có tác dụng nhân phần tử thứ i của vector đó với a .
3. Đổi phần tử ở hàng thứ i , cột thứ j ($j \neq i$) của I từ 0 thành hằng số a . Ma trận này nhân bên trái một vector có tác dụng lấy phần tử thứ j nhân với a rồi cộng vào phần tử thứ i .

Ta gọi ba loại ma trận này là ma trận sơ cấp.

Nhân ma trận với vector thường cần dùng $O(n)$ bộ nhớ phụ. Sau khi xử lý, ma trận A có thể được tách trực tiếp thành tích của một dãy ma trận sơ cấp

$$A = E_1 E_2 \cdots E_k.$$

Do đó

$$A^T = E_k^T E_{k-1}^T \cdots E_1^T.$$

Xét chuyển vị của ma trận sơ cấp: loại thứ nhất và loại thứ hai không đổi sau khi chuyển vị; loại thứ ba biến thao tác cộng phần tử thứ j vào phần tử thứ i thành thao tác cộng phần tử thứ i vào phần tử thứ j . Vì vậy thuật toán sau chuyển vị chính là thực hiện mọi câu lệnh của thuật toán ban đầu theo thứ tự ngược lại, đồng thời đổi câu lệnh dạng $x \leftarrow x + ay$ trong thuật toán ban đầu thành $y \leftarrow y + ax$. Với thao tác cộng hằng số, có thể bổ sung một 1 vào ma trận để thực hiện.

Trong ứng dụng thực tế, có thể ta cần nhập vào một ma trận, rồi dựa trên ma trận đó tính ra mọi hằng số mà thuật toán dùng, từ đó nhận được một thuật toán tuyến tính.

Với tham số hàm, phần liên quan tới hằng số giữ nguyên; còn biến thì đổi từ truyền vào thành trả về hoặc sửa đổi, và từ trả về hoặc sửa đổi thành truyền vào.

Lời gọi hàm chỉ cần được đảo ngược trực tiếp như bình thường, lần lượt thực hiện chuyển vị các hàm khác được gọi bên trong; đệ quy cũng có thể xem như lời gọi tới hàm khác. Điểm cần chú ý là vì tại mỗi thời điểm khi chạy ngược, ta cần tính được mọi hằng số của trạng thái chạy xuôi tới đó, nên thường tiền xử lý trước sẽ thuận tiện hơn.

7.3 Ví dụ

7.3.1 Biến đổi Fourier rời rạc

Xét ma trận của DFT:

$$\begin{bmatrix} 1 & 1 & 1 & \dots & 1 \\ 1 & \omega_n & \omega_n^2 & \dots & \omega_n^{n-1} \\ 1 & \omega_n^2 & \omega_n^4 & \dots & \omega_n^{2(n-1)} \\ \vdots & \vdots & \vdots & \ddots & \vdots \\ 1 & \omega_n^{n-1} & \omega_n^{2(n-1)} & \dots & \omega_n^{(n-1)(n-1)} \end{bmatrix}.$$

Vì ma trận này là ma trận đối xứng, chuyển vị của thuật toán này chính là bản thân nó. IDFT cũng tương tự.

7.3.2 Biến đổi Fourier nhanh

Dạng FFT được dùng rộng rãi là DIT (decimation in time, tách theo miền thời gian)-FFT: tách hệ số đa thức thành các hạng chẵn và hạng lẻ, rồi dễ dàng đệ quy thành các bài toán con theo tính chất của căn đơn vị; ở đây không nhắc lại. Khi viết thành dạng không đệ quy, cách thường dùng là đảo bit của chỉ số mảng; khi đó mỗi lần lấy bit thấp nhất sẽ trở thành lấy bit cao nhất, các hạng chẵn và hạng lẻ trở thành nửa trái và nửa phải, nên dễ xử lý.

Mặt khác, ta xét việc tách trực tiếp thành nửa trước và nửa sau:

$$\begin{aligned} \text{DFT}(a, n)_k &= \sum_{i=0}^{n-1} a_i \omega_n^{ik} \\ &= \sum_{i=0}^{n/2-1} (a_i^{(left)} + a_i^{(right)} \omega_n^{n/2 k}) \omega_n^{ik}. \end{aligned}$$

Vì $\omega_n^{n/2 k} = (-1)^k$, bằng cách xét tính chẵn lẻ của k , ta có thể nhận được một thuật toán đặt đáp án của các hạng chẵn ở bên trái, đáp án của các hạng lẻ ở bên phải. Khi đổi thuật toán này sang dạng không đệ quy, chỉ cần đảo bit của chỉ số mảng ở bước cuối.

Thuật toán này được gọi là DIF (decimation in frequency, tách theo miền tần số)-FFT. Quan sát quá trình thuật toán, hai thuật toán có cùng chức năng này là chuyển vị của nhau, điều đó cũng xác nhận rằng chuyển vị của DFT là chính nó.

7.3.3 Nhân đa thức

Ta dùng $M(n)$ để biểu thị thời gian tính $2n$ vị trí đầu của tích chập vòng của hai đa thức, và giả định $M(n) = O(n \log n)$.

Với hai đa thức a và b lần lượt có bậc cao nhất là $n-1$ và $m-1$, chương trình nhân đa thức thô để thu được đa thức $c = ab$ bậc $n+m-1$ như sau:

```
1  c ← 0
2  for i ← 0 to n + m - 2
```

```

3         for j ← max(0, i - n + 1) to min(i, m - 1)
4             ci ← ci + ai- j b j

```

Ta xem b là hằng số và chuyển viết trực tiếp theo quy tắc trên, nhận được chuyển vị của phép nhân: cho hai đa thức a và b lần lượt có bậc cao nhất là $n - 1$ và $m - 1$, kết quả là đa thức $c = \text{mulT}(a, b)$ bậc $n + m$:

```

1  c ← 0
2  for i ← 0 to n + m - 2
3      for j ← min(i, m - 1) downto max(0, i - (n - m))
4          ci- j ← ci- j + ai b j

```

Từ đó có thể thấy $\text{mulT}(a, b)$ là các vị trí từ $m - 1$ tới $n - 1$ trong kết quả tích chập của a với b sau khi đảo hệ số.

Thời gian của phép nhân đa thức đương nhiên là $\frac{1}{2}M(n + m)$, còn thời gian của phép nhân chuyển vị có thể đạt $\frac{1}{2}M(n)$, bởi phần tràn của tích chập vòng không giao với phần ta cần. Cùng cách làm này cũng dễ suy ra từ kết luận ở mục 4.1.

7.4 Chuyển vị của ma trận Vandermonde

Định nghĩa 4. Ma trận Vandermonde là một ma trận vuông $n \times n$ có dạng

$$V_a = \begin{bmatrix} 1 & \alpha_0 & \alpha_0^2 & \cdots & \alpha_0^{n-1} \\ 1 & \alpha_1 & \alpha_1^2 & \cdots & \alpha_1^{n-1} \\ \vdots & \vdots & \vdots & \ddots & \vdots \\ 1 & \alpha_{n-1} & \alpha_{n-1}^2 & \cdots & \alpha_{n-1}^{n-1} \end{bmatrix}.$$

Định nghĩa 5. Với đa thức b có bậc cao nhất $n - 1$ (vector hệ số của nó cũng được viết là b),

$$V_a b^T = (b(\alpha_0), b(\alpha_1), \dots, b(\alpha_{n-1}))^T.$$

Phép tính này thường được gọi là tính giá trị đa thức tại nhiều điểm.

Sau đây ta sẽ chỉ ra rằng ứng dụng nguyên lý chuyển vị có thể giảm đáng kể hằng số thời gian của phép tính giá trị tại nhiều điểm. Trước hết giới thiệu vài bài toán liên quan.

7.4.1 Chia đa thức và lấy phần dư

Cho đa thức f bậc $n - 1$ và đa thức g bậc $m - 1$ ($m \leq n$), cần tìm h, r sao cho $f = gh + r$ và $|r| < n$ (giả sử m xấp xỉ $0.5n$).

Với bài toán này, thuật toán Sieveking–Kung cho công thức tính như sau: gọi $\text{rev}(n, a)$ là kết quả đảo hệ số của đa thức a bậc $n - 1$, ta có

$$\text{rev}(n - m + 1, h) = \text{rev}(n, f) \frac{1}{\text{rev}(m, g)} \pmod{x^{n-m+1}}.$$

Tính thương của chuỗi lũy thừa hình thức có thể hoàn thành trong thời gian $\frac{7}{3}M(m) = \frac{7}{6}M(n)$ [1]. Sau đó tính $r = f - gh$ còn cần dùng thời gian $M(m) = \frac{1}{2}M(n)$, nên tổng thời gian là $\frac{5}{3}M(n)$.

7.4.2 Tính tích của nhiều nhị thức

Cho $a_1, a_2, \dots, a_n$, xuất hệ số của đa thức

$$(1 + a_1x)(1 + a_2x) \cdots (1 + a_nx).$$

Ta có thể giải trực tiếp bằng chia để trị. Cụ thể, đặt

$$F(l, r) = \prod_{i=l}^r (1 + a_i x),$$

và tính bằng

$$F(l, r) = F\left(l, \left\lfloor \frac{l+r}{2} \right\rfloor\right) F\left(\left\lfloor \frac{l+r}{2} \right\rfloor + 1, r\right).$$

Thời gian là

$$T(n) = 2T(n/2) + M(n/2) = \frac{1}{2}M(n) \log_2 n + O(M(n)).$$

7.4.3 Tổng lũy thừa

Cho $a_1, a_2, \dots, a_n$, đặt

$$f(k) = \sum_{i=1}^n a_i^k,$$

cần tính $f(0), f(1), \dots, f(n-1)$.

Xét chuỗi lũy thừa hình thức

$$F(z) = \sum_{k \geq 0} f(k) z^k = \sum_{k \geq 0} \sum_{i=1}^n a_i^k z^k = \sum_{i=1}^n \frac{1}{1 - a_i z}.$$

Tức là cần tính $F(z) \bmod z^n$.

Dễ biết

$$\prod_{i=1}^n (1 - a_i z) F(z) = \sum_{i=1}^n \prod_{j \neq i} (1 - a_j z).$$

Gọi hai vế là $A(z)F(z) = B(z)$, ta chỉ cần lần lượt tính $A, B \bmod z^n$, rồi thực hiện một lần chia chuỗi lũy thừa hình thức.

Ta định nghĩa $A(l, r), B(l, r)$ tương tự $F(l, r)$ ở mục 4.2. Khi đó A có thể được tính tương tự, còn

$$\begin{aligned} B(l, r) &= A\left(l, \left\lfloor \frac{l+r}{2} \right\rfloor\right) B\left(\left\lfloor \frac{l+r}{2} \right\rfloor + 1, r\right) \\ &\quad + B\left(l, \left\lfloor \frac{l+r}{2} \right\rfloor\right) A\left(\left\lfloor \frac{l+r}{2} \right\rfloor + 1, r\right). \end{aligned}$$

Tính bằng chia để trị như vậy, dễ thấy thời gian là $\frac{3}{2}M(n) \log_2 n + O(M(n))$.

7.4.4 Tính giá trị tại nhiều điểm và nội suy

Tính giá trị tại nhiều điểm theo cách truyền thống dùng một thuật toán chia để trị. Xét việc tính giá trị của b tại các điểm trong α . Khi $|b| \geq |\alpha|$, có thể lấy b modulo $(x - \alpha_0)(x - \alpha_1) \cdots (x - \alpha_{|\alpha|-1})$, nhờ đó luôn làm cho $|b| < |\alpha|$. Sau đó, mỗi lần ta chia tập điểm cần tính thành hai nửa và đệ quy xuống.

Thời gian của cách này bằng thời gian của mục 4.3 cộng với thời gian của mục 4.2 trong mỗi lời gọi chia để trị; sau khi tính toán nhận được $\frac{11}{3}M(n) \log_2 n + O(M(n))$. Với cách cài đặt nghịch đảo bằng phương pháp Newton phổ biến nhưng chưa tối ưu, tốc độ còn chậm hơn trên gấp đôi.

Tiếp theo, ta xét chuyển vị của ma trận Vandermonde:

$$V_a^T = \begin{bmatrix} 1 & 1 & \cdots & 1 \\ \alpha_0 & \alpha_1 & \cdots & \alpha_{n-1} \\ \alpha_0^2 & \alpha_1^2 & \cdots & \alpha_{n-1}^2 \\ \vdots & \vdots & \ddots & \vdots \\ \alpha_0^{n-1} & \alpha_1^{n-1} & \cdots & \alpha_{n-1}^{n-1} \end{bmatrix}.$$

Xét bài toán sau chuyển vị $V_a^T b = c$, khi đó

$$c_i = \sum_{j=0}^{n-1} \alpha_j^i b_j.$$

Bài toán này về cơ bản tương tự bài toán ở mục 4.3; sau một sửa đổi nhỏ sẽ thu được thuật toán chia để trị thời gian $\frac{2}{3}M(n) \log_2 n + O(M(n))$.

Viết thuật toán này trở lại phiên bản trước chuyển vị sẽ biến một phép nhân ở mỗi tầng chia để trị ban đầu thành hai phép nhân, dẫn tới thời gian tăng gấp đôi. Thời gian của thuật toán sau khi viết lại là $\frac{5}{2}M(n) \log_2 n + O(M(n))$.

Thuật toán nội suy chủ yếu gọi một lần tính giá trị tại nhiều điểm, nên cũng có cùng thời gian với tính giá trị tại nhiều điểm.

7.4.5 Thực hành

Cài đặt tính giá trị tại nhiều điểm của chúng tôi được nộp tại <https://www.luogu.com.cn/record/29053749>; trong bài đó, cấp độ của n là 2^{16} . Chương trình của chúng tôi là nhanh nhất vào thời điểm hiện tại (trước ngày 2020-01-08), và nhanh hơn gấp đôi so với chương trình nhanh nhất chưa tối ưu tăng thấp, phù hợp với tính toán lý thuyết ở trên.

Trước đây, tính giá trị tại nhiều điểm không thường gặp trong thi đấu, không nhất thiết vì nó ít hữu dụng, mà phần lớn vì tính giá trị tại nhiều điểm phải lồng ghép nhiều thuật toán, khó nhớ và khó viết; đồng thời bản thân thuật toán có hằng số rất lớn, người ra đề chưa chắc có thể dùng giới hạn thời gian hợp lý để loại lời giải thô, còn thí sinh sau khi vất vả viết xong lại có thể không qua được vì hằng số. Thuật toán cải tiến nói trên chỉ cần dùng chia để trị và nhân đa thức, hằng số cũng giảm đi nhiều, nên ở một mức độ nhất định đã mở rộng tính thực dụng của tính giá trị tại nhiều điểm.

7.5 Thuật toán nhanh cho một lớp ma trận đặc biệt nhân vector

7.5.1 Mô tả

Ta xét một biến đổi tuyến tính $A\alpha = \beta$. Bây giờ xét hàm sinh hai biến của các hệ số trong ma trận $n \times n$ là A :

$$A(x, y) = \sum_{i=0}^{n-1} \sum_{j=0}^{n-1} A_{i,j} x^i y^j.$$

Nếu $A(x, y)$ có thể biểu diễn thành

$$u(x)v(y)f(g(x)h(y)),$$

trong đó g và h được tạo bởi phép hợp của một dãy (số lượng hằng số) các hàm đơn giản sẽ được nêu dưới đây, thì ta có thể hoàn thành phép nhân bên trái của ma trận này và ma trận nghịch đảo của nó với vector trong thời gian $\tilde{O}(n)$. Cụ thể, nếu trong g, h có chứa $\exp, \log$ thì độ phức tạp là $O(M(n) \log n)$, nếu không thì là $O(M(n))$ (độ phức tạp này tốt hơn mọi phương pháp đã biết trước đó). Để bảo đảm các phép toán dưới đây có nghĩa, còn yêu cầu hạng tử hằng của gh là 0, hạng tử bậc nhất của g và h đều khác

0; hạng tử hằng của u và v khác 0, và mọi hạng tử của f đều khác 0 (nếu hoàn toàn không cần nghịch đảo thì có thể bỏ điều kiện này).

7.5.2 Phép hợp

Định nghĩa 6. Xét một đa thức F bậc $n - 1$ và một biến đổi tuyến tính:

$$F(G(x)) \bmod x^n,$$

trong đó $G(x)$ là hằng. Ta gọi phép tính này là hợp bên phải.

Phép tính này không có phương pháp tốt trong trường hợp tổng quát; nhưng nếu $G = g_1 \circ g_2 \circ \dots \circ g_k$, trong đó các g_i là một số hàm đơn giản, thì ta có thể dùng tính kết hợp của phép hợp: $A(B(C(x))) = A \circ B(C(x))$, rồi lần lượt hợp F với các hàm đơn giản k lần để tính ra kết quả. Một số ít hàm không đóng sẽ dùng tới thông tin của $C(x)$, cần đặc biệt chú ý. Nếu thỏa các điều kiện nêu trên, ngoài các tính toán đó ra, ở mỗi bước đều có thể cắt còn n vị trí mà không mất độ chính xác.

Ta xem các hàm đơn giản là những loại sau:

1. Phép cộng $x + k$.

Có

$$\begin{aligned} F(x + k) &= \sum_{i=0}^{n-1} F_i(x + k)^i = \sum_{i=0}^{n-1} F_i \sum_{j=0}^i \binom{i}{j} x^j k^{i-j} \\ &= \sum_{j=0}^{n-1} \frac{x^j}{j!} \sum_{i=j}^{n-1} i! F_i \frac{k^{i-j}}{(i-j)!}. \end{aligned}$$

Rõ ràng có thể hoàn thành trong thời gian $M(n)$.

2. Phép nhân kx . Chỉ cần nhân các lũy thừa của k vào hệ số, thời gian $O(n)$.
3. Lũy thừa x^k ($k \in \mathbb{Z}^+$). Chỉ cần biến đổi chỉ số, không thực hiện phép toán số học nào.
4. Nghịch đảo x^{-1} . Chú ý chuỗi lũy thừa không được có số mũ âm. Xét $F(x^{-1}) = (\text{rev}(n, F))(x)x^{1-n}$, trong đó x là chuỗi lũy thừa được hợp phía sau. Việc đảo hệ số không thực hiện phép toán số học nào; tính hệ số của nghịch đảo chuỗi lũy thừa phía sau cần thời gian $O(M(n))$.
5. Căn $x^{1/k}$ ($k \in \mathbb{Z}^+$). Ở đây cần bảo đảm căn bậc k của x là duy nhất; khi suy ra dạng $g(x)h(y)$ phía trước, cần kiểm tra một số hạng để loại bỏ nghiệm không hợp lệ.

Giả sử chuỗi lũy thừa phía sau tương ứng với x là g , và $g = h^k$ (ở đây cần bảo đảm h là duy nhất). Ta chia chỉ số của F có dư, F_{ik+j} , rồi phân loại theo modulo k :

$$F_i(x) = \sum_j F_{jk+i} h^j.$$

Khi đó

$$F(h) = \sum_i F_i(g) h^i.$$

Sau khi đệ quy thành n/k bài toán con kích thước k rồi ghép lại là được. Tính h cần thời gian $O(M(n))$ [1]; sau đó cần thời gian $O(kM(n))$ để tính các lũy thừa bậc $0, \dots, k - 1$ của h , vì vậy cần bảo đảm k là hằng số.

6. Hàm mũ $\exp(x) - 1$. Trừ 1 là để nó và logarit là nghịch đảo hợp của nhau; tạm thời có thể không xét phần trừ 1. Khi đó

$$\begin{aligned} F(e^x) &= \sum_{i=0}^{n-1} F_i e^{ix} = \sum_{i=0}^{n-1} F_i \sum_{j \geq 0} \frac{i^j}{j!} x^j \\ &= \sum_{j \geq 0} \frac{x^j}{j!} \sum_{i=0}^{n-1} F_i i^j. \end{aligned}$$

Ta xét việc tính

$$\sum_{j \geq 0} x^j \sum_{i=0}^{n-1} F_i i^j.$$

Tất nhiên kết quả cần được cắt còn n vị trí; sau đó chỉ cần chia mỗi vị trí cho $j!$.

Bài toán trên giống bài toán chuyển vị của tính giá trị tại nhiều điểm đã nhắc tới trước đó, nên có thể giải bằng chia để trị và nhân đa thức. Độ phức tạp là $O(M(n) \log n)$.

7. Logarit $\log(x + 1)$. Làm trực tiếp không dễ phân tích, nên ta xét nghịch đảo của thuật toán hợp với hàm mũ.

Thuật toán hợp với hàm mũ có thể tóm tắt là: chạy bài toán chuyển vị của tính giá trị tại nhiều điểm trên F với các điểm $0, 1, \dots, n-1$, rồi nhân mỗi vị trí với $1/j!$.

Chuyển vị thuật toán này nhận được: trước hết nhân mỗi vị trí với $1/j!$, rồi chạy tính giá trị tại nhiều điểm trên $0, 1, \dots, n-1$. Nghịch đảo của nó hiển nhiên là chạy nội suy nhiều điểm rồi nhân mỗi vị trí với $j!$. Sau đó chuyển vị ngược lại, độ phức tạp không đổi, nên là $O(M(n) \log n)$.

Một điểm tốt hơn nữa là mọi hàm ở trên đều có nghịch đảo hợp.

7.5.3 Thuật toán

Ta quay lại bài toán ban đầu. Trước hết giả sử $u = v = 1$, ta có

$$F_{i,j} = \sum_k g_i^k f_k h_j^k.$$

Khi đó biến đổi tuyến tính này là hợp của ba biến đổi tuyến tính: hợp bên phải với g , nhân từng điểm với f , rồi chuyển vị của hợp bên phải với h . Chỉ cần tính tuần tự theo phương pháp ở mục trước.

Sau khi thêm u, v , biến đổi tuyến tính ban đầu trở thành hợp của ba biến đổi tuyến tính: nhân với u , biến đổi tuyến tính ban đầu, rồi chuyển vị của phép nhân với v ; cũng có thể tính dễ dàng.

Trong các điều kiện giới hạn, mọi thao tác trên đều có nghịch đảo và có thể lấy nghịch đảo trực tiếp, nên ma trận nghịch đảo cũng có thể được tính tương ứng. Tóm lại, ta đã giải quyết lớp bài toán này với độ phức tạp rất thấp.

7.6 Ứng dụng

7.6.1 Đổi cơ sở

Định nghĩa 7. Xét một dãy đa thức $\varphi_0(x), \varphi_1(x), \dots, \varphi_n(x)$. Nếu mọi đa thức bậc n đều có thể được biểu diễn tuyến tính bằng chúng, thì đây là một cơ sở của không gian đa thức modulo x^n . Các cơ sở thường gặp trong OI có cơ sở đơn thức và cơ sở giai thừa giảm.

Một đa thức có thể được biểu diễn dưới những cơ sở khác nhau. Trong đại số tuyến tính, ta biết rằng việc chuyển từ biểu diễn theo một cơ sở sang cơ sở khác tương ứng với một biến đổi tuyến tính. Đối cơ sở đa thức thường tương ứng với bài toán đã nhắc ở mục 5. Ta có các kết quả sau cho việc chuyển đổi qua lại giữa cơ sở đơn thức và các cơ sở khác:

- Có thể tính trong thời gian $O(M(n))$: đa thức Laguerre, đa thức Hermite, đa thức Jacobi, đa thức Fibonacci, đa thức Euler, đa thức Bernoulli, đa thức Mott, đa thức Spread, đa thức Bessel.
- Có thể tính trong thời gian $O(M(n) \log n)$: giai thừa giảm, đa thức Bell, đa thức Bernoulli loại hai, đa thức Poisson–Charlier, đa thức Actuarial, đa thức Narumi, đa thức Peters, đa thức Meixner–Pollaczek, đa thức Meixner, đa thức Krawtchouk.

Ứng dụng đẹp của nhiều cơ sở trong danh sách trên vào OI vẫn còn chờ được khai phá.

7.6.2 Bài ví dụ

Con đường đời chân thật và vô vọng của họ. Nguồn bài: Comet OJ #2 F.

Ý chính đề bài: Có n ($n \leq 10^5$) vật phẩm; vật phẩm thứ i ($1 \leq i \leq n$) có thuộc tính p_i ($0 < p_i \leq 1$). Cấp độ ban đầu của nhân vật chính là 0. Dùng vật phẩm thứ i có xác suất p_i làm cấp độ tăng 1, và xác suất $1 - p_i$ giữ nguyên. Nếu cấp độ cuối cùng là j thì sinh ra lực tấn công a_j . Gọi f_i là kỳ vọng lực tấn công của nhân vật chính sau khi dùng $n - 1$ vật phẩm trừ vật phẩm thứ i . Cần tính $f_1, f_2, \dots, f_n$ trong $F_{998244353}$.

Lời giải: Đặt

$$C_i(x) = 1 - p_i + p_i x, \quad C(x) = \prod_i C_i(x).$$

Khi đó tổng các hệ số của tích vô hướng giữa C/C_i và a chính là f_i .

Xem a là vector đầu vào, đây là bài toán biến đổi tuyến tính $Aa = f$, trong đó $A_{i,j} = (C/C_i)[x^j]$. Xét bài toán chuyển vị $A^T a = g$, ta có

$$g = \sum_i a_i C/C_i.$$

Biểu thức này có thể được tính bằng chia để trị và nhân đa thức tương tự bài toán tổng lũy thừa.

Bước đi ngẫu nhiên. **Ý chính đề bài:** Cho n, a, b ($n \leq 10^5$). Trên một mặt phẳng hai chiều vô hạn, một người xuất phát tại $(0, 0)$; mỗi bước có thể đi theo các vector $(1, 1)$, $(1, 0)$, và $(2, 0)$, với trọng số lần lượt là $1, a, b$. Người này có thể dừng lại bất cứ lúc nào. Nếu dừng ở (x, y) thì sinh ra trọng số w_y . Trọng số của một đường đi được định nghĩa là tích các trọng số đó. Với mọi $1 \leq a \leq n$, cần tính tổng trọng số của mọi đường đi kết thúc tại các điểm có tọa độ x bằng a , ký hiệu là g_a , trong $F_{998244353}$.

Lời giải: Không xét w_y , gọi $f_{x,y}$ là tổng trọng số đường đi tới (x, y) , với $f_{0,0} = 1$, và ngoài ra

$$f_{i,j} = f_{i-1,j-1} + af_{i-1,j} + bf_{i-2,j} \quad (i, j \geq 0),$$

còn các $f_{i,j}$ khác bằng 0. Khi đó

$$g_a = \sum_y f_{a,y} w_y,$$

đây là một bài toán biến đổi tuyến tính.

Đặt hàm sinh hai biến của f là

$$F(x, y) = \sum_{i,j} f_{i,j} x^i y^j.$$

Khi đó phương trình $F = 1 + x(a + y)F + x^2 F$ đúng, suy ra

$$F = \frac{1}{1 - (a + y)x - x^2} = \frac{1}{(1 - x^2) \left(1 - \frac{a+y}{1-x^2}\right)}.$$

Công thức này có thể chuyển thành dạng ở mục 5, trong đó

$$u(x) = \frac{1}{1 - x^2}, \quad v(y) = 1, \quad f(x) = \frac{1}{1 - x}, \quad g(x) = \frac{1}{1 - x^2}, \quad h(y) = a + y.$$

Áp dụng thuật toán là nhận được $O(M(n))$; trên thực tế đây chính là bài toán đổi cơ sở sang một lớp đa thức trực giao.

Đếm cây nhị phân. Ý chính đề bài: Cho n ($n \leq 10^5$). Định nghĩa trọng số của một cây nhị phân có x lá là w_x . Gọi $f(t)$ là tổng trọng số của mọi cây nhị phân không gán nhãn có t đỉnh. Hãy tính $f(1), f(2), \dots, f(n)$ trong $F_{998244353}$.

Lời giải: Tương tự trên, gọi $f_{i,j}$ là số cây nhị phân có i đỉnh và j lá, F là hàm sinh hai biến của nó. Khi đó có thể lập

$$F = x(y + 2F + F^2),$$

suy ra

$$\begin{aligned} F &= \frac{1 - 2x - \sqrt{(2x - 1)^2 - 4x^2y}}{2x} \\ &= \frac{1}{2x}(1 - 2x) \left(1 - \sqrt{1 - \left(\frac{1}{2x - 1} + 1 \right)^2 y} \right). \end{aligned}$$

Ta đặt

$$u(x) = 1 - 2x, \quad v(y) = 1, \quad f(x) = 1 - \sqrt{1 - x}, \quad g(x) = \left(\frac{1}{2x - 1} + 1 \right)^2, \quad h(y) = y.$$

Áp dụng thuật toán là có thể tính phần bên trong; bên ngoài nhân với $\frac{1}{2x}$ và xử lý đơn giản là xong, độ phức tạp $O(M(n))$.

7.7 Tổng kết

Bài viết này khảo sát nguyên lý chuyển vị và các ứng dụng của nó. Nguyên lý chuyển vị đưa ra một cách nhìn mới để xử lý các bài toán biến đổi tuyến tính: xem xét sự thay đổi của ma trận từ góc độ thay đổi thuật toán. Dùng nguyên lý chuyển vị, ta tối ưu mạnh hằng số và độ khó lập trình của tính giá trị tại nhiều điểm, đồng thời hé lộ bản chất chung bên trong nhiều bài toán kinh điển tưởng như khác nhau. Ở phần cuối, bài viết giải quyết một lớp khá rộng các bài toán ma trận nhân vector, khiến nhiều bài toán đếm vốn khó trở nên dễ xử lý; những mặt này đều cho thấy phạm vi ứng dụng rộng, sức mạnh lớn và tiềm năng vô hạn của nguyên lý chuyển vị. Ngoài các nội dung được nhắc tới trong bài, vẫn còn những cách vận dụng khéo léo khác của nguyên lý chuyển vị đang chờ độc giả khai phá. Hy vọng bài viết có thể gợi mở tư duy cho độc giả, mở rộng hướng ra đề và giải đề.

7.8 Lời cảm ơn

Cảm ơn China Computer Federation đã cung cấp nền tảng trao đổi và học tập.

Cảm ơn huấn luyện viên đội tuyển quốc gia Gao Wenyuan vì sự hướng dẫn.

Cảm ơn thầy Ye Guoping vì sự giúp đỡ trong học tập và cuộc sống.

Cảm ơn các anh, bạn học Zhu Zhenting, Zhou Yuyang và những người khác vì sự chỉ dẫn.

Cảm ơn các bạn Luo Kai, Li Baitian, Zhang Haofeng và những người khác đã đọc bản thảo bài viết này.

Tài liệu tham khảo

1. Về hằng số trong phương pháp Newton tối ưu tính toán chuỗi lũy thừa hình thức. <http://negiizhao.blog.uoj.ac/blog/4671>.

2. A. Bostan, G. Lecerf, và E. Schost. *Tellegen's Principle into Practice*. 2005.
3. Alin Bostan, Bruno Salvy, và Eric Schost. *Power Series Composition and Change of Basis*. 2008.

8 Bàn về duy trì động giá trị cực trị của hàm số

Li Baitian, Trường Trung học trực thuộc Đại học Bắc Kinh

Tóm tắt

Bằng cách đưa vào thảo luận về cấu trúc đường bao trên của nhiều hàm số, bài viết này chủ yếu cải tiến một số bài toán nổi tiếng trong OI như sau:

- Cộng vào một đoạn của dãy một cấp số cộng có công sai dương, và truy vấn giá trị cực trị trên đoạn. Cách làm vốn được biết rộng rãi cho bài toán này là $\Theta(\sqrt{n})$, còn cách làm trong bài viết này hiện có cận trên $O(\log^2 n)$.
- Cây đoạn Li Chao có thể thêm một đoạn thẳng trong mặt phẳng trong $\Theta(\log^2 n)$, và truy vấn giá trị y lớn nhất tại một vị trí x trong $\Theta(\log n)$. Bài viết này dùng một phương pháp hoàn toàn khác để giữ nguyên độ phức tạp truy vấn, đồng thời cải tiến độ phức tạp thêm một đoạn thẳng thành độ phức tạp lý thuyết trung bình $\Theta(\alpha(n) \log n)$. Hơn nữa, cách làm này có thể mở rộng một cách tổng quát cho các hàm bậc cao.

8.1 Tổng quan

Với dãy hàm $f_i : \mathbb{R} \rightarrow \mathbb{R}$, ta muốn hỗ trợ một số thao tác sửa đổi trên dãy hàm, và ít nhất phải hỗ trợ truy vấn cực trị toàn cục: cho x , hỏi

$$\max_{j=1}^n f_j(x).$$

Trong phân tích tiếp theo, ta cần đưa vào khái niệm dãy Davenport–Schinzel.

8.1.1 Dãy Davenport–Schinzel

Định nghĩa 1. Gọi một dãy độ dài m , $\sigma_1, \sigma_2, \dots, \sigma_m$, là một dãy Davenport–Schinzel (n, s) , viết tắt là dãy $DS(n, s)$, khi và chỉ khi σ_i là số nguyên từ 1 đến n , đồng thời thỏa:

- Hai phần tử kề nhau trong σ có giá trị khác nhau.
- Với mọi $x \neq y$, nếu một dãy luân phiên tạo bởi x, y là dãy con của σ , thì độ dài của nó không vượt quá $s + 1$.

Định lý sau mô tả dãy DS một cách đặc biệt hữu hiệu.

Định lý 1. Ký hiệu $\lambda_s(n)$ là độ dài lớn nhất có thể của một dãy $DS(n, s)$. Khi đó

$$\lambda_s(n) = \begin{cases} n, & s = 1, \\ 2n - 1, & s = 2, \\ 2n\alpha(n) + O(n), & s = 3, \\ \Theta(n2^{\alpha(n)}), & s = 4, \\ \Theta(n\alpha(n)2^{\alpha(n)}), & s = 5, \\ n2^{\alpha(n)^t/t! + O(\alpha(n)^{t-1})}, & s \geq 6, \quad t = \left\lfloor \frac{s-2}{2} \right\rfloor. \end{cases}$$

Ở đây $\alpha(n)$ là hàm Ackermann ngược. Từ tốc độ tăng của nó, ta biết rằng với mọi hằng số s , $\lambda_s(n)$ đều gần như tuyến tính.

Sau đây ta chứng minh hai trường hợp s nhỏ.

8.1.2 Chứng minh trường hợp $s = 1$

Giả sử trong dãy tồn tại hai phần tử bằng nhau $\sigma_i = \sigma_j$ ($i < j$). Khi đó $\sigma_{i+1} \neq \sigma_i$, và $\sigma_i, \sigma_{i+1}, \sigma_j$ tạo thành một dãy con luân phiên độ dài 3, mâu thuẫn.

Vì vậy dãy không có phần tử lặp. Do các phần tử của σ đều là số nguyên dương từ 1 đến n , ta có $m \leq n$. Mọi hoán vị của $1, \dots, n$ đều đạt dấu bằng, nên $\lambda_1(n) = n$.

8.1.3 Chứng minh trường hợp $s = 2$

Ta chứng minh bằng quy nạp mạnh. Với $n = 1$, $\lambda_2(1) = 1$ là hiển nhiên.

Xét $n > 1$. Khi đó với mọi $n' < n$ đều đã có $\lambda_2(n') = 2n' - 1$. Xét một dãy $DS(n, 2)$, không mất tính tổng quát giả sử $\sigma_1 = 1$.

- Nếu 1 chỉ xuất hiện một lần, thì $m \leq 1 + \lambda_2(n-1) = 2n-2 \leq 2n-1$.
- Giả sử vị trí tiếp theo mà 1 xuất hiện là i . Với mọi phần tử σ_j có $1 < j < i$, do không tồn tại dãy con luân phiên độ dài 4, phần tử đó sẽ không xuất hiện lại ở vị trí $> i$. Giả sử trong các vị trí từ 2 đến $i-1$ có tổng cộng k loại phần tử, thì các vị trí từ 2 đến $i-1$ tạo thành một dãy $DS(k, 2)$, còn phần tử i trở đi tạo thành một dãy $DS(n-k, 2)$. Vì vậy

$$m \leq 1 + \lambda_2(k) + \lambda_2(n-k) = 1 + 2k - 1 + 2(n-k) - 1 = 2n - 1.$$

Mặt khác, $m = 2n - 1$ dễ đạt được bằng dãy có dạng

$$1, 2, \dots, n-1, n, n-1, \dots, 2, 1.$$

Tổng hợp lại, $\lambda_2(n) = 2n - 1$ được chứng minh bằng quy nạp.

8.1.4 Liên hệ giữa dãy DS và bài toán

Ta xem s như một đại lượng phần nào đo được “độ phức tạp” của họ hàm cần duy trì. Chú ý rằng với một họ hàm ta đang duy trì, nếu với mọi hai hàm f, g , số đoạn của $\max(f(x), g(x))$ không vượt quá $s + 1$, thì cách chia đoạn của đường bao trên của n hàm trong họ đó tương đương với một dãy $DS(n, s)$. Ta gọi họ hàm này là luân phiên bậc s . Vì vậy độ dài chia đoạn của đường bao trên tối đa chỉ là $\lambda_s(n)$.

Trường hợp thường gặp nhất là các đa thức bậc không quá s . Vì hai đa thức bậc s có nhiều nhất s giao điểm, số đoạn của cực trị không vượt quá $s + 1$. Do đó, với n đa thức bậc không quá s , độ dài chia đoạn của đường bao trên không vượt quá $\lambda_s(n)$.

Một trường hợp khác đáng chú ý là đa thức bậc s được định nghĩa trên một đoạn. Ta có thể định nghĩa hàm từng đoạn như sau: ngoài miền xác định ban đầu, giá trị hàm đều là $-\infty$. Như vậy hai hàm từng đoạn bậc s bất kỳ là luân phiên bậc $s + 2$, nên độ dài chia đoạn của đường bao trên của n hàm từng đoạn bậc s không vượt quá $\lambda_{s+2}(n)$.

Trong các bài toán đã được nghiên cứu trước đây trong OI, thường gặp nhất là hàm bậc nhất và hàm bậc nhất từng đoạn; hai bài toán này lần lượt có $s = 1$ và $s = 3$.

Trong phần thảo luận tiếp theo, ta sẽ thêm các ràng buộc đặc biệt khác nhau lên bài toán, và trên các ràng buộc đó thiết kế được những thuật toán khá hiệu quả dựa trên cận trên của $\lambda_s(n)$.

1. Không sửa hàm, chỉ duy trì chúng như một tập hợp. Dưới ràng buộc này có thể thiết kế bằng ý tưởng nhóm nhị phân.
2. Các giá trị x trong truy vấn được bảo đảm tăng đơn điệu. Dưới ràng buộc này có thể thiết kế bằng ý tưởng tương tự Segment Tree Beats.

8.2 Duy trì tập hàm

Cho n hàm $f_i : \mathbb{R} \rightarrow \mathbb{R}$. Với mỗi x_k trong $x_1, x_2, \dots, x_m$, cần tính

$$\max_{j=1}^n f_j(x_k).$$

Khi mọi hàm đã được xác định ngay từ đầu, ta có thể chia để trị: mỗi lần chia tập hàm hiện tại thành hai phần để xử lý, rồi gộp hai nhóm hàm từng đoạn thu được. Độ phức tạp thỏa

$$T(n) = 2T\left(\frac{n}{2}\right) + \Theta(\lambda_s(n)).$$

Theo định lý chính, suy ra $T(n) = \Theta(\lambda_s(n) \log n)$. Thời gian trả lời là $\Theta(m \log n)$.

Với trường hợp các hàm được xác định lần lượt, ta có thể dùng nhóm nhị phân.

Nếu chèn n hàm bậc không quá s , ta có thể duy trì trong tổng thời gian $\Theta(\lambda_s(n) \log n)$ và tổng không gian $\Theta(\lambda_s(n))$; cách cài đặt mộc mạc trả lời mỗi truy vấn trong $\Theta(\log^2 n)$. Thực ra ta còn có thể trả lời mỗi truy vấn trong $\Theta(\log n)$.

So với trường hợp $s = 3$, thuật toán này hỗ trợ chèn đoạn thẳng với thời gian trung bình $\Theta(\alpha(n) \log n)$, còn truy vấn đạt $\Theta(\log n)$. Độ phức tạp này tốt hơn cây đoạn Li Chao. Ngoài ra còn có một ưu điểm phụ: thuật toán này không cần biết trước các giá trị x sẽ được truy vấn, mà hoàn toàn trực tuyến.

8.2.1 Cài đặt mộc mạc

Đặt $k = \lfloor \log_2 n \rfloor$, phân rã nhị phân của n là

$$\sum_{j=0}^k n_j \cdot 2^j \quad (n_j \in \{0, 1\}).$$

Với mỗi j có $n_j = 1$, ta duy trì một khối L_j , gồm các đoạn tạo nên đường bao trên của 2^j hàm trong khối đó. Mỗi khi thêm một hàm, nếu tồn tại hai khối chứa cùng số hàm, ta gộp chúng lại, bằng cách dùng các đoạn của hai khối ban đầu để tính các đoạn sau khi gộp. Khi quá trình gộp kết thúc, kích thước các khối còn lại đúng bằng phân rã nhị phân của n .

Toàn bộ quá trình có thể xem như thực hiện không đầy đủ một phép chia để trị cỡ 2^{k+1} , nên độ phức tạp duy trì là $\Theta(\lambda_s(2^k)k) = \Theta(\lambda_s(n) \log n)$.

Với truy vấn, cách một cách là tìm kiếm nhị phân trên mỗi khối để tìm đoạn chứa tọa độ truy vấn. Độ phức tạp thời gian là $\Theta(\log^2 n)$.

8.2.2 Fractional cascading

Bằng kỹ thuật fractional cascading, ta có thể tối ưu độ phức tạp truy vấn.

Ta duy trì các mảng phụ $T_k = L_k$. Với $0 \leq j < k$, lấy mọi vị trí bội của 3 trong T_{j+1} , rồi trộn với L_j để thu được T_j . Đồng thời, trong T_j ghi lại nguồn của mỗi phần tử và tiền nhiệm/hậu nhiệm theo từng nguồn khác nhau; nhờ đó, từ vị trí lowerbound trong T_j , ta có thể tìm lowerbound trong L_j và trong T_{j+1} trong $\Theta(1)$. Vì vậy độ phức tạp truy vấn là $\Theta(\log n)$.

Do $|L_j| \leq \lambda_s(2^j)$, và $\lambda_s(n) = o(n^{1+\epsilon})$, ta có thể chặn độ dài của T_j như sau:

$$\begin{aligned} |T_j| &\leq \sum_{t=j}^k \frac{\lambda_s(2^t)}{3^{t-j}} \\ &= \sum_{t=0}^{k-j} \Theta\left(\lambda_s\left(\frac{2^{t+j}}{3^t}\right)\right) \\ &= \Theta\left(\lambda_s\left(\sum_{t=0}^{k-j} \frac{2^{t+j}}{3^t}\right)\right) \\ &= \Theta(\lambda_s(3 \cdot 2^j)) \\ &= \Theta(\lambda_s(2^j)). \end{aligned}$$

Khi một lần cộng nhử lên tới bit thứ j , độ phức tạp dựng lại là

$$\sum_{t=0}^j \Theta(\lambda_s(2^t)) = \Theta(\lambda_s(2^j)).$$

Vì vậy tổng độ phức tạp duy trì vẫn là $\Theta(\lambda_s(n) \log n)$.

8.2.3 Ứng dụng

Duy trì cực trị của hàm bậc nhất từng đoạn. Khi cấu trúc dữ liệu này được dùng để duy trì hàm bậc nhất từng đoạn, thay $s = 1$ vào ta được tổng độ phức tạp duy trì

$$\Theta(\lambda_{1+2}(n) \log n) = \Theta(n \alpha(n) \log n).$$

Về mặt lý thuyết, độ phức tạp này tốt hơn độ phức tạp $\Theta(\log^2 n)$ để thêm một đoạn thẳng của cây đoạn Li Chao trước đây. Đáng nói là tồn tại cách dựng khiến số đoạn trên đường bao trên của n đoạn thẳng đạt bậc $\Theta(n \alpha(n))$. Vì vậy phân tích độ phức tạp ở trên là chặt.

Bài toán quy hoạch động. Với bài toán quy hoạch động có dạng

$$a_i = \max_{0 \leq j < i} a_j + w_{j,i},$$

nếu $a_j + w_{j,i}$ có thể viết lại thành dạng $f_j(x_i)$, và họ hàm f_i có bậc luân phiên thấp, thì ta có thể dùng cấu trúc trên để tối ưu chuyển trạng thái.

Trong các bài toán kiểu này trước đây, thường cần viết lại bài toán thành dạng hàm bậc nhất, hoặc dựa vào tính đơn điệu của quyết định.

Thực ra có thể chỉ ra rằng cả hai trường hợp đều là những trường hợp đặc biệt khi hàm luân phiên bậc $s = 1$. Với trường hợp viết lại được thành hàm bậc nhất thì điều này hiển nhiên. Với tính đơn điệu của quyết định, ta thường yêu cầu w thỏa bất đẳng thức tứ giác, từ đó thu được

$$\begin{aligned} \forall i < j, \quad w_{i,j+1} + w_{i+1,j} &\leq w_{i,j} + w_{i+1,j+1}, \\ a_i + w_{i,j+1} + a_{i+1} + w_{i+1,j} &\leq a_i + w_{i,j} + a_{i+1} + w_{i+1,j+1}, \\ f_i(j+1) + f_{i+1}(j) &\leq f_i(j) + f_{i+1}(j+1), \\ \Delta f_i &\leq \Delta f_{i+1}, \\ 0 &\leq \Delta(f_{i+1} - f_i). \end{aligned}$$

Điều này cho thấy với những bài toán có thể tối ưu bằng tính đơn điệu quyết định thông qua bất đẳng thức tứ giác, họ hàm f ứng với $i \leq i_0$ trên miền xác định $j \geq i_0$ là luân phiên bậc $s = 1$.

Do đó, cấu trúc dữ liệu trên tương thích tốt với một số điều kiện quen thuộc trước đây khi tối ưu quy hoạch động. Tuy trong các bài toán đơn điệu quyết định độ phức tạp không đạt tối ưu, cấu trúc dữ liệu trên lại xử lý được các hàm chi phí chuyển trạng thái có tính chất phức tạp hơn đôi chút.

8.3 Điểm truy vấn tăng đơn điệu

Cho n hàm $f_i : \mathbb{R} \rightarrow \mathbb{R}$. Với mỗi x_k trong $x_1 < x_2 < \dots < x_m$, cần tính

$$\max_{j=1}^n f_j(x_k).$$

Trong phần thảo luận tiếp theo, ta giả sử họ hàm cần xử lý là luân phiên bậc s , và có thể tìm vị trí luân phiên trong thời gian hằng số.

8.3.1 Cây Kinetic Tournament

Ta xét một cây đoạn, trong đó mỗi lá biểu diễn một hàm, và mỗi đỉnh của cây lưu hàm trong các lá của cây con đạt giá trị lớn nhất tại x hiện tại. Khi x tăng, ta cần liên tục sửa giá trị tại một số đỉnh. Ta duy trì giá trị x đầu tiên lớn hơn hiện tại khiến một hàm đang được lấy bởi một phép max trong cây con bị thay thế; khi truy vấn tiếp theo vượt quá giá trị x đó, ta đệ quy đi xuống và đặt lại phần xảy ra thay thế. Cây đoạn duy trì thông tin kiểu này được gọi là cây Kinetic Tournament. Sau đây ta viết tắt là KTT.

Xét số lần mỗi đỉnh trên cây đoạn bị sửa. Đại lượng này tương đương với số đoạn trên đường bao của mọi hàm trong cây con đó, nên tổng số lần sửa của mọi đỉnh, tương tự chia để trị, là $\Theta(\lambda_s(n) \log n)$. Vì vậy nếu duy trì trực tiếp trên cây đoạn, do mỗi lần sửa cần đi xuống tới lá tương ứng, độ phức tạp của thuật toán là $\Theta(\lambda_s(n) \log^2 n)$. Kết quả này kém hơn chia để trị trực tiếp, nhưng ta sẽ thấy tiềm năng của phương pháp này trong bài toán mạnh hơn.

8.3.2 Bài toán cực trị của dãy hàm có sửa đổi

Phiên bản có sửa đổi của bài toán cho phép gán lại trực tuyến một hàm nào đó trong quá trình xử lý. Giả sử có tổng cộng m lần sửa và q lần truy vấn.

Ta xét dùng KTT để duy trì. Khi gán lại một hàm, ta sửa đỉnh tương ứng trên cây đoạn, rồi cập nhật một chuỗi đỉnh lên tới gốc.

Độ phức tạp duy trì là bao nhiêu? Ta xét phép biến đổi tương đương sau cho quá trình duy trì: giả sử lá thứ i bị sửa k_i lần, ta thay lá đó bằng $k_i + 1$ đỉnh đặt bên dưới nó; đỉnh đầu tiên là hàm ban đầu, các đỉnh sau là k_i hàm còn lại, và tất cả được xem như các hàm từng đoạn theo khoảng thời gian chúng tồn

tại. Khi đó, với một đỉnh quản lý đoạn $[l, r]$, số đoạn trên đường bao trên trong cây con của nó không vượt quá $\lambda_{s+2}(r - l + 1 + \sum_{i=l}^r k_i)$. Vì vậy tổng số đoạn đường bao ở cùng một tầng không vượt quá

$$\begin{aligned} \sum_{[l,r]} \lambda_{s+2} \left(r - l + 1 + \sum_{i=l}^r k_i \right) &\leq \lambda_{s+2} \left(\sum_{[l,r]} \left(r - l + 1 + \sum_{i=l}^r k_i \right) \right) \\ &= \lambda_{s+2}(n + m). \end{aligned}$$

Qua phân tích trên, với truy vấn cực trị có sửa đổi trực tuyến, KTT có thể hoàn thành tính toán trong thời gian

$$\Theta(\lambda_{s+2}(n + m) \log^2 n + q).$$

Ta sẽ thấy cấu trúc này được dùng như thế nào để tối ưu một bài toán kinh điển liên quan tới hàm bậc nhất.

8.4 Mở rộng cho trường hợp tuyến tính

Do hàm tuyến tính có tính chất tốt, ta có thể nghiên cứu một số bài toán mở rộng trên chúng. Đáng tiếc là tác giả tạm thời chưa xác nhận được liệu các bài toán duy trì tương tự trong trường hợp bậc cao hơn có độ phức tạp gần như vậy hay không.

8.4.1 Bài toán cực trị của dãy có hai loại sửa đoạn

Với các dãy k_i, b_i , ta có hai loại thao tác sửa sau:

- Cho l, r, x . Với $l \leq i \leq r$, đặt $b_i \leftarrow k_i x + b_i$.
- Cho l, r, c, k', b' . Với $l \leq i \leq r$, đặt $k_i \leftarrow c k_i + k'$, $b_i \leftarrow c b_i + b'$.

Ngoài ra cần truy vấn cực trị của b_i trên một đoạn.

Cách làm cho bài toán này được cải tiến từ ý tưởng của Daniel Zhang.

Bài toán này xuất hiện một phần trong một số đề OI; trước đây các bài đó về cơ bản chỉ có cách làm chia khối với độ phức tạp $\Theta(n + (m + q)\sqrt{n})$. Ta sẽ thấy bằng KTT có thể hoàn thành thao tác và truy vấn trong

$$O(n \log^2 n + m \log^3 n + q \log n),$$

với ràng buộc bổ sung là thao tác sửa loại thứ nhất bảo đảm $x > 0$, và để trình bày ý chính, ở đây chỉ xét trường hợp $c > 0$.

Ý tưởng cụ thể khá đơn giản: trên cây đoạn, lưu trữ lười biểu diễn hiệu ứng tích lũy của thao tác 2 và tổng x tích lũy của thao tác 1. Chú ý rằng thao tác 1 về bản chất là bắt đầu thực hiện thao tác “ x tăng” trên một cây con nào đó của KTT, thay vì chỉ bắt đầu từ gốc như KTT nguyên bản.

Phân tích độ phức tạp. Ta phân tích độ phức tạp của thuật toán này bằng thế năng.

Định nghĩa một tập P gồm các đỉnh không phải lá. Với một đỉnh không phải lá v , nếu đường thẳng tương ứng với con đang giữ giá trị lớn hơn của v có hệ số góc nhỏ hơn hẳn con còn lại, thì $v \in P$.

Định nghĩa hàm thế

$$\Phi = \sum_{v \in P} d(v),$$

trong đó $d(v)$ là độ sâu của đỉnh v trên cây đoạn, và độ sâu của gốc là 1.

Xét trường hợp xấu nhất của thao tác 1, tức mỗi thao tác cập nhật đỉnh được thực hiện đơn lẻ. Ta định nghĩa chi phí của một thao tác cập nhật là 1, còn chi phí khấu hao của nó là $1 + \Delta \Phi$. Với đỉnh bị sửa v , ta

biết v chắc chắn chuyển từ thuộc P sang không thuộc P , còn cha p của v có thể chuyển từ không thuộc P sang thuộc P . Do đó

$$\begin{aligned}\hat{c} &= 1 + \Delta\Phi \\ &\leq 1 - d(v) + d(p) \\ &= 1 - d(v) + (d(v) - 1) \\ &= 0.\end{aligned}$$

Định nghĩa này khiến những thao tác cập nhật mà ban đầu ta không xác định được số lần phát sinh không cần được xét trong chi phí khấu hao. Tiếp theo xét thao tác 1 và thao tác 2; về bản chất chúng tương tự nhau. Chú ý rằng khi thẻ lười của một đỉnh bị sửa, hoặc các số trong cây con lấy đỉnh đó làm gốc bị cập nhật, cha của đỉnh này có thể chuyển từ không thuộc P sang thuộc P , làm thế năng tăng nhiều nhất $d(p) = \Theta(\log n)$. Một thao tác ảnh hưởng tới $\Theta(\log n)$ đỉnh, nên một thao tác làm thế năng tăng nhiều nhất $\Theta(\log^2 n)$.

Còn một điểm cần chú ý: do

$$\sum \hat{c}_i = \sum c_i + \Phi_t - \Phi_s,$$

ta phải cộng thêm $\Phi_s - \Phi_t = \Theta(n \log n)$ vào tổng chi phí, vì trường hợp xấu nhất là ban đầu mọi đỉnh đều thuộc P .

Tổng hợp lại, trong quá trình thao tác nhiều nhất sẽ phát sinh $\Theta(n \log n + m \log^2 n)$ lần cập nhật, nên độ phức tạp có cận trên

$$O(n \log^2 n + m \log^3 n + q \log n).$$

Trường hợp đặc biệt. Trong các bài toán như “cộng vào đoạn một cấp số cộng có công sai dương”, ta rút ra được một điều kiện ẩn: dãy k tăng đơn điệu. Vì vậy các thao tác cập nhật được phát sinh chắc chắn là một đỉnh chuyển lựa chọn max từ con trái sang con phải.

Khi đó ta định nghĩa thế năng Φ là số đỉnh có “max nằm ở con trái”. Vì quá trình thao tác nhiều nhất phát sinh $\Theta(n + m \log n)$ lần cập nhật, ta có cận trên độ phức tạp

$$O(n \log n + m \log^2 n + q \log n).$$

8.4.2 Ví dụ

Ví dụ 1. Bear and Bowling. Nguồn bài: Codeforces 573E, <https://codeforces.com/problemset/problem/573/E>.

Ở đây chỉ thảo luận một lời giải liên quan tới bài viết này. Trong lời giải đó, ta cần lấy lần lượt từng số khỏi một dãy a_n theo một thứ tự nào đó. Giả sử bên trái số được lấy đã có $k_i - 1$ số bị lấy, còn tổng các số đã bị lấy ở bên phải là s_i . Mỗi lần, số được lấy bắt buộc là số làm $k_i \cdot a_i + s_i$ lớn nhất.

Nếu dùng cấu trúc dữ liệu nói trên, chỉ cần ban đầu đặt $b_i = a_i$. Mỗi lần lấy một số, ta cộng $-\infty$ vào b_i , cộng a_i vào đoạn $[1, i - 1]$, và thực hiện $b_j \leftarrow b_j + a_j$ trên đoạn $[i + 1, n]$.

Ví dụ 2. Innophone. Nguồn bài: ROI 2018, <https://loj.ac/problem/2845>.

Cho một số cặp x_i, y_i , chọn a, b để cực đại hóa

$$\sum_{i=1}^n [a \leq x_i]a + [a > x_i][b \leq y_i]b.$$

Ta sắp xếp theo y , trong quá trình quét thì tăng hệ số góc trên đoạn trong KTT, rồi truy vấn theo giá trị y tương ứng là được.

8.4.3 Tổng đoạn con liên tiếp lớn nhất

Ý chính đề bài: Với một dãy a_n , hỗ trợ cộng một số dương vào toàn bộ một đoạn, và truy vấn tổng đoạn con lớn nhất trên một đoạn.

Lời giải: Đây là một bài toán khá phức tạp. Trước hết nhắc lại cách làm khi không có thao tác sửa. Trên mỗi đỉnh của cây đoạn, ta có thể duy trì bốn giá trị $sum, lmax, rmax, totmax$, với cách gộp:

$$\begin{aligned} sum &= ls.sum + rs.sum, \\ lmax &= \max(ls.lmax, ls.sum + rs.lmax), \\ rmax &= \max(rs.rmax, rs.sum + ls.rmax), \\ totmax &= \max(ls.totmax, rs.totmax, ls.rmax + rs.lmax). \end{aligned}$$

Ta xét biểu diễn cả bốn thông tin cần duy trì hiện nay thành các hàm bậc nhất. Khi đó, mỗi đỉnh duy trì giá trị x tại thời điểm bị đánh bại, tức giá trị nhỏ nhất mà ở đó hàm được chọn trong mọi phép $\max$ ở các công thức trên thay đổi.

Ta gọi giá trị rank của một biến có chứa so sánh $\max$ là $r(v, \text{para})$:

- Với $r(v, lmax)$ hoặc $r(v, rmax)$: nếu công thức quyết định của biến đó là $f(x) = \max(a(x), b(x))$, giá trị rank là số đường thẳng trong a, b có hệ số góc lớn hơn hệ số góc của f , nhân với $d(v)^2$.
- Với $r(v, totmax)$: nếu công thức quyết định của biến đó là $f(x) = \max(a(x), b(x), c(x))$, giá trị rank là số đường thẳng trong a, b, c có hệ số góc lớn hơn hệ số góc của f , nhân với $d(v)$.

Ta trực tiếp định nghĩa thế năng

$$\Phi = \sum_v \sum_{\text{para}} r(v, \text{para}).$$

- Khi xảy ra một lần đánh bại ở $lmax$ hoặc $rmax$, ta có

$$\Delta \Phi \leq 1 - d^2 + (d - 1)^2 + d - 1 = 1 - d \leq 0.$$

- Khi xảy ra một lần đánh bại ở $totmax$, ta có

$$\Delta \Phi \leq 1 - d + d - 1 = 0.$$

Từ đó nhận được một cận trên độ phức tạp cho thuật toán là $\Theta(n \log^3 n + m \log^4 n + q \log n)$, trong đó m là số lần sửa, q là số lần truy vấn.

Nhưng tương tự phân tích cho trường hợp đặc biệt có hệ số góc đơn điệu ở trên, ta xét số lần quyết định của $lmax$ và $rmax$ ở mỗi đỉnh thay đổi. Chú ý rằng điểm quyết định của $lmax$ chỉ tăng, còn điều kiện cần để một đỉnh xảy ra chuyển hệ số góc là vị trí quyết định chuyển từ $[l, \text{mid}]$ sang $[\text{mid} + 1, r]$. Mỗi đỉnh chỉ kích hoạt nhiều nhất một lần, nên tổng độ phức tạp phụ của $lmax, rmax$ là $\Theta(n \log n)$. Vì vậy lúc này ta biết tổng số đỉnh mà KTT đi DFS tới để duy trì $lmax$ và $rmax$ trong bài toán này là $\Theta((n + q) \log n)$. Tiếp tục dùng phân tích thế năng cho $totmax$, độ phức tạp là

$$O((n + m) \log^3 n + q \log n).$$

Tuy nhiên điều này không có nghĩa phân tích thế năng trước đó hoàn toàn dư thừa. Phân tích $\log^4 n$ vừa nêu có thể hiểu là đang xét một thao tác tổng quát hơn: phép cộng có dạng $a_i \leftarrow a_i + k_i x$, còn trong bài toán gốc ẩn điều kiện $k_i = 1$, tức $k_i > 0$.

8.4.4 henry_y và dãy số

Ý chính đề bài: Cho một dãy A_i độ dài n , hỗ trợ thao tác sửa: cho l, r, a, b, c , với $l \leq i \leq r$, đặt

$$A_i \leftarrow A_i + ai^2 + bi + c,$$

bảo đảm $a, b \geq 0$; đồng thời truy vấn giá trị nhỏ nhất trên một đoạn.

Lời giải: Trong bài toán này tuy nhìn qua có hạng bậc hai, nhưng i^2 trong hạng bậc hai chỉ liên quan tới chỉ số, nên thực ra bài toán gần hơn với duy trì cực trị của một số hàm bậc nhất hai biến.

Trước hết vẫn dùng ý tưởng tương tự KTT: xét dùng cây đoạn duy trì giá trị nhỏ nhất hiện tại, cũng như thời điểm sau khi cộng $(ai^2 + bi)$ khiến một quan hệ so sánh nào đó trong cây con thay đổi. Do $a, b \geq 0$ và $i \geq 1$, giá trị nhỏ nhất chỉ có thể chuyển từ cây con phải sang cây con trái.

Giả sử giá trị nhỏ nhất của cây con trái nằm ở A_i , còn của cây con phải nằm ở A_j . Nếu $A_i > A_j$, thì trong tương lai A_j có thể vượt lại A_i ; điều này cần

$$\begin{aligned} A_i + ai^2 + bi &\leq A_j + aj^2 + bj, \\ A_i - A_j &\leq a(j^2 - i^2) + b(j - i), \\ \frac{A_i - A_j}{j - i} &\leq a(i + j) + b. \end{aligned}$$

Từ đó biết rằng khi (a, b) đi vào một nửa mặt phẳng, giá trị nhỏ nhất của cây con sẽ chuyển. Nói ngược lại, nếu nó nằm trong phần bù của nửa mặt phẳng này, thì giá trị nhỏ nhất vẫn chưa chuyển. Vì vậy, nếu thao tác sửa hiện tại với (a, b) không làm cây con chuyển, thì điểm đó phải nằm trong mọi ràng buộc của cây con, tức nằm trong giao của mọi nửa mặt phẳng. Do đường thẳng của nửa mặt phẳng có hệ số góc $i + j$, mọi hệ số góc ở cây con trái đều nhỏ hơn mọi hệ số góc ở cây con phải. Ta có thể dùng cây cân bằng bền vững để duy trì giao nửa mặt phẳng, đồng thời tìm kiếm nhị phân trên cây cân bằng để nhận kết quả gộp của hai giao nửa mặt phẳng trong thời gian $\Theta(\log n)$.

Vì bài toán này có tính đơn điệu tương tự bài toán trước, ta có thể đặt thể năng Φ là số đỉnh có “min nằm ở con phải”. Quá trình thao tác nhiều nhất phát sinh $\Theta(n + m \log n)$ lần cập nhật, và mỗi thao tác tốn $\Theta(\log^2 n)$ để cập nhật thông tin. Vì vậy ta có cận trên độ phức tạp

$$O((n + m \log n) \log^2 n + q \log n).$$

8.5 Tổng kết

Bài viết này chủ yếu xoay quanh một cận trên tốt cho độ dài của dãy $DS(n, s)$. Thông qua cách thiết kế dựa trên việc “giảm số giao điểm cần thiết phải duy trì”, ta thu được một số phương pháp có độ phức tạp khá tốt để duy trì cực trị của hàm số. Hy vọng những ý tưởng trên đây có thể gợi mở thêm cho các bài toán liên quan tới duy trì cực trị trong OI.

8.6 Lời cảm ơn

Cảm ơn China Computer Federation đã cung cấp nền tảng học tập và trao đổi.

Cảm ơn thầy Xiao Ran ở Trường Trung học trực thuộc Đại học Bắc Kinh vì sự quan tâm và hướng dẫn.

Cảm ơn gia đình, bạn bè vì sự ủng hộ và động viên.

Cảm ơn bạn Dai Jiangqi và anh Liu Cheng'ao đã thảo luận và gợi ý cho tôi.

Cảm ơn bạn Fang Lixing đã chỉ lỗi cho bài viết này.

Tài liệu tham khảo

1. Micha Sharir và Pankaj K. Agarwal. *Davenport-Schinzel Sequences and their Geometric Applications*. 1995.
2. Seth Pettie. *Sharp Bounds on Davenport-Schinzel Sequences of Every Order*. 2015.
3. Ady Wiernik và Micha Sharir. *Planar realizations of nonlinear Davenport-Schinzel sequences by segments*. 1988.
4. Xu Yixuan. Chuẩn bị đề bài cho bài tập giai đoạn một của đội tuyển quốc gia Trung Quốc IOI 2020. 2019.
5. Daniel Zhang. <https://codeforces.com/blog/entry/68534?comment=530381>. 2019.

9 Báo cáo ra đề Trò chơi trên đồ thị

Zuo Junchi, Trường Trung học số 2 trực thuộc Đại học Sư phạm Hoa Đông

Tóm tắt

Bài viết này giới thiệu một bài tương tác do tác giả ra cho một số trường trong một đợt huấn luyện chọn đội tuyển cấp tỉnh. Bài khai thác sâu các kiến thức về tính liên thông của đồ thị, cây khung của đồ thị, thứ tự DFS của cây có gốc, đồng thời kết hợp tự nhiên các thuật toán cơ bản như tìm kiếm nhị phân, chia để trị và tư tưởng ngẫu nhiên hóa. Kiến thức dùng trong bài không cao cấp, nhưng đòi hỏi thí sinh hiểu sâu các tư tưởng quan trọng ẩn sau những thuật toán cơ bản.

Ngoài ra, bài có nhiều điểm thành phần, có thể dẫn dắt thí sinh đi tới lời giải đúng và phân biệt thí sinh ở các mức độ suy nghĩ khác nhau. Lời giải đúng đại khái chia thành ba phần, và nghĩ ra mỗi phần đều có điểm tương ứng.

Bài viết này nêu một mục tiêu trong ra đề: thiết kế điểm thành phần phải khớp chặt với lời giải đúng.

9.1 Ý chính bài toán và phạm vi dữ liệu

9.1.1 Mô tả bài toán

Bạn Z có một đồ thị vô hướng gồm n đỉnh và m cạnh, có thể có cạnh song song nhưng không có khuyên. Bạn U muốn biết hình dạng đồ thị này, nhưng bạn Z không chịu nói.

Bạn Z: Tôi có thể cho bạn biết đồ thị này liên thông, các đỉnh được đánh số từ 0 đến $n - 1$, các cạnh được đánh số từ 0 đến $m - 1$.

Bạn U: Vậy sau khi xóa đi một cạnh, tình trạng liên thông của đồ thị này thế nào?

Bạn Z: Dù sao bạn cũng không đoán ra đâu, thế này nhé. Mỗi lần bạn cho tôi một tập S gồm các số hiệu cạnh, rồi cho thêm một đỉnh u , tôi sẽ giúp bạn tính xem sau khi nối các cạnh có số hiệu nằm trong S , đỉnh 0 có liên thông với đỉnh u hay không.

Nghe xong, bạn U suy nghĩ rất lâu nhưng vẫn không biết làm thế nào để hỏi ra hai đầu mút của từng cạnh trong đồ thị.

Vì vậy bạn ấy tìm tới bạn, muốn nhờ bạn hỏi ra thông tin của đồ thị này. Ta bảo đảm đồ thị đã được sinh sẵn từ trước, tức là nó không thay đổi theo các truy vấn của bạn U. Đồng thời bạn không được hỏi quá nhiều lần: bạn cần thu được thông tin của đồ thị trong 30000 truy vấn.

9.1.2 Cách tương tác

Bạn không cần cài đặt hàm chính; bạn chỉ cần cài đặt một hàm: `bool query(int u, std::vector<int> s);`

Hàm này cần trả về một mảng e có độ dài m , trong đó $e[i]$ biểu thị hai đầu mút của cạnh thứ i (không xét thứ tự).

Hàm bạn cài đặt chỉ được gọi hàm trong thư viện tương tác: `bool query(int u, std::vector<int> s);`

Trong hàm này, u là một số nguyên thuộc $[0, n - 1]$, còn s là một mảng `int` có độ dài m , các giá trị chỉ có thể là 0 hoặc 1. Nếu $s[i] = 1$ thì biểu thị i nằm trong tập S mà bạn U hỏi, ngược lại là không nằm trong S . Ý nghĩa của hàm là: cho u và S , hỏi đỉnh 0 có thể đi tới u bằng các cạnh có số hiệu nằm trong S hay không.

9.1.3 Phạm vi dữ liệu và ràng buộc đặc biệt

Với mọi dữ liệu, bảo đảm $1 \leq n, m \leq 600$, và đồ thị là liên thông.

- Subtask 1 (6 pts): $n, m \leq 25, m = n - 1$.
- Subtask 2 (10 pts): $n, m \leq 25$.
- Subtask 3 (17 pts): $m = n - 1$, mỗi đỉnh trong đồ thị có bậc không quá 2, và đỉnh 0 có bậc 1.
- Subtask 4 (24 pts): $m = n - 1$.
- Subtask 5 (19 pts): bảo đảm khi xét riêng các cạnh có số hiệu $< n - 1$, cạnh số i nối hai đỉnh i và $i + 1$.
- Subtask 6 (13 pts): bảo đảm khi xét riêng các cạnh có số hiệu $< n - 1$, đồ thị tạo thành một cây.
- Subtask 7 (11 pts): không có ràng buộc đặc biệt.

9.2 Kỹ năng chuẩn bị

9.2.1 Khái niệm và thuật ngữ cơ bản của đồ thị

Với một đồ thị vô hướng G , nếu giữa hai đỉnh bất kỳ đều tồn tại một đường đi nối hai đỉnh đó, ta gọi đồ thị là liên thông.

Nếu một đồ thị vô hướng G thỏa mãn rằng giữa hai đỉnh bất kỳ có đúng một đường đi nối chúng, thì G được gọi là một cây (hay cây không gốc).

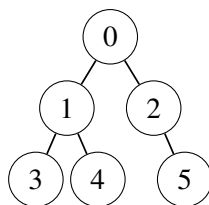
Với một đồ thị vô hướng liên thông tổng quát G , nếu cây T có cùng tập đỉnh với G và tập cạnh của T là tập con của tập cạnh G , thì T được gọi là một cây khung của G .

Sau khi chọn gốc cho một cây không gốc, ta thu được một cây có gốc. Với mỗi đỉnh không phải gốc u , đỉnh đầu tiên trên đường đi từ u tới gốc, không tính điểm xuất phát, được gọi là cha của u . Nếu v là cha của u , thì u được gọi là con của v . Nếu một đỉnh không có con, ta gọi nó là lá. Ta còn gọi mọi đỉnh trên đường đi từ gốc tới u là tổ tiên của u . Với một đỉnh cụ thể u , sau khi bỏ cạnh nối u với cha của nó (nếu cạnh này tồn tại), tập các đỉnh mà u còn đi tới được gọi là cây con của u .

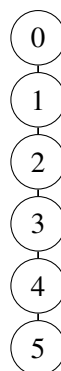
Ngoài ra, nếu một cây có gốc T thỏa mãn mỗi đỉnh của T có không quá một con, thì T được gọi là một dây.

Hình 1 là một ví dụ về cây có gốc, trong đó 0 có hai con 1, 2, 1 có hai con 3, 4, 2 có một con là 5, còn 3, 4, 5 không có con, nên chúng là lá.

Hình 2 biểu thị một dây. Trong đó 0, 1, 2, 3, 4 mỗi đỉnh vừa đúng có một con, còn 5 là lá.



Hình 1: ví dụ về cây có gốc lấy 0 làm gốc



Hình 2: ví dụ về dây lấy 0 làm gốc

9.2.2 Thứ tự DFS của cây

Với một cây có gốc T được đánh số từ 0, ta có một thuật toán DFS thường gặp như sau:

```

DFS(u)
1  PUSH(Ans, u)
2  for v in child(u)
3      DFS(v)
  
```

Trong mã giả, $\text{child}(u)$ biểu thị một thứ tự nào đó của các con của u .

Sau khi chạy thuật toán DFS này, Ans ghi lại dãy đỉnh được tạo ra trong quá trình DFS. Ký hiệu vị trí của u trong thứ tự DFS, đánh số từ 0, là id_u . Khi đó ta có tính chất sau:

Tính chất 1. Nếu v là tổ tiên của u , thì $\text{id}_u \geq \text{id}_v$.

Chứng minh. Trong quá trình DFS, nếu đã thăm một đỉnh thì cha của nó chắc chắn cũng đã được thăm, nên $\text{id}_u \geq \text{id}_v$.

9.2.3 Một phương pháp cơ bản

Ta xét bài toán đơn giản sau: trong các số $1, 2, \dots, N$ có M số tốt (ở đây ta xem M khá nhỏ, còn N khá lớn). Hiện tại bạn có thể chỉ định một vài số và hỏi trong các số đó có số tốt hay không; bạn cần dùng $O(M \log N)$ thao tác để tìm tất cả các số tốt.

Tư tưởng này được dùng nhiều lần trong bài tương tác này, và cũng xuất hiện trong rất nhiều bài tương tác ở các kỳ thi IOI.

Ta dùng tư tưởng chia để trị. Mã giả như sau; ở đây giá trị trả về của $\text{Ask}(S)$ biểu thị trong S có tồn tại số tốt hay không, còn Ans cuối cùng sẽ chứa tập tất cả số tốt.

```

SOLVE(left, right)
1  mid <- (left + right) / 2
2  if left == right
3      Ans <- Ans union {mid}
4  else
  
```



```

5      S1 <- {left, left + 1, ..., mid}
6      S2 <- {mid + 1, mid + 2, ..., right}
7      if Ask(S1)
8          SOLVE(left, mid)
9      if Ask(S2)
10         SOLVE(mid + 1, right)

```

Xét cấu trúc cây chia để trị do thuật toán này tạo ra. Các lá của nó có khoảng độ dài 1, và số trong khoảng đó chắc chắn là số tốt. Mỗi lá có độ sâu $O(\log N)$, nên tổng số nút của cây chia để trị là $O(M \log N)$. Vì vậy số lần hỏi cũng là $O(M \log N)$.

9.3 Giới thiệu thuật toán

Các thuật toán được giới thiệu sẽ đi sâu dần theo gợi ý từ điểm thành phần, nên ta sẽ trình bày theo thứ tự dây, cây, rồi đồ thị liên thông.

9.3.1 Dây

Phần điểm này có nhiều cách làm $O(n \log n)$, về cơ bản đều mượn tư tưởng ngẫu nhiên hóa.

Ở đây ta đưa ra một cách làm.

Trước hết, xáo trộn ngẫu nhiên tập đỉnh $\{1, 2, \dots, n-1\}$ thành $p_1, p_2, \dots, p_{n-1}$. Sau đó lần lượt thêm từng đỉnh p_i . Ta cần duy trì:

1. thứ tự tương đối giữa các đỉnh đã thêm;
2. với mỗi cạnh, tập mọi đỉnh nằm bên trái nó, tức phía gần đỉnh 0.

Ban đầu ta xem đỉnh 0 nằm ngoài cùng bên trái, đồng thời thêm một đỉnh ảo số n ở ngoài cùng bên phải.

Với mỗi đỉnh mới thêm p_i , trước hết ta có thể tìm kiếm nhị phân trên dãy hiện tại để biết nó phải được chèn vào vị trí nào giữa các đỉnh đã thêm. Cụ thể có thể giữ lại các cạnh nằm bên trái một đỉnh nào đó, rồi phán đoán 0 có đi tới được đỉnh mới thêm hay không, để thu hẹp khoảng đỉnh sẽ chèn vào.

Tiếp theo, trong các cạnh nằm giữa hai đỉnh được chèn vào là u_l, u_r , ta duyệt thô từng cạnh e để phán đoán nó nằm bên trái hay bên phải p_i . Bước này có thể thực hiện bằng cách xóa cạnh đó, giữ lại mọi cạnh khác, rồi phán đoán 0 có liên thông với p_i hay không.

Sau đây ta giải thích sơ lược vì sao độ phức tạp là $O(n \log n)$.

Do độ phức tạp của mỗi lần tìm kiếm nhị phân đã là $O(n \log n)$, ta chỉ cần xét độ phức tạp của phần duyệt thô để phán đoán vị trí từng cạnh. Giả sử đã thêm i đỉnh, thì theo nghĩa ngẫu nhiên, chi phí cần bỏ ra là $O(n/i)$. Vì $\sum_{i=1}^{n+2} 1/i = O(\ln n)$, nên $\sum_{i=1}^{n+2} n/i = O(n \ln n)$. Từ đó tổng độ phức tạp là $O(n \log n)$.

Vì vậy ta giải được bài con này trong $O(n \log n)$ truy vấn.

9.3.2 Cây

Trong phần sau, ta đều xem cây là cây có gốc tại 0.

Cách làm $O(n^2)$. Với mỗi cạnh e_i , ta xóa cạnh này và giữ lại tất cả các cạnh còn lại. Sau đó lần lượt hỏi đỉnh 0 có thể đi tới những đỉnh nào. Các đỉnh không đi tới được sẽ tạo thành cây con của một đỉnh nào đó.

Ngược lại, ta cũng có thể biết mọi cạnh trên đường đi từ mỗi đỉnh tới gốc, nhờ đó biết được độ sâu của mỗi đỉnh.

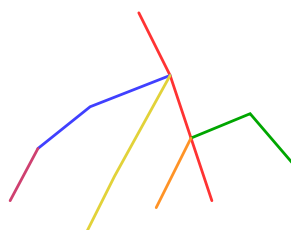
Sau đó, với mỗi cạnh, ta xét các đỉnh mà gốc không đi tới được sau khi xóa cạnh đó, rồi tìm đỉnh có độ sâu nhỏ nhất trong các đỉnh này. Đỉnh đó chính là gốc của cây con, cũng là đầu mút sâu hơn của cạnh này.

Tiếp đó, với mỗi đỉnh u , ta tìm cây con có kích thước nhỏ nhất trong các cây con chứa nó, không tính cây con của chính u , thì thu được cha của nó.

Cuối cùng, từ một đầu mút sâu hơn của mỗi cạnh, ta suy ra được cả hai đầu mút và giải quyết bài toán.

Ta phải xóa tổng cộng $n - 1$ cạnh, và với mỗi cạnh hỏi $n - 1$ lần, nên tổng độ phức tạp là $O(n^2)$.

Cách chia cạnh thành các dây. Ta xét gán cho mỗi đỉnh một số $0, 1, \dots, c - 1$, sao cho các đỉnh có cùng số tạo thành một dây, và cha của đỉnh trên cùng của mỗi dây thuộc một dây có số hiệu nhỏ hơn. Ban đầu mọi cạnh trên cây đều chưa được đánh dấu.



Hình 3: cách chia

Hình 3 là một ví dụ về cách chia đúng. Trong đó, mỗi cạnh của dây màu đỏ có đầu mút sâu hơn và gốc được đánh số 0, dây màu cam được đánh số 1, dây màu vàng được đánh số 2, dây màu lục được đánh số 3, dây màu lam được đánh số 4, dây màu tím được đánh số 5. Cách hiểu trực quan của phép chia này là: các dây phía sau đều được “gắn” dưới một dây nào đó phía trước. Điều này khiến về sau ta chỉ cần giải riêng từng dây, rồi tìm “điểm nối” giữa mỗi dây và phần trước đó.

Cố định một đỉnh u , định nghĩa mọi cạnh chưa đánh dấu trên đường đi từ 0 tới u là cạnh tốt. Nếu phần bù của tập được hỏi trong một truy vấn có chứa cạnh tốt, thì 0 và u sẽ không liên thông.

Lần lượt xét các đỉnh $1, 2, \dots, n - 1$.

Trường hợp một: nếu chỉ giữ lại các cạnh đã đánh dấu mà 0 vẫn liên thông với u , điều này cho thấy chính u nằm trên một dây đã có. Khi đó ta có thể giữ lại các cạnh thuộc những dây có số hiệu không quá mid và hỏi 0 có liên thông với u hay không. Bằng tìm kiếm nhị phân, ta biết được số hiệu dây chứa u .

Trường hợp hai: nếu chỉ giữ lại các cạnh đã đánh dấu mà 0 không liên thông với u , lúc này ta tìm tất cả các cạnh tốt chưa đánh dấu. Bước này có thể thực hiện bằng phương pháp chia để trị ở mục 2.3. Cuối cùng, ta cho các đỉnh sâu hơn của những cạnh này tạo thành một dây mới có số hiệu c , rồi đặt c mới bằng $c + 1$.

Chú ý rằng khi gặp trường hợp hai, bạn có thể biết tập số hiệu các cạnh nối các đỉnh của dây mới tới cha của chúng, nhưng chưa biết dây này gồm những đỉnh nào. Tuy nhiên, về sau có thể dùng cách làm của trường hợp một để xác định mọi đỉnh trên dây này.

Vì vậy, cuối cùng ta biết được mỗi dây gồm những đỉnh nào, đồng thời biết tập số hiệu các cạnh nối các đỉnh này với cha của chúng.

Cách làm sau khi đã chia thành các dây. Nếu biết các thông tin đó, ta có thể theo thứ tự số hiệu tăng dần để lần lượt xác định thứ tự trên từng dây, cũng như số hiệu cạnh nối mỗi đỉnh trên dây với cha của nó;

bước này dùng trực tiếp cách làm của phần dây. Do đó ta chỉ cần biết số hiệu đỉnh là cha của đỉnh trên cùng của mỗi dây.

Bước này có thể dùng cách tìm kiếm nhị phân trên thứ tự DFS. Cụ thể, giả sử dây đang được thêm có số hiệu i , và mọi đỉnh có số hiệu $< i$ tạo thành một cây T đã biết, chỉ còn cạnh nối từ dây $i - 1$ sang dây i là chưa biết. Ta giữ lại các cạnh trong T mà thứ tự DFS của hai đầu mút đều không quá mid , rồi giữ lại số hiệu cạnh nối đỉnh trên cùng của dây này tới cha nó, hỏi có thể đi tới đỉnh trên cùng của dây hay không, từ đó biết thứ tự DFS của cha của cạnh này có không quá mid hay không.

Chú ý rằng ở đây ta dùng tính chất 1 trong mục 2.2. Tính chất này bảo đảm rằng sau khi giữ lại các cạnh có hai đầu mút đều có thứ tự DFS không quá mid , đồ thị con thu được liên thông mọi đỉnh có thứ tự DFS không quá mid .

9.3.3 Đồ thị liên thông

Thuật toán $O(nm)$ để tìm cây khung. Ta chỉ cần tìm một tập chỉ số cạnh sao cho các cạnh mang những chỉ số này tạo thành một cây khung. Khi đó có thể áp dụng toàn bộ các phương pháp tìm cây ở trên.

Ban đầu, mọi cạnh đều ở trạng thái chưa đánh dấu. Sau đó lần lượt thêm các đỉnh $1, 2, \dots, n - 1$. Giả sử đang thêm tới đỉnh u , và các cạnh đã đánh dấu trước đó liên thông 0 với $1, 2, \dots, u - 1$.

Tính chất 1. Lần lượt quét mọi cạnh chưa đánh dấu. Nếu sau khi xóa cạnh này, đỉnh 0 không thể đi tới u , thì đánh dấu cạnh đó; ngược lại bỏ qua cạnh đó. Khi đó các cạnh mới được đánh dấu theo cách này có thể liên thông 0 với u , và các cạnh được đánh dấu vẫn tạo thành một rừng.

Chứng minh. Do ban đầu đồ thị liên thông, hiển nhiên 0 có thể đi tới u . Khi ta lần lượt quét mọi cạnh chưa đánh dấu, việc bỏ qua một cạnh không ảnh hưởng tới tính liên thông từ 0 tới u trước đó.

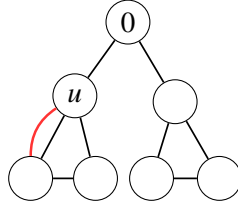
Tiếp theo chứng minh đồ thị con tạo bởi các cạnh mới được đánh dấu là không có chu trình. Phản chứng, giả sử tồn tại một chu trình C , gồm các cạnh $e_{i_1}, e_{i_2}, \dots, e_{i_k}$, trong đó $i_1 < i_2 < \dots < i_k$ theo thời điểm chúng được đánh dấu. Khi đánh dấu e_{i_1} , hai đầu mút của e_{i_1} đã được nối bằng các cạnh $e_{i_2}, e_{i_3}, \dots, e_{i_k}$. Vì vậy xóa e_{i_1} không ảnh hưởng tới tính liên thông, tức e_{i_1} không phải là cầu; nó đáng lẽ phải bị bỏ qua chứ không bị đánh dấu, mâu thuẫn.

Nhờ tính chất 1, mỗi vòng ta có thể dùng $O(m)$ truy vấn để mở rộng rừng sinh ra trên các đỉnh $0, 1, \dots, u - 1$ thành rừng sinh ra trên $0, 1, \dots, u$. Từ đó tìm được cây khung trong $O(nm)$ truy vấn.

Thuật toán $O(n \log m)$ để tìm cây khung. Chú ý rằng cuối cùng ta chỉ cần đánh dấu $n - 1$ cạnh. Ta có thể đổi quá trình quét tuần tự của thuật toán trước thành: mỗi lần tìm một tiền tố dài nhất có thể xóa mà 0 vẫn đi tới được u . Quá trình này có thể thực hiện bằng tìm kiếm nhị phân đơn giản.

Như vậy ta lần lượt tìm kiếm nhị phân cạnh tiếp theo cần được đánh dấu thay vì bị xóa. Có tổng cộng n vòng tìm kiếm nhị phân; trong mỗi vòng, trừ lần tìm kiếm nhị phân cuối cùng, các lần còn lại đều khiến ta đánh dấu thêm một cạnh. Do đó nhiều nhất có $n + n - 1 = O(n)$ lần tìm kiếm nhị phân. Tổng độ phức tạp thời gian là $O(n \log m)$.

Thuật toán $O(nm)$ để tìm cạnh không thuộc cây. Trước hết áp dụng thuật toán cho cây để tìm hai đầu mút của mỗi cạnh trong cây khung T .



Hình 4: cạnh đỏ biểu thị một cạnh không thuộc cây mà ta chưa biết

Với mỗi cạnh cây (u, v) , giả sử u là đỉnh sâu hơn. Giả sử cạnh cần hỏi là $e_i = (w_1, w_2)$, trong đó w_1, w_2 chưa biết. Ta xóa (u, v) , giữ lại mọi cạnh còn lại, rồi giữ thêm e_i , và hỏi 0 có liên thông với u hay không. Nếu đáp án là liên thông, điều đó cho thấy trong w_1, w_2 có đúng một đỉnh nằm trong cây con của u (như tình huống ở Hình 4), tức là đường đi từ w_1 tới w_2 trên T đi qua (u, v) .

Với mỗi cạnh không thuộc cây, ta lần lượt biết được mỗi cạnh cây có nằm trên đường đi mà nó “băng qua” hay không. Như vậy có thể suy ra w_1, w_2 từ hai đầu mút của đường đi.

Thuật toán một $O(m \log m)$ để tìm cạnh không thuộc cây. Phần này có thể làm từ trên xuống, cũng có thể làm từ dưới lên. Vì cách từ dưới lên có hằng số nhỏ hơn và dễ hiểu hơn, trước hết ta trình bày cách này.

Xét việc xóa một lá u của cây khung; khi xóa, ta cần biết mọi cạnh kề với u . Sau khi xóa hết mọi đỉnh, ta sẽ thu được mọi cạnh không thuộc cây.

Bước thứ nhất: gọi các cạnh kề với u là cạnh tốt. Xóa cạnh duy nhất của T kề với u . Nếu một truy vấn chứa toàn bộ $n - 2$ cạnh còn lại của T và một cạnh tốt, thì 0 liên thông với u . Nếu chỉ chứa $n - 2$ cạnh còn lại của T mà không chứa cạnh tốt, thì 0 không liên thông với u . Ta lại áp dụng phương pháp tìm số tốt ở trên để tìm mọi chỉ số cạnh tốt.

Bước thứ hai: với mỗi cạnh không thuộc cây, tìm đầu mút còn lại. Bước này có thể dùng tư tưởng tìm kiếm nhị phân trên thứ tự DFS đã nói trước đó. Cụ thể, giữ lại các đỉnh có thứ tự DFS không quá mid và cạnh không thuộc cây e_i đang xét. Nếu lúc này 0 có thể đi tới u , thì đầu mút còn lại của e_i có thứ tự DFS không quá mid; ngược lại thì lớn hơn mid. Như vậy ta tìm kiếm nhị phân được đầu mút còn lại của cạnh không thuộc cây nối với lá này.

Cuối cùng, ta giải bài này trong $O(m \log m)$ truy vấn.

9.3.4 Thuật toán hai $O(m \log m)$ để tìm cạnh không thuộc cây

Ta xét các cạnh cây theo một thứ tự tùy ý. Nếu đang xét cạnh cây (u, v) , giả sử v là cha của u , thì sau khi xóa cạnh này ta thu được hai thành phần liên thông C_1, C_2 . Ta cần tìm mọi cạnh không thuộc cây băng qua C_1, C_2 .

Ban đầu, đánh dấu mọi cạnh không thuộc cây là “chưa biết”. Mỗi lần chỉ xóa đúng cạnh (u, v) , giữ lại $n - 2$ cạnh cây còn lại. Nếu trong các cạnh không thuộc cây ta thêm vào có cạnh tốt, thì 0 liên thông với u , ngược lại 0 không liên thông với u . Vì vậy ta lại có thể dùng phương pháp chia để trị đã nói ở trên để tìm mọi cạnh tốt chưa biết. Đánh dấu các cạnh tốt này là đã biết; về sau không cần xét chúng nữa.

Tiếp theo, với một cạnh không thuộc cây e vừa tìm được, ta đánh số các đỉnh trong cây con C_1 theo thứ tự DFS, rồi giữ lại các cạnh có đầu mút có thứ tự DFS không quá mid, giữ lại mọi cạnh cây trong C_2 , giữ lại e , và hỏi 0 có đi tới được u hay không. Nếu có, thì đầu mút nằm trong C_1 của cạnh đó có thứ tự DFS không quá mid; ngược lại thì lớn hơn mid. Tương tự, ta làm điều giống vậy với thành phần liên thông C_2 , là biết được hai đầu mút của cạnh không thuộc cây e .

Sau khi xét xong $n - 1$ cạnh, ta có thể thu được mọi cạnh không thuộc cây, vì mỗi cạnh không thuộc cây chắc chắn “băng qua” ít nhất một cạnh cây.

Thực ra thuật toán một là biến thể của thuật toán hai dưới một thứ tự xét cạnh đặc biệt. Nếu trong thuật toán hai, ta đổi thứ tự xét cạnh thành: mỗi lần bóc dần một cạnh nối lá với cha của nó, thì tương đương với thuật toán một. Tuy nhiên khi bóc lá, một đầu mút của mỗi “cạnh tốt” luôn cố định, nên có thể đổi hai lần tìm kiếm nhị phân trên thứ tự DFS thành một lần tìm kiếm nhị phân trên thứ tự DFS, từ đó giảm hằng số.

9.4 Quá trình ra đề

Vấn đề này được tác giả nghĩ ra khi thử ra một bài tương tác trên cây. Ban đầu, tác giả suy nghĩ về tính chất của các thành phần liên thông trong đồ thị con trên cây, và thu được phiên bản của bài chỉ có phần điểm cây. Sau đó, qua suy nghĩ, tác giả tìm được cách khéo léo chuyển cây thành dây, rồi giải quyết bài toán “dây”.

Nhưng tôi cho rằng bài toán đẹp này vẫn có thể mở rộng, nên bắt đầu suy nghĩ xem trên đồ thị liên thông tổng quát, bài toán còn có thể được giải hiệu quả hay không. Tôi dùng cùng một phương pháp chuyển cái tổng quát về cái đặc biệt, tìm được cách tìm một cây khung. Sau đó tác giả tiếp tục nghĩ xem có thể dùng cạnh cây làm công cụ để tìm tất cả các cạnh không thuộc cây hay không, và thu được một cách làm hoàn chỉnh.

Khi thiết kế điểm thành phần, để nâng cao độ phân hóa, tác giả cố gắng giữ lại những chặng trung gian mà mình đã đi qua khi suy nghĩ bài này. Giả sử lời giải đúng có ba điểm khó A, B, C . Tôi cho rằng một mô hình tốt hơn để thiết kế điểm thành phần là chẳng hạn: làm được brute force được 20 điểm; nghĩ ra một trong A, B, C thì thêm 20 điểm; nghĩ ra hai điểm thì thêm 40 điểm; nghĩ ra cả ba điểm thì thêm 60 điểm; kết hợp được A, B, C để thu được lời giải đúng thì được 100 điểm.

Trong khi đó, nhiều cách cho điểm thành phần hiện nay lại là: trước khi nghĩ ra B bắt buộc phải nghĩ ra A , trước khi nghĩ ra C bắt buộc phải nghĩ ra A, B , thì mới lấy được nhiều điểm hơn. Mô hình này khiến thí sinh nghĩ ra B nhưng chưa nghĩ ra A nhận cùng điểm với thí sinh brute force, có thể làm tăng tính ngẫu nhiên của cuộc thi và giảm độ phân hóa. Vì vậy, tôi cho rằng trong thiết kế điểm thành phần của đề OI, chúng ta còn rất nhiều không gian để cải thiện.

9.5 Tình hình điểm số

9.5.1 Tình hình điểm số dự kiến

Hai subtask đầu cần thí sinh hiểu ý nghĩa đề bài và suy nghĩ đơn giản để thu được một thuật toán đa thức; dự kiến 70% thí sinh có thể lấy được. Subtask 3 có nhiều cách làm và không có điểm nào quá khó nghĩ, dự kiến 60% thí sinh có thể lấy được điểm này. Subtask 4, 5, 6 là các trường hợp đặc biệt của lời giải đúng ở những phương diện khác nhau; dự kiến 50% thí sinh có thể lấy đúng một subtask, và 30% thí sinh có thể lấy hai subtask. Còn để đạt điểm 70–100, cần kết hợp nhiều cách làm khác nhau một cách hữu cơ, độ khó tư duy rất lớn, nên ước tính không quá 5% thí sinh có thể AC bài này.

9.5.2 Tình hình điểm số thực tế

Trong số các thí sinh có điểm, có 1 người đạt 71 điểm, 1 người đạt 52 điểm, 1 người đạt 47 điểm, 4 người đạt 33 điểm, 2 người đạt 17 điểm, 4 người đạt 16 điểm, và 3 người đạt 6 điểm. Có thể thấy tình hình điểm số thực tế không tốt hơn dự kiến của tôi.

Từ góc độ thí sinh, điều này cho thấy thí sinh cần tăng cường luyện tập bài tương tác, nắm một số khuôn mẫu cơ bản của bài tương tác, như phương pháp tư duy tôi nêu ở mục 2.3, đồng thời nâng cao năng lực xử lý các vấn đề phức tạp, nhiều bước. Từ góc độ người ra đề, tôi cảm thấy trong các cuộc thi sau này, mình cần tăng cường hơn nữa tác dụng gợi ý và dẫn dắt của điểm thành phần để nâng cao độ phân hóa.

9.6 Tổng kết

Nhìn từ bốn kỳ thi WC2016, WC2018, WC2019 và NOI2019, ta có thể thấy các tác giả đề đang thử nâng cao tỷ trọng của bài tương tác trong OI. Vì vậy, tôi chọn báo cáo ra đề cho một bài tương tác làm chủ đề luận văn đội tuyển quốc gia của mình. Ngoài ra, vì đồng thời mang thân phận thí sinh và người ra đề, tôi không tránh khỏi có cách hiểu riêng về quá trình ra đề, nên trong bài viết này cũng đưa vào một vài suy nghĩ độc đáo về quá trình ra đề, với hy vọng có thể gợi mở cho độc giả.

Tóm lại, bài viết này giới thiệu một bài tương tác do tôi ra. Trong quá trình giới thiệu bài tương tác này, bài viết trước hết trình bày kiến thức cơ bản về đồ thị và các phương pháp tư duy cơ bản của bài tương tác, rồi phân tích sâu các thuật toán khác nhau cho bài.

Cuối cùng, từ bài toán này, bài viết mở rộng sang giới thiệu quan điểm riêng của chúng tôi về khâu thiết kế điểm thành phần khi ra đề, đồng thời nhìn về sự phát triển tương lai của lĩnh vực “bài tương tác” trong OI.

Hy vọng bài viết này không chỉ giúp mọi người học được lời giải của bài toán này, mà còn lĩnh hội được tư tưởng cơ bản của bài tương tác. Đồng thời cũng hy vọng sau khi bài viết này được công bố, trên sân thi OI sẽ xuất hiện nhiều bài tương tác hơn, với thiết kế điểm thành phần khoa học hơn.

9.7 Lời cảm ơn

Cảm ơn CCF đã cho tôi một nền tảng quý báu để chia sẻ và giao lưu.

Cảm ơn cha mẹ vì công ơn nuôi dưỡng.

Cảm ơn huấn luyện viên Jin Jing của Trường Trung học số 2 trực thuộc Đại học Sư phạm Hoa Đông vì sự hướng dẫn.

Cảm ơn các huấn luyện viên đội tuyển quốc gia vì sự tận tụy.

Cảm ơn bạn Zhang Bowei đã cùng tôi thảo luận nhiều cách giải của bài này, đặc biệt là thuật toán đầu tiên $O(n \log n)$ để xác định cạnh không thuộc cây, và cũng giúp tôi rất nhiều trong việc viết luận văn. Cảm ơn bạn Lyu Shiqing và bạn Chen Siyuan đã dạy tôi cách dùng LaTeX, giúp tôi dàn trang luận văn.

Cảm ơn tất cả các bạn đã giúp đỡ tôi.

Tài liệu tham khảo

1. https://csacademy.com/app/graph_editor/.
2. [https://zh.wikipedia.wikimirror.org/wiki/%E5%9B%BE_\(%E6%95%B0%E5%AD%A6\)](https://zh.wikipedia.wikimirror.org/wiki/%E5%9B%BE_(%E6%95%B0%E5%AD%A6)).
3. <https://zh.wikipedia.wikimirror.org/wiki/%E6%A0%91%E7%8A%B6%E5%9B%BE>.
4. [https://zh.wikipedia.wikimirror.org/wiki/%E6%A0%91_\(%E5%9B%BE%E8%AE%BA\)](https://zh.wikipedia.wikimirror.org/wiki/%E6%A0%91_(%E5%9B%BE%E8%AE%BA)).
5. <https://max.book118.com/html/2019/0810/6243233210002053.shtm> (*Bàn về tác hại của đề lệch*, Wang Tianyi).
6. <http://vfleaking.blog.uoj.ac/blog/909>.

10 Một loại DFT không bình thường

Zhou Yuyang, Trường Trung học số 1 Thiệu Hưng

Tóm tắt

Biến đổi Fourier nhanh là một thuật toán rất hữu dụng trong Olympic Tin học. Bài viết này giới thiệu một số ứng dụng khá tinh tế của biến đổi Fourier nhanh. Bài viết trình bày thuật toán Bluestein, áp dụng nó vào tính giá trị tại nhiều điểm và nội suy nhiều điểm của đa thức khi mô-đun nhỏ, đồng thời thử dùng thuật toán Bluestein để giải một số bài toán đa thức tiêu biểu.

10.1 Dẫn nhập

Biến đổi Fourier nhanh là một thuật toán rất hữu dụng. Nhờ biến đổi Fourier nhanh, ta có thể tính tích của hai đa thức trong thời gian $O(n \log n)$. Đồng thời, phép nhân đa thức có thể được dùng để giải nhiều bài toán đa thức khác, chẳng hạn nghịch đảo đa thức, exp của đa thức, v.v.

Tuy nhiên, có một số bài toán đặc biệt tuy có thể giải bằng biến đổi Fourier nhanh, nhưng cần các phép biến đổi khéo léo. Bài viết này tập trung tổng kết cách làm của một lớp bài toán đặc biệt, rồi thử áp dụng chúng vào các bài toán thường gặp hơn.

Bài viết chủ yếu gồm ba phần. Phần thứ nhất giới thiệu một số khái niệm liên quan tới biến đổi Fourier. Phần thứ hai bắt đầu từ vài bài ví dụ, giới thiệu cách suy ra thuật toán Bluestein và các ứng dụng cơ bản hơn của nó. Phần thứ ba xoay quanh tính giá trị tại nhiều điểm và nội suy nhiều điểm của đa thức; thông qua hai bài toán thuật toán kinh điển, phần này trình bày công dụng của thuật toán Bluestein khi mô-đun nhỏ, đồng thời thử mở rộng cách làm này sang các bài toán tổng quát hơn.

10.2 Định nghĩa và thuyết minh liên quan

10.2.1 Quy ước liên quan

Trong bài viết này, ta dùng ký hiệu $|f(x)|$ để chỉ bậc của đa thức $f(x)$. Ở đây ta quy ước hệ số của hạng tử bậc cao nhất của mọi đa thức khác không đều khác 0.

Trong bài viết này, ta định nghĩa giá trị của đa thức $f(x)$ tại p là kết quả thu được khi thay p vào $f(x)$.

Trong bài viết này, ta dùng ký hiệu ω_n để chỉ căn bậc n của đơn vị, tức là

$$\omega_n = \cos \frac{2\pi}{n} + \sin \frac{2\pi}{n} i.$$

10.2.2 Biến đổi Fourier rời rạc (DFT)

Biến đổi Fourier rời rạc là dãy điểm trị độ dài n nhận được khi lần lượt thay $\omega_n^0, \omega_n^1, \dots, \omega_n^{n-1}$ vào đa thức $f(x)$. Trong bài viết này, ta gọi tham số n trong biến đổi Fourier rời rạc là độ dài mô-đun. Trong thi tin học, biến đổi Fourier rời rạc thường cần bảo đảm $n > |f(x)|$.

Khi đã biết dãy điểm trị a độ dài n , ta dựng đa thức

$$g(x) = \sum_{i=0}^{n-1} a_i x^i.$$

Nếu lần lượt thay $\omega_n^0, \omega_n^{-1}, \dots, \omega_n^{-(n-1)}$ vào $g(x)$ và thu được dãy điểm trị b độ dài n , thì

$$f(x) = \frac{1}{n} \sum_{i=0}^{n-1} b_i x^i.$$

Chứng minh có thể xem ở blog², ở đây lược bỏ. Vì vậy, ta có thể dùng điểm trị của hai đa thức tại $\omega_n^0, \omega_n^1, \dots, \omega_n^{n-1}$ để nhanh chóng tính tích của hai đa thức.

²Liên kết blog: <https://www.cnblogs.com/RabbitHu/p/FFT.html>.

Biến đổi Fourier nhanh (FFT) là một tối ưu của biến đổi Fourier rời rạc dựa trên chia để trị. Thời gian chạy một lần là $O(n \log n)$. Phần sau sẽ trình bày cách làm của nó.

10.2.3 Biến đổi Fourier nhanh với mô-đun tổng quát (MTT)

Giả sử ta cần tính các hệ số của $A(x) \times B(x)$ theo mô-đun $p = 10^9 + 7$. Nếu trực tiếp dùng biến đổi Fourier rời rạc để giải, giá trị tuyệt đối của kết quả chạy sẽ đạt cỡ $p^2 \times |A(x)|$, không chấp nhận được về độ chính xác.

Xét tách mỗi hệ số của đa thức thành dạng $x^C + y$, trong đó $0 \leq y < C$ và C là một số nguyên dương. Theo đó, ta có thể tách đa thức $A(x)$ thành $A_1(x)C + A_2(x)$, trong đó mọi hệ số của $A_2(x)$ đều là số nguyên trong $[0, C - 1]$. Tương tự, với $B(x)$ ta cũng tách thành $B_1(x)C + B_2(x)$. Khi đó tích của hai đa thức là

$$A_1(x)B_1(x)C^2 + (A_1(x)B_2(x) + A_2(x)B_1(x))C + A_2(x)B_2(x).$$

Với các hạng tử có bậc theo C khác nhau, ta trực tiếp dùng phương pháp trên để giải. Lấy $C = \sqrt{p}$, giá trị tuyệt đối của kết quả sinh ra khi chạy phép nhân đa thức chỉ còn $p \times |A(x)|$, đủ chấp nhận được về độ chính xác.

Hoặc ta tìm ba số nguyên tố khác nhau, mỗi số có dạng $p \times 2^k + 1$ ($k \geq 22$); sau khi tìm căn nguyên thủy, dùng biến đổi Fourier nhanh để giải riêng rẽ, cuối cùng dùng định lý phần dư Trung Hoa để gộp đáp án. Thuật toán này có hằng số khá kém.

Trong luận văn đội tuyển quốc gia năm 2016 của anh Mao Xiao có nhắc tới tối ưu cho thuật toán thứ nhất; độc giả quan tâm có thể tự tra cứu.

10.3 DFT với độ dài mô-đun tổng quát

10.3.1 Dẫn vào

Ví dụ 1. ³

Đề bài. Có một dãy tuần hoàn vô hạn $x_1, x_2, \dots$, chu kỳ là n . Sau khi thực hiện một thao tác, chu kỳ của dãy x' sau thao tác vẫn là n , và thỏa

$$x'_i = x_i + x_{i+1}.$$

Có q truy vấn. Mỗi truy vấn cho n, m, k, i, p , yêu cầu tính: khi chu kỳ của dãy là n , ban đầu chỉ có $x_i = v$, còn các $x_j = 0$ ($j \neq i$), sau khi thực hiện k thao tác, giá trị của x_m ($m \leq n$) lấy mô-đun p là bao nhiêu. Bảo đảm tồn tại căn bậc n của đơn vị theo nghĩa mô-đun p .

$$k \leq 2 \times 10^9, \quad p \leq 2 \times 10^9, \quad \sum n \leq 10^6.$$

Cách làm. Sau khi xét ý nghĩa tổ hợp của đáp án, ta thực hiện một phép đảo qua căn đơn vị đơn giản:

$$\begin{aligned} \sum_{t=0}^{\infty} \binom{K}{tn+m} &= \sum_{i=0}^K \binom{K}{i} \frac{\sum_{j=0}^{n-1} \omega_n^{(i-m)j}}{n} \\ &= \frac{1}{n} \sum_{j=0}^{n-1} \omega_n^{-mj} \sum_{i=0}^K \binom{K}{i} \omega_n^{ij} \\ &= \frac{1}{n} \sum_{j=0}^{n-1} \omega_n^{-mj} (1 + \omega_n^j)^K. \end{aligned}$$

Sau khi tìm căn đơn vị, chỉ cần dùng lũy thừa nhanh để cộng trực tiếp. Độ phức tạp là $O(n \log K)$.

³Nguồn: Peking University Training 2019.

Ví dụ 2. Đề bài. Có q truy vấn. Mỗi truy vấn cho đa thức $A(x)$ có bậc hạng tử cao nhất nhỏ hơn n , hỏi kết quả từng hệ số của $A(x)^K$ sau khi lấy phần dư theo $(x^n - 1)$, rồi lấy mô-đun số nguyên tố p . Bảo đảm tồn tại căn bậc n của đơn vị theo nghĩa mô-đun p .

$$k \leq 2 \times 10^9, \quad p \leq 2 \times 10^9, \quad \sum n \leq 10^6.$$

Suy nghĩ. Trực tiếp đẩy công thức để tính đáp án rất rườm rà. Dùng lũy thừa nhanh đa thức $O(n \log n \log K)$ thì độ phức tạp quá cao. Đồng thời, vì có phép lấy phần dư theo $x^n - 1$, ta không thể giải bài toán này bằng exp đa thức và ln đa thức.

Vì DFT độ dài mô-đun n về bản chất là thực hiện một phép tích chập vòng độ dài n , nếu có thể nhanh chóng thực hiện DFT với độ dài mô-đun đúng bằng n , ta có thể giải bài toán trong độ phức tạp $O(n(\log n + \log K))$ với một hằng số nhỏ.

Nhưng FFT chỉ xử lý được trường hợp đặc biệt độ dài mô-đun là 2^k , còn DFT trực tiếp có thời gian quá lớn. Vì vậy, ta cần một phương pháp tốt hơn và tổng quát hơn để giải FFT với độ dài mô-đun bất kỳ.

10.3.2 Biến đổi Fourier nhanh

Trước hết ta ôn lại cách làm của biến đổi Fourier nhanh. Giả sử độ dài mô-đun $n = 2^k$, và ta cần tìm dãy điểm trị a độ dài n thu được khi lần lượt thay $\omega_n^0, \omega_n^1, \dots, \omega_n^{n-1}$ vào đa thức $f(x)$. Đặt

$$f(x) = \sum_{i=0}^{n-1} f_i x^i.$$

Đặt

$$f_1(x) = \sum_{i=0}^{n/2-1} f_{2i} x^i, \quad f_2(x) = \sum_{i=0}^{n/2-1} f_{2i+1} x^i,$$

thì $f(x) = f_1(x^2) + x f_2(x^2)$.

Xét giá trị của đa thức $f(x)$ sau khi thay ω_n^k và $\omega_n^{k+n/2}$. Ta lần lượt thu được:

$$\begin{aligned} f(\omega_n^k) &= f_1(\omega_n^{2k}) + \omega_n^k f_2(\omega_n^{2k}) \\ &= f_1(\omega_{n/2}^k) + \omega_n^k f_2(\omega_{n/2}^k), \\ f(\omega_n^{k+n/2}) &= f_1(\omega_n^{2k+n}) + \omega_n^{k+n/2} f_2(\omega_n^{2k+n}) \\ &= f_1(\omega_{n/2}^k) - \omega_n^k f_2(\omega_{n/2}^k). \end{aligned}$$

Dễ thấy kết quả chạy chỉ liên quan tới $f_1(\omega_{n/2}^k)$ và $f_2(\omega_{n/2}^k)$. Vì vậy, có thể dùng độ phức tạp $O(n)$ để chuyển nó thành hai bài toán con kích thước $n/2$. Áp dụng định lý chính, không khó để thu được độ phức tạp thời gian $O(n \log n)$. Đây chính là biến đổi Fourier nhanh (FFT) thường nói.

10.3.3 Biến đổi Fourier nhanh dựa trên chia để trị

Tương tự biến đổi Fourier nhanh, ta xét mở rộng nó sang độ dài mô-đun tổng quát hơn. Nói chung, bây giờ giả sử ta cần tính

$$V(j) = \sum_{i=0}^{n-1} a_i \omega_n^{ij}.$$

Tương tự ý tưởng phía trên, đặt d là thừa số nguyên tố nhỏ nhất của n , $m = n/d$, $j = pd + q$ ($0 \leq p < m$, $0 \leq q < d$), và $i = xm + y$ ($0 \leq x < d$, $0 \leq y < m$). Thay các định nghĩa trên vào công thức DFT, ta

được:

$$\begin{aligned}
V(pd + q) &= \sum_{0 \leq i < n} \omega_n^{i(pd+q)} a_i \\
&= \sum_{0 \leq i < n} \omega_n^{ipd} \omega_n^{iq} a_i \\
&= \sum_{0 \leq i < n} \omega_m^{ip} \omega_n^{iq} a_i \\
&= \sum_{0 \leq y < m} \sum_{0 \leq x < d} \omega_m^{(xm+y)p} \omega_n^{(xm+y)q} a_{xm+y} \\
&= \sum_{0 \leq y < m} \sum_{0 \leq x < d} \omega_m^{yp} \omega_d^{xq} \omega_n^{yq} a_{xm+y}.
\end{aligned}$$

Lần lượt liệt kê giá trị của q . Khi đó $\sum_{x < d} \omega_d^{xq} \omega_n^{yq} a_{xm+y}$ là một hằng số chỉ phụ thuộc vào y ; ta đặt nó là $B(y)$ và thay vào công thức ban đầu:

$$\begin{aligned}
V(pd + q) &= \sum_{0 \leq y < m} \sum_{0 \leq x < d} \omega_m^{yp} \omega_d^{xq} \omega_n^{yq} a_{xm+y} \\
&= \sum_{0 \leq y < m} \omega_m^{yp} B(y).
\end{aligned}$$

Đối chiếu với định nghĩa DFT ban đầu, không khó nhận ra rằng khi q đã xác định, ta đã chuyển nó thành đúng một bài toán con kích thước m . Điều này gợi ý rằng ta có thể giải bài toán con này bằng đệ quy.

Xét độ phức tạp thời gian. Gọi thời gian giải bài toán DFT có độ dài mô-đun n là $F(n)$. Theo định nghĩa thuật toán, ta có

$$F(n) = dF\left(\frac{n}{d}\right) + O(dn), \quad F(1) = O(1).$$

Giả sử phân tích thừa số nguyên tố của n có dạng $\prod a_i^{p_i}$. Theo dạng phân tích này, đặt hàm

$$\Omega_0(n) = \sum_i a_i p_i.$$

Khi đó $F(1)$ thỏa $F(n) = O(n\Omega_0(n))$. Nếu với mọi số nguyên nhỏ hơn n đều thỏa $F(n) = O(n\Omega_0(n))$, thì

$$F(n) = O\left(\Omega_0\left(\frac{n}{d}\right)n\right) + O(dn),$$

và không khó thấy nó vẫn thỏa $F(n) = O(n\Omega_0(n))$. Từ đó ta tính được độ phức tạp.

Nhưng trong trường hợp xấu nhất, $\Omega_0(n)$ vẫn có thể đạt cấp $O(n)$. Vì vậy, ta cần một thuật toán có độ phức tạp tốt hơn.

10.3.4 Biến đổi Fourier nhanh dựa trên tích chập

Ta quay lại công thức của DFT và IDFT để xét bài toán này:

$$V(j) = \sum_{0 \leq i < n} \omega_n^{ij} a_i.$$

Xét ý nghĩa thực tế của $i \times j$: có thể xem là số cách chọn một vật từ mỗi đồng trong hai đồng vật, đồng thứ nhất có kích thước i , đồng thứ hai có kích thước j .

Trên số cách này, ta xét bao hàm–loại trừ. Lấy số cách gộp hai đồng vật rồi chọn hai phần tử khác nhau không xét thứ tự, trừ đi tổng của số cách chọn hai phần tử khác nhau không xét thứ tự trong đồng thứ nhất và trong đồng thứ hai; kết quả chính là số cách chọn riêng một vật từ mỗi đồng. Chuyển thành công thức toán học:

$$\binom{i+j}{2} - \binom{i}{2} - \binom{j}{2} = i \times j.$$

Thay công thức bao hàm–loại trừ trên vào công thức DFT, ta được:

$$V(j)\omega_n^{-(j)} = \sum_{0 \leq i < n} a_i \omega_n^{-(i)} \omega_n^{(i+j)}.$$

Công thức này có thể được xem là kết quả của một phép tích chập hiệu giữa

$$C(x) = \sum \omega_n^{-(i)} x^i$$

và

$$B(x) = \sum a_i \omega_n^{(i)} x^i,$$

rồi nhân từng hệ số với giá trị hằng số tương ứng. Lúc này ta không quan tâm tới ảnh hưởng do độ dài mô-đun của tích chập gây ra; ta chỉ quan tâm tới kết quả tích chập, không quan tâm tới vấn đề độ dài mô-đun của tích chập. Vì vậy, có thể trực tiếp dùng MTT để giải. Độ phức tạp thời gian là $O(n \log n)$.

Tương tự, với công thức IDFT, thay công thức bao hàm–loại trừ phía trên ta được:

$$V(j)\omega_n^{-(j)} = \sum_{0 \leq i < n} a_i \omega_n^{(i)} \omega_n^{-(i+j)}.$$

Tương tự, công thức này vẫn có thể được xem là một phép tích chập hiệu, rồi nhân từng hệ số với giá trị hằng số tương ứng. Vì vậy, vẫn có thể trực tiếp dùng một lần MTT để giải. Độ phức tạp thời gian $O(n \log n)$. Thuật toán này cũng được gọi là thuật toán Bluestein hoặc biến đổi Z.

Thật ra, với bất kỳ số khác không a , ta đều có thể dùng ý tưởng này để tính các giá trị

$$V(j) = \sum_{0 \leq i < n} x_i a^{ij}.$$

Ta chỉ cần bảo đảm tồn tại a^{ij} và nghịch đảo của nó. Nói cách khác, với số phức, thuật toán này vẫn áp dụng được.

- Mẹo tối ưu nhỏ: ban đầu độ dài mô-đun của phần tích chập hiệu cần mở tới cấp $3n$. Nhưng theo bản chất của việc dùng FFT để tăng tốc nhân đa thức, ta đã thực hiện một phép tích chập vòng. Vì chỉ quan tâm tới tính đúng của n phần tử ở giữa, việc cho n phần tử cuối trùng với n phần tử đầu trong kết quả sẽ không ảnh hưởng tới tính đúng đắn của đáp án. Vì vậy, chỉ cần mở độ dài mô-đun tới cấp $2n$.

10.3.5 Ví dụ

Ví dụ 3. ⁴

Đề bài. Định nghĩa tích của hai đồ thị vô hướng đơn $G_1 = (V_1, E_1)$, $G_2 = (V_2, E_2)$ là một đồ thị mới

$$G_1 \times G_2 = (V^*, E^*).$$

Trong đó tập đỉnh mới thỏa

$$V^* = \{(a, b) \mid a \in V_1, b \in V_2\},$$

còn tập cạnh mới thỏa

$$E^* = \{((u_1, v_1), (u_2, v_2)) \mid (u_1, u_2) \in E_1, (v_1, v_2) \in E_2\}.$$

⁴Nguồn: UOJ498, Cuộc đua bắt năm mới.

Cho số nguyên dương n , cùng n số nguyên dương $m_1, m_2, \dots, m_n$. Cần tính kỳ vọng số thành phần liên thông của đồ thị mới

$$H = (((G_1 \times G_2) \times G_3) \times \dots) \times G_n$$

theo mô-đun 998244353. Trong đó, mỗi G_i được sinh ngẫu nhiên đều nhau từ toàn bộ các đồ thị có m_i đỉnh.

$$n, m_i \leq 100000.$$

Cách làm. Ta xét cách tính số thành phần liên thông sau khi đã cho một dãy các G_k .

Trước hết xét chọn một đỉnh trong mỗi đồ thị. Nếu bậc của một đỉnh được chọn nào đó bằng 0 (ta gọi nó là “đỉnh cô lập”), thì dãy đỉnh này tương ứng với một đỉnh cô lập trên H . Nếu không, ta xét chọn một thành phần liên thông có kích thước > 1 trong mỗi đồ thị. Xét số thành phần liên thông thu được từ tích của các thành phần liên thông này. Chú ý rằng hiện tại mỗi đỉnh đều có cạnh kề và đồ thị là vô hướng, nên ta có thể đi qua lại trên một cạnh. Tính liên thông giữa hai dãy đỉnh có thể được giản lược thành tính chẵn lẻ của độ dài đường đi.

Nếu hai đỉnh nằm trong một đồ thị có chu trình lẻ, thì hiển nhiên đều tồn tại đường đi độ dài lẻ và đường đi độ dài chẵn. Nếu hai đỉnh nằm trong một đồ thị hai phía, điều này liên quan tới việc chúng có ở cùng một phía hay không. Vì vậy, ta có kết luận: nếu trong n thành phần liên thông được chọn có k thành phần không chứa chu trình lẻ, thì tích của các thành phần liên thông này sẽ đóng góp $2^{\max(k-1, 0)}$ thành phần liên thông vào đáp án. Do đó, ta chỉ cần biết trong toàn bộ các đồ thị kích thước m_k có thể có bao nhiêu đỉnh cô lập, bao nhiêu thành phần liên thông không có chu trình lẻ, và bao nhiêu thành phần liên thông; từ đó có thể tính đáp án.

Gọi EGF của đồ thị vô hướng đơn là

$$G(x) = \sum_{n \geq 0} \frac{2^{\binom{n}{2}}}{n!} x^n,$$

thì EGF của đồ thị liên thông là $C = \ln G$. Không khó thu được EGF của số thành phần liên thông bằng cách liệt kê cách chèn thành phần liên thông vào một đồ thị, tức là $G \ln G$.

Xét EGF của đồ thị hai phía có tô màu:

$$B = \sum_{n \geq 0} \sum_{m \geq 0} \binom{n+m}{n} 2^{nm} \frac{x^{n+m}}{(n+m)!}.$$

Một thành phần liên thông không có chu trình lẻ hiển nhiên có đúng 2 cách tô màu, nên EGF của nó là $\frac{\ln B}{2}$. Số thành phần liên thông không có chu trình lẻ có thể được biểu diễn bởi $G \frac{\ln B}{2}$. Ở đây ta có thể dùng thuật toán Bluestein để tối ưu quá trình tính EGF của đồ thị hai phía có tô màu.

Độ phức tạp thời gian $O(n \log n)$.

Ví dụ 4. ⁵

Đề bài. Cho hai dãy số nguyên không âm b_i và c_i độ dài n , đều đánh chỉ số từ 0.

Biết dãy số nguyên không âm a thỏa

$$c_i = \sum_{k=0}^{n-1} (b_{k-i \bmod n} - a_k)^2,$$

cần tìm mọi nghiệm hợp lệ của a_i . Bảo đảm mọi dịch vòng độ dài n của b đều độc lập tuyến tính.

⁵Nguồn: Codeforces 901E, *Cyclic Cipher*.

$$n \leq 10^5, \quad b_i \leq 10^3, \quad c_i \leq 5 \times 10^6.$$

Cách làm. Để tiện mô tả, sau đây mọi dãy đều là dãy vô hạn có chu kỳ n .

Vì $(a - b)^2 = a^2 + b^2 - 2ab$, nên

$$c_k - c_{k-1} = \sum_{i=0}^{n-1} b_i(a_{i+k} - a_{i-k+1}).$$

Đặt $c'_k = c_k - c_{k-1}$, $a'_k = a_k - a_{k-1}$, ta có

$$c'_k = \sum_{y-x \bmod n = k} b_x a'_y.$$

Do đó, c'_k có thể được xem là kết quả tích chập độ dài mô-đun n của $b_i x^i$ với $a'_i x^{n-i}$. Dùng thuật toán Bluestein theo nghĩa mô-đun, ta có thể nhanh chóng tính được duy nhất một bộ a'_i hợp lệ.

Đặt $a_0 = x$, các số còn lại đều có thể biểu diễn bằng x . Khi đó ta lập được một phương trình bậc hai một ẩn, từ đó giải ra không quá hai nghiệm hợp lệ; chỉ cần trực tiếp thay vào để kiểm tra tính hợp lệ.

Độ phức tạp thời gian $O(n \log n)$.

10.4 Ứng dụng của DFT trong tính giá trị tại nhiều điểm

10.4.1 Ý tưởng chung

Bài toán tính giá trị tại nhiều điểm của đa thức là bài toán có nhiều truy vấn, mỗi truy vấn hỏi giá trị của đa thức $f(x)$ tại y .

Vì biến đổi Fourier rời rạc là dãy điểm trị độ dài n nhận được khi lần lượt thay $\omega_n^0, \omega_n^1, \dots, \omega_n^{n-1}$ vào đa thức $f(x)$, khi giải bài toán tính giá trị tại nhiều điểm của đa thức, nếu các điểm truy vấn thỏa một số tính chất đặc biệt, ta có thể sắp xếp lại chúng rồi dùng thuật toán Bluestein để tăng tốc quá trình tính giá trị tại nhiều điểm.

Ví dụ 5. ⁶

Đề bài. Cho đa thức bậc n

$$f(x) = \sum_{i=0}^n f_i x^i.$$

Có Q truy vấn; truy vấn thứ i hỏi kết quả của $f(q_i)$ lấy mô-đun $p = 998244353$. Các q_i được sinh theo cách sau:

$$\forall 1 \leq i \leq Q, \quad q_i = (q_{i-1} \times a + b) \bmod 998244353.$$

$$1 \leq n \leq 2.5 \times 10^5, \quad 1 \leq Q \leq 10^6, \quad 2 \leq a < 998244353, \quad 0 \leq q_0, b < 998244353.$$

Cách làm. Giả sử ta đã biết một đa thức $f(x)$ bậc n bất kỳ. Khi đó ta có thể:

- tìm trong $O(n)$ đa thức duy nhất $g(x)$ bậc không quá n thỏa $g(x) = f(x * k)$;
- tìm trong $O(n \log n)$ đa thức duy nhất $h(x)$ bậc không quá n thỏa $h(x) = f(x + k)$.

Quan sát thấy giá trị truy vấn khá đặc biệt, ta thử dùng một vài biến đổi trên, thông qua cộng, trừ, nhân, chia để chuyển giá trị truy vấn thành một dạng đẹp hơn.

⁶Nguồn: UOJ500, DFT cơ sở tùy ý.

Trước hết quy nạp thu được

$$q_i = q_0 a^i + \sum_{j=0}^{i-1} b a^j.$$

Tiếp theo thực hiện các biến đổi sau:

1. Nhân vế phải với $a - 1$, thu được $q_i = q_0(a - 1)a^i + b a^i - b$.
2. Cộng b vào vế phải, thu được $q_i = q_0(a - 1)a^i + b a^i$.
3. Chia cho $q_0(a - 1) + b$, thu được $q_i = a^i$.

Chú ý rằng khi $q_0(a - 1) + b = 0$, ta có $q_i = q_0$, nên chỉ cần tính một điểm trị đơn lẻ tại q_0 . Ngược lại, dưới phạm vi dữ liệu đã cho, mọi phép toán trên đều hợp lệ. Vì vậy, ta có thể thực hiện biến đổi tương tự với $f(x)$: tìm đa thức bậc n $g(x)$ thỏa

$$g(x) = f\left(\frac{x(q_0(a - 1) + b) - b}{a - 1}\right).$$

Theo tính chất của các biến đổi trên, có đúng một đa thức bậc không quá n thỏa điều kiện.

Bây giờ ta cần tính giá trị của đa thức $g(x)$ tại $a^1, a^2, \dots, a^Q$, tức là cần tính

$$V(i) = \sum_{j=0}^n g_j a^{ij}.$$

Không khó nhận ra công thức cần giải thỏa mọi tính chất của thuật toán Bluestein, nên có thể dùng thuật toán này để giải.

Độ phức tạp thời gian $O((n + Q) \log(n + Q))$.

Ví dụ 6. ⁷

Đề bài. Cho đa thức bậc n

$$f(x) = \sum_{i=0}^n a_i x^i.$$

Có Q truy vấn, mỗi truy vấn cho một số nguyên dương y , hỏi giá trị $f(y)$ lấy mô-đun $p = 786433 = 3 \cdot 2^{18} + 1$.

$$n, Q \leq 250000, \quad 0 \leq y < p.$$

Cách làm. Xét ý nghĩa thực tế của biến đổi Fourier rời rạc, không khó nhận ra DFT độ dài mô-đun n có thể được xem là các điểm trị sinh ra khi thay $\omega_n^0, \omega_n^1, \dots, \omega_n^{n-1}$ vào đa thức $A(x)$.

Vì p là số nguyên tố nên tồn tại căn nguyên thủy theo mô-đun p . Điều này có nghĩa là tồn tại g thỏa $\omega_{p-1} \equiv g \pmod{p}$. Đồng thời, theo định nghĩa căn nguyên thủy, $g^0, g^1, \dots, g^{p-2}$ đôi một khác nhau theo nghĩa mô-đun và vừa đúng lấy được mọi số nguyên dương nhỏ hơn p , nguyên tố cùng nhau với p . Nói cách khác, chúng tạo thành một hoán vị của $[1, p - 1]$.

Vì vậy, nếu ta chạy một DFT độ dài mô-đun $p - 1$ trên $A(x)$, rồi sắp xếp lại dãy điểm trị thu được, ta sẽ nhận được các điểm trị tại $1, 2, \dots, p - 1$. Đồng thời, điểm trị tại 0 hiển nhiên là a_0 . Do đó, mọi điểm trị đều đã được tính.

Độ phức tạp thời gian $O(p \log p)$.

⁷Nguồn: CODECHEF POLYEVAL.

10.4.2 Mở rộng

Ta thử mở rộng mô-đun của Ví dụ 6 sang trường hợp tổng quát hơn. Bây giờ giả sử ta cần tính điểm trị theo mô-đun $p = q^c$ ($c \geq 1$), trong đó q là một số nguyên tố bất kỳ lớn hơn 2.

Vì q^c có căn nguyên thủy, nên tồn tại g thỏa

$$\omega_{\varphi(q^c)} \equiv g \pmod{q^c}.$$

Đồng thời, theo định nghĩa căn nguyên thủy,

$$g^0, g^1, \dots, g^{\varphi(q^c)-1}$$

đôi một khác nhau theo nghĩa mô-đun và vừa đúng lấy được mọi số nguyên dương nhỏ hơn q^c , nguyên tố cùng nhau với q^c . Với điểm trị tại các điểm này, ta vẫn có thể dùng một lần DFT để tính.

Với các số nguyên không nguyên tố cùng nhau với q^c , đặt số đó là x . Hiển nhiên $x^c \equiv 0 \pmod{q^c}$, nên ta chỉ cần xét đóng góp của các hạng tử có bậc nhỏ hơn c ; trực tiếp tính thô $c - 1$ hạng tử đầu là đủ để tính được điểm trị. Vì $c = O(\log p)$, phần này vẫn có độ phức tạp thời gian $O(p \log p)$.

Riêng với mô-đun đặc biệt $p = 2^c$ ($c \geq 3$), tuy nó không có căn nguyên thủy, nhưng vẫn có thể chứng minh

$$\pm 3^0, \pm 3^1, \dots, \pm 3^{2^{c-2}}$$

đôi một khác nhau theo nghĩa mô-đun và đều nguyên tố cùng nhau với 2. Quá trình chứng minh khá phức tạp nên ở đây lược bỏ. Với điểm trị tại các điểm này, ta vẫn có thể dùng hai lần DFT để tính. Tương tự, với các điểm không nguyên tố cùng nhau với 2, vẫn chỉ cần xét các hạng tử có bậc nhỏ hơn c .

Nhưng vì độ dài mô-đun của biến đổi Fourier nhanh không nguyên tố cùng nhau với 2, sẽ xảy ra vấn đề khi làm IDFT. Vì vậy, ta cần giải quyết bằng cách nhân mô-đun của phần biến đổi Fourier nhanh với độ dài mô-đun, rồi cuối cùng trực tiếp chia kết quả cho độ dài mô-đun; hoặc đặt độ dài mô-đun của phần FFT là 3^k , rồi dùng biến đổi Fourier nhanh dựa trên chia để trị. Do đó, bài toán vẫn có thể được giải trong thời gian $O(p \log p)$.

Nếu dùng định lý phần dư Trung Hoa để gộp các đáp án theo các q^c khác nhau, thì với mô-đun bất kỳ, ta đều có thể tính giá trị tại nhiều điểm của đa thức trong $O(p \log p)$.

10.5 Ứng dụng của DFT trong nội suy nhiều điểm

Bài toán nội suy nhiều điểm của đa thức là bài toán: cho điểm trị của đa thức $f(x)$ tại một số vị trí y , hỏi các hệ số của $f(x)$.

Vì DFT có thể được ứng dụng tốt vào tính giá trị tại nhiều điểm của đa thức, ta xét mở rộng nó sang thao tác nghịch đảo, tức là nội suy nhiều điểm của đa thức.

10.5.1 Nội suy nhiều điểm từ các điểm trị liên tiếp

Đề bài. Cho dãy điểm trị a của đa thức bậc n $f(x)$ tại $0 \sim n$, hỏi từng hệ số của $f(x)$ lấy mô-đun p .

$$0 < n < p \leq 500000, \quad p \text{ là số nguyên tố.}$$

Thuật toán. Vì thao tác nghịch đảo của tính giá trị tại nhiều điểm là nội suy nhiều điểm, và thao tác nghịch đảo của DFT là IDFT, nếu xem tính giá trị tại nhiều điểm là một thao tác DFT, điều này gợi ý rằng ta có thể thử dùng một lần IDFT để giải bài toán nội suy nhiều điểm của đa thức.

Cụ thể, trước hết ta tính các điểm trị tại $0 \sim p-1$ từ các điểm trị tại $0 \sim n$. Việc này có thể thực hiện bằng cách chuyển qua lại giữa điểm trị và tính chất giai thừa giảm của đa thức; phần này không triển khai thêm.

Vì ta đã biết các điểm trị tại $1 \sim p-1$, ta có thể dùng một lần IDFT để xác định một đa thức $f(x)$ bậc không quá $p-1$, sao cho điểm trị của $f(x)$ tại $1 \sim p-1$ vừa đúng là $a_1, a_2, \dots, a_{p-1}$. Tính đúng đắn của phần này có thể thu được từ tính đúng đắn của DFT và IDFT. Do đó, bây giờ chỉ cần dựng đa thức $h(x)$ thỏa

$$h(x) \equiv [x = 0] \pmod{p}.$$

Không khó nhận ra khi đó

$$f(x) + h(x)(a_0 - f(0))$$

chính là đáp án.

Ta có thể dựng đa thức sau:

$$h(x) = \sum_{j=0}^{p-2} x^j.$$

Khi $x = 0$, giá trị của nó là 1; khi $x = 1$, giá trị của nó là p , nên theo nghĩa mô-đun giá trị là 0. Khi $x > 1$, theo công thức tổng cấp số nhân, đáp án là

$$\frac{x^{p-1} - 1}{x - 1}.$$

Theo định lý nhỏ Fermat, $x^{p-1} \equiv 1 \pmod{p}$, nên theo nghĩa mô-đun giá trị vẫn là 0. Thay trực tiếp nó vào công thức phía trên sẽ thu được đáp án.

Độ phức tạp thời gian $O(p \log p)$.

10.5.2 Nội suy nhiều điểm từ các điểm trị không liên tiếp

Đề bài. Cho các điểm trị $g_1, g_2, \dots, g_{n+1}$ của đa thức bậc n $f(x)$ tại $a_1, a_2, \dots, a_{n+1}$, hỏi từng hệ số của $f(x)$ lấy mô-đun p .

$$0 < n < p \leq 300000, \quad p \text{ là số nguyên tố}, \quad n < p - 2.$$

Bảo đảm $a_i > 0$, và các a_i đôi một khác nhau.

Thuật toán. Theo nội suy Lagrange, ta có thể rất tiện tính đáp án là

$$\sum_i g_i \prod_{j \neq i} \frac{x - a_j}{a_i - a_j}.$$

Đặt $F(x) = \prod_i (x - a_i)$, và biến đổi công thức một chút, đáp án có thể được rút gọn thành

$$\left(\sum_i \frac{g_i}{(x - a_i) F'(a_i)} \right) F(x).$$

Với phần tính $F'(a_i)$, ta có thể giải bằng tính giá trị tại nhiều điểm của đa thức; phần nhân với $F(x)$ cuối cùng có thể xử lý sau. Vì vậy, hiện tại ta chuyển nó thành hai bài toán con có giá trị hơn:

- Tính $F(x) = \prod_i (x - a_i)$.
- Tính $G(x) = \sum_i \frac{g_i}{x - a_i}$.

Bài toán con 1.

$$\begin{aligned}
F(x) &= \prod_i (x - a_i), \\
\frac{F(x)}{\prod_i (-a_i)} &= \prod_i \left(-\frac{x}{a_i} + 1 \right), \\
\ln \left(\frac{F(x)}{\prod_i (-a_i)} \right) &= \sum_i \ln \left(-\frac{x}{a_i} + 1 \right), \\
\ln' \left(\frac{F(x)}{\prod_i (-a_i)} \right) &= \sum_i \sum_j \frac{1}{(a_i)^{j+1}} x^j.
\end{aligned}$$

Theo công thức trên, bài toán con 1 có thể dùng một lần exp đa thức và một lần tích phân để chuyển thành bài toán con 2 trong độ phức tạp $O(p \log p)$. Tiếp theo ta chỉ thảo luận cách làm của bài toán con 2.

Bài toán con 2.

$$\begin{aligned}
G(x) &= \sum_i \frac{v_i}{x - a_i} \\
&= \sum_i v_i \sum_j \left(\frac{1}{a_i} \right)^{j+1} x^j \\
&= \sum_j x^j \sum_i v_i \left(\frac{1}{a_i} \right)^{j+1} \\
&= \sum_j x^j \sum_k \sum_i [\omega_{p-1}^k = a_i] v_i \omega_{p-1}^{-k(j+1)}.
\end{aligned}$$

Đặt

$$H(k) = \sum_i [\omega_{p-1}^k = a_i] v_i,$$

thì

$$\begin{aligned}
G(x) &= \sum_j x^j \sum_k H(k) \omega_{p-1}^{-k(j+1)}, \\
G(x)x &= \sum_j x^j \sum_k H(k) \omega_{p-1}^{-kj}.
\end{aligned}$$

Quan sát công thức cuối, không khó nhận ra nó thỏa tính chất của thuật toán Bluestein. Vì vậy, vẫn có thể dùng thuật toán Bluestein để tăng tốc tính $G(x)x$.

Do đó, cả hai bài toán con đều có thể giải trong độ phức tạp thời gian $O(p \log p)$, tổng độ phức tạp là $O(p \log p)$.

Trường hợp đặc biệt. Với trường hợp $a_i = 0$, vì trong quá trình suy luận khi giải bài toán con 1 và 2 ta đã dùng tới $\frac{v_i}{x}$ và đạo hàm của nó, nên suy luận phía trên sẽ sinh lỗi nghiêm trọng.

Một phương án xử lý là dùng cách dựng $g(x) = f(x + k)$ đã nhắc tới trong Ví dụ 3, tịnh tiến toàn bộ các a_i sao cho mọi a_i đều khác 0. Theo đó ta có thể tính được duy nhất các g_i . Cuối cùng tịnh tiến ngược lại đa thức $g(x)$ đã tính.

Một phương án khác là khi nội suy, bỏ qua điểm trị tại $a_i = 0$ để nội suy, cuối cùng cộng thêm một bội số của

$$\prod_{a_i \neq 0} (x - a_i)$$

sao cho điểm trị của $f(0)$ là đúng. Phương pháp này cũng được gọi là phương pháp nội suy Lagrange bộ phận.

10.5.3 Mở rộng

Bài toán con 2 có thể được xem là kết quả của việc lấy một vector cột đã cho rồi nhân bên phải với một ma trận Vandermonde có định thức khác 0. Thật ra, ta cũng có thể dùng thuật toán trên để giải ma trận Vandermonde khi mô-đun nhỏ.

Giả sử ta cần giải hệ phương trình

$$G(j) = \sum_i a_i x_i^j.$$

Tương tự bài toán 2, đặt

$$H(k) = \sum_i [\omega_{p-1}^k = x_i] a_i,$$

thì hệ phương trình có thể được chuyển thành

$$G(j) = \sum_k H(k) \omega_{p-1}^{kj}.$$

Tương tự các bài toán đã thảo luận trước đó, không khó nhận ra ta đã chuyển bài toán thành nội suy nhiều điểm của đa thức từ các điểm trị liên tiếp. Vì vậy, có thể trực tiếp dùng thuật toán này để giải trong độ phức tạp thời gian $O(p \log p)$.

10.6 Lời kết

Phần lớn các thuật toán phía trên đều là ứng dụng của thuật toán Bluestein trong trường mô-đun. Thật ra vào năm 2003, giới học thuật đã đề xuất một phép nghịch đảo của thuật toán Bluestein có độ phức tạp thời gian $O(n \log n)$. Độc giả quan tâm có thể tự tra cứu các bài báo liên quan.

10.7 Triển vọng

Bài viết này đã nhắc tới một số phương pháp và kỹ thuật để giải các bài toán biến đổi Fourier nhanh, nhưng trong đại dương thuật toán mênh mông, chúng vẫn chỉ là phần nổi của tảng băng. Hy vọng trong tương lai sẽ có nhiều ứng dụng hơn của thuật toán Bluestein được phát hiện và phát minh, và cũng hy vọng có thêm nhiều bài biến đổi Fourier nhanh thú vị xuất hiện trong Olympic Tin học.

10.8 Lời cảm ơn

Cảm ơn China Computer Federation đã cung cấp nền tảng học tập và giao lưu.

Cảm ơn thầy Chen Heli và thầy Dong Yehua của Trường Trung học số 1 Thiệu Hưng vì sự quan tâm và hướng dẫn.

Cảm ơn huấn luyện viên đội tuyển quốc gia Gao Wenyuan vì sự hướng dẫn và giúp đỡ.

Cảm ơn bạn Dai Jiangqi vì gợi mở về thuật toán nội suy nhiều điểm từ các điểm trị không liên tiếp.

Cảm ơn bạn Deng Mingyang và anh Kong Chaozhe đã đọc soát bài viết này.

Cảm ơn các thầy cô và bạn học khác từng giúp đỡ và gợi mở cho tôi.

Cảm ơn cha mẹ vì sự quan tâm, ủng hộ và chăm sóc chu đáo.

Tài liệu tham khảo

1. Zhang Jialin, *Phép nhân đa thức*, luận văn đội tuyển quốc gia năm 2002.
2. Mao Xiao, *Tìm hiểu lại biến đổi Fourier nhanh*, luận văn đội tuyển quốc gia năm 2016.
3. Bluestein, L. (1970). "A linear filtering approach to the computation of discrete Fourier transform".

11 Bài toán rừng hướng vào nhỏ nhất

Zhang Zheyu, Trường Trung học Học Quân Hàng Châu, tỉnh Chiết Giang

Tóm tắt

Bài toán rừng hướng vào nhỏ nhất là: với một đồ thị có hướng có trọng số, tìm rừng hướng vào nhỏ nhất với số cạnh cố định. Bài viết này xoay quanh bài toán rừng hướng vào nhỏ nhất và đưa ra một thuật toán có độ phức tạp thời gian $O(E \log E)$.

11.1 Dẫn nhập

Với một đồ thị có hướng có trọng số và một số tự nhiên k , ta cần tìm đồ thị con nhỏ nhất gồm đúng k cạnh, sao cho mỗi thành phần liên thông yếu của đồ thị con này đều là một cây hướng vào. Đây chính là bài toán rừng hướng vào nhỏ nhất. Bài toán này chỉ mới xuất hiện trong thi thuật toán vài năm gần đây, nên còn khá mới.

Từ hơn mười năm trước, các hệ thống chấm bài trong nước đã có bài toán cây phân nhánh nhỏ nhất. Tuy vậy cho tới hiện nay, các bài toán liên quan tới cây phân nhánh vẫn là một mảng tương đối ít phổ biến. Bài toán rừng hướng vào nhỏ nhất có tiềm năng nhất định, có thể trở thành một mô hình đồ thị được dùng rộng rãi, vì vậy đáng được nghiên cứu.

Hiện nay, cách giải bài toán rừng hướng vào nhỏ nhất thường dựa trên tối ưu lồi và thuật toán cây phân nhánh nhỏ nhất. Thuật toán đó kém hiệu quả, chức năng yếu và khó mở rộng. Bài viết này đề xuất một thuật toán thời gian $O(E \log E)$; thuật toán này hiệu quả hơn, mạnh hơn và dễ mở rộng hơn.

Bài viết gồm bảy mục, nội dung đại khái như sau.

Mục 2 định nghĩa chính xác bài toán rừng hướng vào nhỏ nhất.

Mục 3 giới thiệu ngắn gọn về matroid. Matroid là một nhánh của toán tổ hợp, hiện thường được dùng để giải nhiều bài toán tối ưu trong đồ thị. Mục này sẽ dùng matroid để mô tả bài toán rừng hướng vào nhỏ nhất và dựa vào matroid để phát hiện các tính chất.

Mục 4 đưa ra thuật toán dựa trên tối ưu lồi và cây phân nhánh nhỏ nhất. Bài toán cây phân nhánh nhỏ nhất đã được giải từ thập niên 1960; thuật toán giải nó được gọi là “thuật toán Chu–Liu” hoặc “Edmonds’ algorithm”. Bài toán cây phân nhánh nhỏ nhất và bài toán rừng hướng vào nhỏ nhất đều liên quan tới cây phân nhánh, nên có một số điểm giống nhau. Mục này sẽ giới thiệu chi tiết cách dùng tối ưu lồi để chuyển bài toán rừng hướng vào nhỏ nhất thành bài toán cây phân nhánh nhỏ nhất. Thuật toán này cần tìm kiếm nhị phân; số lần nhị phân quyết định độ chính xác, và khi số lần nhị phân là $\log \frac{1}{2\epsilon}$, độ phức tạp thời gian là

$$O\left(E \log E \log \frac{1}{2\epsilon}\right).$$

Mục 5 phân tích các mục trước và thu được thuật toán mở rộng cây hướng vào theo ưu tiên. Thuật toán dựa trên tối ưu lồi và cây phân nhánh nhỏ nhất có nhiều thao tác thừa; phân tích ý nghĩa của các thao tác đó giúp thuật toán gọn hơn. Mục này sẽ trình bày chi tiết cách thu được thuật toán mở rộng cây hướng vào theo ưu tiên, sau đó mô tả hình thức thuật toán, và cuối cùng giải thích ngắn gọn cách đạt độ phức tạp $O(E \log E)$.

Mục 6 nêu một ứng dụng của bài toán rừng hướng vào nhỏ nhất.

Mục 7 tổng kết nội dung bài viết.

11.2 Định nghĩa

Cho một đồ thị có hướng $G = (V, E)$, trong đó V là tập đỉnh và E là tập cạnh. Có hàm trọng số cạnh $w : E \rightarrow \mathbb{R}$.

Định nghĩa **cây hướng vào** là một thành phần liên thông yếu, không có chu trình, trong đó có một đỉnh gốc và mọi đỉnh không phải gốc có đúng một cạnh đi ra.

Định nghĩa **rừng hướng vào** là một đồ thị mà mỗi thành phần liên thông yếu đều là một cây hướng vào.

Định nghĩa **tập con cạnh** là một tập con của tập cạnh E .

Định nghĩa **đồ thị con sinh bởi cạnh** của một tập con cạnh E^* là đồ thị có mọi đỉnh của đồ thị gốc và mọi cạnh trong E^* .

Quy ước trọng số của một tập con cạnh E^* bằng tổng trọng số của các cạnh trong đó, tức

$$w(E^*) = \sum_{e \in E^*} w(e).$$

Khi đó, bài toán rừng hướng vào nhỏ nhất có thể được mô tả như sau: cho một đồ thị có hướng và một số nguyên k , tìm tập con cạnh kích thước k có trọng số nhỏ nhất, sao cho đồ thị con sinh bởi cạnh của nó là một rừng hướng vào.

Với cùng một đồ thị có hướng, ký hiệu đáp án của bài toán rừng hướng vào nhỏ nhất theo k là hàm $\text{MDF}(k)$.

11.3 Matroid

11.3.1 Định nghĩa

Có một hệ tập con $M = (S, \mathcal{I})$. Nếu M thỏa ba tiên đề sau, thì M là một matroid.

Tiên đề 1. $\emptyset \in \mathcal{I}$.

Tiên đề 2. Với mọi $A \subset B$, nếu $B \in \mathcal{I}$, thì $A \in \mathcal{I}$.

Tiên đề 3. Với mọi $|A| < |B|$, nếu $A \in \mathcal{I}$ và $B \in \mathcal{I}$, thì tồn tại $x \in B - A$ sao cho $A \cup \{x\} \in \mathcal{I}$.

Với matroid $M = (S, \mathcal{I})$, các phần tử của $\mathcal{I}$ được gọi là các tập độc lập.

11.3.2 Tính chất cắt cụt

Bổ đề 3.1. Cho matroid $M = (E, \mathcal{I})$ và một số tự nhiên bất kỳ s . Sau khi chỉ giữ các tập độc lập có kích thước không vượt quá s , ta vẫn thu được một matroid, tức $M^* = (E, \mathcal{I}^*)$ là matroid, trong đó

$$\mathcal{I}^* = \{S \in \mathcal{I} : |S| \leq s\}.$$

Chứng minh. Vì $s \geq 0$, tập rỗng là tập độc lập, tức $\emptyset \in \mathcal{I}^*$.

Với mọi $A \subset B$, nếu $B \in \mathcal{I}^*$, thì $B \in \mathcal{I}$, nên $A \in \mathcal{I}$. Lại vì $|A| \leq |B| \leq s$, nên $A \in \mathcal{I}^*$.

Với mọi $|A| < |B|$, nếu $A \in \mathcal{I}^*$ và $B \in \mathcal{I}^*$, thì $A \in \mathcal{I}$ và $B \in \mathcal{I}$. Do đó tồn tại $x \in B - A$ sao cho $A \cup \{x\} \in \mathcal{I}$. Vì $|A| < |B| \leq s$, ta có $|A| + 1 \leq s$, nên $A \cup \{x\} \in \mathcal{I}^*$. $\square$

11.3.3 Matroid và bài toán rừng hướng vào nhỏ nhất

Tiếp theo ta dùng matroid để biểu diễn bài toán rừng hướng vào nhỏ nhất.

Đặt $M_1 = (E, \mathcal{I}_1)$, trong đó E là tập cạnh, còn $\mathcal{I}_1$ là tập các tập con cạnh sao cho trong đồ thị con sinh bởi cạnh, nếu xem các cạnh là vô hướng thì không có chu trình.

Đặt $M_2 = (E, \mathcal{I}_2)$, trong đó E là tập cạnh, còn $\mathcal{I}_2$ là tập các tập con cạnh sao cho trong đồ thị con sinh bởi cạnh, bậc ra của mỗi đỉnh không vượt quá 1.

Ta lần lượt giải thích rằng M_1 và M_2 là matroid, tức cả hai đều thỏa ba tiên đề.

Với M_1 , rõ ràng Tiên đề 1 và Tiên đề 2 được thỏa. Xét Tiên đề 3: với mọi $|A| < |B|$, nếu $A \in \mathcal{I}_1$ và $B \in \mathcal{I}_1$, thì số thành phần liên thông của A lớn hơn số thành phần liên thông của B . Vì vậy chắc chắn tồn tại một cạnh $e \in B$ nối hai thành phần liên thông trong A ; đây chính là phần tử thuộc $B - A$ có thể thêm vào A . Do đó Tiên đề 3 cũng được thỏa, và M_1 là một matroid.

Với M_2 , rõ ràng Tiên đề 1 và Tiên đề 2 được thỏa. Xét Tiên đề 3: với mọi $|A| < |B|$, nếu $A \in \mathcal{I}_2$ và $B \in \mathcal{I}_2$, gọi $U(A)$ là tập các đỉnh có cạnh đi ra trong A , và $U(B)$ là tập các đỉnh có cạnh đi ra trong B . Khi đó $|U(A)| \leq |U(B)|$; chỉ cần chọn một cạnh đi ra trong B có đỉnh đầu thuộc $U(B) - U(A)$, rồi thêm cạnh đó vào A . Do đó Tiên đề 3 cũng được thỏa, và M_2 là một matroid.

Bổ đề 3.2. Với một tập cạnh bất kỳ E^* , ta có $E^* \in \mathcal{I}_1 \cap \mathcal{I}_2$ khi và chỉ khi đồ thị con sinh bởi cạnh của E^* là một rừng hướng vào.

Chứng minh. Trước hết chứng minh nếu $E^* \in \mathcal{I}_1 \cap \mathcal{I}_2$, thì đồ thị con sinh bởi cạnh của E^* là rừng hướng vào. Xét một thành phần liên thông yếu trong đồ thị con sinh bởi cạnh của E^* . Theo định nghĩa của $\mathcal{I}_1$, trong thành phần liên thông yếu này, số cạnh ít hơn số đỉnh một đơn vị. Theo định nghĩa của $\mathcal{I}_2$, mỗi đỉnh có số cạnh đi ra không vượt quá 1, nên trong đó đúng một đỉnh không có cạnh đi ra; gọi đỉnh đó là t . Khi đó mọi cạnh kề với t đều hướng tới t ; tiếp tục suy luận như vậy ta thu được một cây hướng vào gốc t . Vì mỗi thành phần liên thông yếu trong đồ thị con sinh bởi cạnh của E^* đều là cây hướng vào, đồ thị con sinh bởi cạnh của E^* là rừng hướng vào.

Chiều ngược lại suy trực tiếp từ định nghĩa của rừng hướng vào. $\square$

Tóm lại, ta đã mô tả bài toán rừng hướng vào nhỏ nhất bằng matroid, tức cần tìm

$$\min_{S \in \mathcal{I}_1 \cap \mathcal{I}_2, |S|=k} w(S).$$

Vì bài toán rừng hướng vào nhỏ nhất cố định số cạnh, ta có thể đồng thời cộng hoặc trừ cùng một lượng vào trọng số mọi cạnh. Để thuận tiện cho thảo luận bên dưới, giả sử

$$\max_{e \in E} w(e) \leq 0.$$

Do đó ta có thể chuyển bài toán thành

$$\min_{S \in \mathcal{I}_1 \cap \mathcal{I}_2, |S| \leq k} w(S).$$

11.3.4 Giao matroid và quy hoạch tuyến tính

Bài toán giao matroid. Định nghĩa bài toán giao matroid: cho hai matroid $M_1 = (E, \mathcal{I}_1)$ và $M_2 = (E, \mathcal{I}_2)$ định nghĩa trên cùng một tập nền, tìm

$$\min_{S \in \mathcal{I}_1 \cap \mathcal{I}_2} w(S).$$

Bài toán rừng hướng vào nhỏ nhất có thể được mô tả thành một bài toán giao matroid chuẩn. Đặt $M_2^* = (E, \mathcal{T}_2^*)$, trong đó

$$\mathcal{T}_2^* = \{S \in \mathcal{T}_2 : |S| \leq k\}.$$

M_2^* là một matroid. Khi đó

$$\min_{S \in \mathcal{T}_1 \cap \mathcal{T}_2^*, |S| \leq k} w(S) = \min_{S \in \mathcal{T}_1 \cap \mathcal{T}_2^*} w(S).$$

Quy hoạch tuyến tính. Bài toán quy hoạch tuyến tính có thể được mô tả như sau: cho một ma trận A và hai vector b, c ,

$$\begin{aligned} \min \quad & c^T x, \\ \text{s.t.} \quad & Ax \leq b, \\ & x \geq 0. \end{aligned}$$

Trong đó x là biến; dòng đầu là mục tiêu tối ưu, các dòng còn lại là ràng buộc đối với x .

Nếu chỉ quan tâm tới ràng buộc, ta thấy miền khả thi của x là một đa diện lồi $|c|$ chiều. Vai trò của vector c chỉ là quy định cuối cùng cần cực điểm theo hướng nào, tức điểm xa nhất theo một hướng nào đó.

Nếu với mọi c , trong nghiệm tối ưu x đều là điểm nguyên, tức mọi cực điểm trên đa diện lồi đều là điểm nguyên, thì quy hoạch tuyến tính như vậy được gọi là unimodular. Khi quy hoạch tuyến tính là unimodular, thông thường có thể bỏ qua ràng buộc nguyên trong bài toán cụ thể.

Định lý 3.1. Với một quy hoạch tuyến tính và một vector d , gọi $F(k)$ là giá trị của quy hoạch tuyến tính sau khi thêm một ràng buộc bất kỳ $d^T x \leq k$. Khi đó đạo hàm $F'(k)$ đơn điệu.

Hình 2.1: miền khả thi sau khi chiếu lên mặt phẳng sinh bởi c và d .

Chứng minh. Xét trước khi thêm ràng buộc, miền khả thi của x là một đa diện lồi. Sau khi chiếu nó lên mặt phẳng chứa c và d , ta chắc chắn thu được một bao lồi hai chiều, như Hình 2.1. Trên mặt phẳng này, nếu lấy $-c$ làm chuẩn, thì độ dốc tại tiếp điểm của d với bao lồi là đơn điệu, nên $F'(k)$ đơn điệu. $\square$

Liên hệ giữa hai khái niệm. Ký hiệu $X(S)$ là điểm trong không gian $|E|$ chiều mà mỗi tọa độ đều bằng 0 hoặc 1, trong đó tọa độ thứ i bằng 1 khi và chỉ khi phần tử thứ i thuộc S .

Đặt

$$X(M_1, M_2) = \{X(S) : S \in \mathcal{T}_1 \cap \mathcal{T}_2\}.$$

Gọi $\text{conv}(X(M_1, M_2))$ là đa diện lồi nhỏ nhất chứa tập điểm $X(M_1, M_2)$.

Đặt

$$\begin{aligned} P(M_1, M_2) = \{x \in \mathbb{R}^{|E|} : & x(S) \leq \max_{T \in \mathcal{T}_1, T \subset S} |T| \quad \forall S \subset E, \\ & x(S) \leq \max_{T \in \mathcal{T}_2, T \subset S} |T| \quad \forall S \subset E, \\ & x_e \geq 0 \quad \forall e \in E\}. \end{aligned}$$

Định lý 3.2.

$$\text{conv}(X(M_1, M_2)) = P(M_1, M_2).$$

Chứng minh của định lý này xem trong [2].

Định nghĩa điểm $X(w)$ trong không gian $|E|$ chiều thỏa $\forall e \in E, X(w)_e = w(e)$. Khi đó bài toán giao matroid có thể được xem là cực điểm của $X(M_1, M_2)$ theo hướng $-X(w)$, đồng thời cũng bằng cực điểm của $\text{conv}(X(M_1, M_2))$ theo hướng $-X(w)$.

Nếu xem các ràng buộc của $P(M_1, M_2)$ là ràng buộc của quy hoạch tuyến tính, và xem hàm trọng số w là hàm mục tiêu của quy hoạch tuyến tính, thì ta có thể chuyển bài toán giao matroid thành bài toán quy hoạch tuyến tính.

Tính chất lồi. Quay lại bài toán này. Gọi đáp án của bài toán rừng hướng vào nhỏ nhất với tập cạnh kích thước k là $\text{MDF}(k)$. Ta có hai matroid trên tập cạnh, và

$$\text{MDF}(k) = \min_{S \in \mathcal{T}_1 \cap \mathcal{T}_2, |S| \leq k} w(S).$$

Thêm vào $P(M_1, M_2)$ ràng buộc theo k , ta được $P(M_1, M_2, k)$. Đặt

$$\begin{aligned} P(M_1, M_2, k) = \{x \in \mathbb{R}^{|E|} : & x(S) \leq \max_{T \in \mathcal{T}_1, T \subset S} |T| \quad \forall S \subset E, \\ & x(S) \leq \max_{T \in \mathcal{T}_2, T \subset S} |T| \quad \forall S \subset E, \\ & x_e \geq 0 \quad \forall e \in E, \\ & \sum_{e \in E} x_e \leq k\}. \end{aligned}$$

Bổ đề 3.3. Mọi cực điểm của $P(M_1, M_2, k)$ đều là điểm nguyên.

Chứng minh. Xây dựng $M_2^* = (E, \mathcal{T}_2^*)$, trong đó $\mathcal{T}_2^* = \{S \in \mathcal{T}_2 : |S| \leq k\}$. M_2^* là matroid, nên $\text{conv}(X(M_1, M_2^*)) = P(M_1, M_2^*)$. Lại vì $P(M_1, M_2, k) = P(M_1, M_2^*)$, ta có

$$\text{conv}(X(M_1, M_2^*)) = P(M_1, M_2, k).$$

□

Vì vậy, sau khi thêm ràng buộc kích thước không vượt quá k , ta vẫn có thể bỏ qua ràng buộc nguyên và chuyển bài toán thành quy hoạch tuyến tính. Gọi $\text{MDF}^*(i)$ là giá trị của quy hoạch tuyến tính lấy $P(M_1, M_2, i)$ làm ràng buộc và lấy w làm hàm tối ưu. Khi đó

$$\forall i \in \mathbb{Z}, \quad \text{MDF}^*(i) = \text{MDF}(i).$$

Đặt $\sum_{e \in E} x_e \leq k$ ra ngoài quy hoạch tuyến tính, ta có $\text{MDF}^{*'}(i)$ là hàm không giảm, nên $\text{MDF}^*(i)$ là hàm lồi. Do đó $\text{MDF}(i)$ cũng là hàm lồi, tức

$$\forall i \in \mathbb{Z}, \quad \text{MDF}(i+1) - \text{MDF}(i) \leq \text{MDF}(i+2) - \text{MDF}(i+1).$$

11.3.5 Tiểu kết

Mục nhỏ này giới thiệu một số kiến thức về matroid và mô tả bài toán rừng hướng vào nhỏ nhất dưới dạng giao matroid. Cuối cùng, thông qua kiến thức quy hoạch tuyến tính, ta thu được một kết luận rất quan trọng: hàm của bài toán rừng hướng vào nhỏ nhất theo số cạnh là hàm lồi. Kết luận này sẽ giúp ích nhiều cho phần sau.

Thật ra, chỉ với nội dung của mục này, ta đã có thể giải bài toán rừng hướng vào nhỏ nhất trong thời gian đa thức. Vì ta có thể biểu diễn bài toán rừng hướng vào nhỏ nhất thành bài toán giao matroid, đồng thời có thể kiểm tra tập độc lập trong thời gian đa thức, nên có thể áp dụng trực tiếp thuật toán giao matroid có trọng số. Tuy nhiên độ phức tạp quá cao, nên không khuyến nghị cách này.

11.4 Thuật toán dựa trên tối ưu lồi và cây phân nhánh nhỏ nhất

11.4.1 Bài toán cây phân nhánh nhỏ nhất

Bài toán cây phân nhánh nhỏ nhất có thể được mô tả như sau.

Cho một đồ thị có hướng $G = (V, E)$, có trọng số cạnh $w : E \rightarrow \mathbb{R}$. Với gốc t cho trước, tìm tập con cạnh có trọng số nhỏ nhất sao cho đồ thị con sinh bởi cạnh của nó là một cây hướng vào gốc t .

11.4.2 Thuật toán Chu–Liu

Tư tưởng cốt lõi của “thuật toán Chu–Liu” là hai bổ đề sau.

Bổ đề 4.1. Trong bài toán cây phân nhánh nhỏ nhất, việc đồng thời cộng hoặc trừ cùng một lượng vào trọng số mọi cạnh đi ra từ một đỉnh không ảnh hưởng tới hình dạng cây của nghiệm tối ưu.

Chứng minh. Trong mọi cây sinh hướng vào, mỗi đỉnh không phải gốc đều có bậc ra bằng 1, nên yếu tố ảnh hưởng tới hình dạng cây tối ưu chỉ là tương quan trọng số giữa các cạnh đi ra từ cùng một đỉnh. $\square$

Bổ đề 4.2. Trong bài toán cây phân nhánh nhỏ nhất, nếu tồn tại một chu trình toàn cạnh trọng số 0, và mọi cạnh đi ra từ các đỉnh trên chu trình đều không âm, thì có thể xem chu trình này là một đỉnh.

Chứng minh. Bổ đề này khá trừu tượng. Cụ thể, ý nghĩa của nó là: với một chu trình toàn cạnh trọng số 0, chắc chắn tồn tại một nghiệm tối ưu sao cho đúng một cạnh trên chu trình không được chọn.

Vì cây hướng vào không có chu trình, ít nhất một cạnh trên chu trình này không được chọn.

Nếu tồn tại một nghiệm tối ưu H sao cho có ít nhất hai cạnh trên chu trình này không được chọn, gọi S là tập các đỉnh đầu của những cạnh đó. Khi đó trong đồ thị H , tồn tại ít nhất một đỉnh trong S mà đường đi từ nó tới gốc không chứa đỉnh nào trên chu trình; gọi đỉnh đó là S_1 . Với H , đổi cạnh đi ra của mỗi đỉnh trong $S - \{S_1\}$ thành cạnh trên chu trình, ta được H^* . Khi đó H^* chắc chắn là một cây sinh hướng vào và không tệ hơn H . Hình 3.1 bên trái là H , bên phải là H^* . $\square$

Hình 3.1: đổi các cạnh đi ra để trên chu trình trọng số 0 chỉ còn một cạnh không được chọn.

Dùng hai bổ đề trên, ta thu được thuật toán Chu–Liu. Sau đây là mô tả hình thức của thuật toán.

1. Chọn tùy ý một đỉnh u chưa xác định cạnh đi ra.
2. Dùng phép cộng trừ đồng thời trọng số các cạnh đi ra từ u để biến cạnh đi ra nhỏ nhất của u thành trọng số 0, rồi chọn cạnh đó làm cạnh đi ra của đỉnh này. Nếu các cạnh đi ra tạo thành chu trình, có chu trình này thành một đỉnh.
3. Nếu vẫn còn đỉnh chưa xác định cạnh đi ra thì quay lại bước 1.

Khi cài đặt, cần xây dựng heap gộp được cho mỗi đỉnh, hỗ trợ cộng trừ toàn cục và duy trì cạnh đi ra nhỏ nhất. Khi nhiều đỉnh được co thành một đỉnh, ta gộp các heap của chúng. Độ phức tạp thời gian là $O(E \log E)$.

11.4.3 Trường hợp đặc biệt của bài toán rừng hướng vào nhỏ nhất

Khi $k = |V| - 1$, bài toán rừng hướng vào nhỏ nhất có thể chuyển rất thuận tiện thành bài toán cây phân nhánh nhỏ nhất.

Khi $k = |V| - 1$, bài toán rừng hướng vào nhỏ nhất tương đương với việc yêu cầu một cây phân nhánh nhỏ nhất có gốc chưa xác định. Ta thêm một đỉnh mới t làm gốc của cây hướng vào, rồi nối từ mỗi đỉnh trong V tới nó một cạnh có cùng trọng số và đủ lớn. Khi đó trong mọi cây hướng vào nhỏ nhất gốc t , bậc vào của t đều đúng bằng 1; cuối cùng chỉ cần trừ đi trọng số của cạnh đó.

11.4.4 Tối ưu lồi

Giới thiệu. Tối ưu lồi là một kỹ thuật thường dùng trong thi thuật toán để tìm một hạng tử nào đó của một hàm lồi.

Điều kiện để dùng tối ưu lồi là: với mọi hệ số góc, ta có thể tìm tiếp điểm của hệ số góc đó trên hàm lồi. Trong thi thuật toán, cách tìm tiếp điểm thường là thêm một hàm bậc nhất vào hàm lồi rồi tìm cực trị toàn cục.

Bằng cách nhị phân hệ số góc, ta có thể tìm khá chính xác một điểm trên hàm lồi.

Cần chú ý rằng chỉ trong một số trường hợp đặc biệt, tối ưu lồi mới tìm được giá trị chính xác của một điểm trên hàm lồi. Khi tối ưu lồi chỉ tìm được giá trị xấp xỉ, độ chính xác có thể đạt mức nhỏ theo hàm mũ.

Ứng dụng vào bài toán rừng hướng vào nhỏ nhất. Từ trường hợp đặc biệt trên, định nghĩa $T(\alpha)$ là đáp án của bài toán cây phân nhánh nhỏ nhất sau khi thêm một đỉnh mới t làm gốc cây hướng vào và nối từ mỗi đỉnh trong V tới t một cạnh có trọng số α .

Xét cách tính $T(\alpha)$. Trước hết liệt kê số cạnh ngoài các cạnh đi vào đỉnh t , gọi số này là x . Chi phí nhỏ nhất của phần này là $\text{MDF}(x)$. Ngoài ra, x cạnh này tạo thành $|V| - x$ cây hướng vào không liên thông với nhau; gốc của chúng trở tới t bằng các cạnh trọng số α , nên chi phí của phần này là $(|V| - x)\alpha$. Do đó $T(\alpha)$ bằng giá trị nhỏ nhất của tổng hai phần chi phí:

$$T(\alpha) = \min_{x \in [0, |V|)} \text{MDF}(x) + (|V| - x)\alpha.$$

Từ mục trước, ta biết MDF là một hàm lồi, mở lên trên. Quan sát công thức này và khai triển ngoặc, ta có thể xem $T(\alpha)$ là giá trị nhỏ nhất sau khi thêm một hàm bậc nhất có hệ số góc $-\alpha$, tức có thể tìm giá trị nhỏ nhất của MDF sau khi thêm một hàm bậc nhất có hệ số góc tùy ý.

Giá trị nhỏ nhất sau khi thêm một hàm bậc nhất có hệ số góc $-\alpha$ tương đương với tiếp điểm trên hàm lồi có hệ số góc α . Đến đây ta có thể dùng tối ưu lồi để tìm một hạng tử của MDF .

11.4.5 Mô tả thuật toán

Ta thu được thuật toán dựa trên tối ưu lồi và cây phân nhánh nhỏ nhất. Sau đây là mô tả hình thức.

1. Nhị phân hệ số góc α .
2. Thêm đỉnh mới t , nối mọi đỉnh tới nó bằng cạnh trọng số α .
3. Từ đây trở đi, trong bước 4 và bước 5, dùng phép cộng trừ đồng thời trọng số cạnh đi ra của một đỉnh để cạnh đi ra nhỏ nhất của mỗi đỉnh luôn có trọng số 0.
4. Chọn tùy ý một đỉnh u chưa xác định cạnh đi ra.

5. Chọn cạnh đi ra nhỏ nhất của u làm cạnh đi ra của đỉnh này. Nếu các cạnh đi ra tạo thành chu trình, co chu trình đó thành một đỉnh.
6. Nếu vẫn còn đỉnh chưa xác định cạnh đi ra thì quay lại bước 4.
7. So sánh bậc vào của gốc với k , rồi quay lại bước 1 để nhị phân lần tiếp theo; hoặc nếu đã đạt độ chính xác mục tiêu thì dừng.

11.4.6 Tiêu kết

Thuật toán cây phân nhánh nhỏ nhất có độ phức tạp thời gian $O(E \log E)$, còn thuật toán tối ưu lỗi có độ phức tạp thời gian bằng logarit của độ chính xác. Vì vậy, cách chuyển bằng tối ưu lỗi thành bài toán cây phân nhánh nhỏ nhất có độ phức tạp

$$O\left(E \log E \log \frac{1}{2\varepsilon}\right),$$

trong đó ε là sai số tuyệt đối chia cho tổng giá trị tuyệt đối của trọng số cạnh, tức số lần nhị phân trong tối ưu lỗi là $\log \frac{1}{2\varepsilon}$.

11.5 Thuật toán mở rộng cây hướng vào theo ưu tiên

11.5.1 Tránh thêm đỉnh mới

Việc thêm đỉnh mới làm mất nhiều tính chất trực quan, nên ta xét cách không thêm đỉnh mới.

Trong thuật toán ở mục 4.5, ở bước 5, nếu sau khi u chọn cạnh đi ra thì tạo chu trình và phải co chu trình, ta gọi thao tác này là **co vào**; ngược lại gọi là **mở rộng**. Gọi một cặp bước 4 và bước 5 là một vòng thao tác co-mở cơ bản.

Quan sát thấy rằng sau khi một đỉnh u thực hiện thao tác co vào, đỉnh thu được sau khi co vẫn là một đỉnh đang chờ chọn cạnh đi ra. Nếu với một cây hướng vào hiện tại chưa từng chọn gốc, ta liên tục chọn gốc của cây hướng vào này cho tới khi nó mở rộng, thì ta sẽ thu được chi phí nhỏ nhất để dẫn một cạnh đi ra từ cây hướng vào đó; gọi chi phí này là **chi phí mở rộng** của cây hướng vào. Cụ thể, chi phí mở rộng là mọi chi phí từ khi gốc của một cây hướng vào chưa từng thực hiện thao tác co-mở cơ bản cho tới khi cây hướng vào dẫn được cạnh ra ngoài, bao gồm cả chi phí ban đầu để biến cạnh đi ra nhỏ nhất của gốc thành 0.

Đỉnh mới t có các cạnh từ mọi đỉnh tới nó với cùng trọng số α . Nhưng ý nghĩa của đỉnh mới không chỉ là: khi chi phí mở rộng của một cây hướng vào không nhỏ hơn α , cây hướng vào đó trở thành tới t .

Sau đây là một ví dụ để giải thích rõ hơn điểm này.

Hình 4.1.1–4.1.4: ví dụ bốn đỉnh cho thấy một cây có chi phí mở rộng nhỏ hơn α vẫn có thể trở tới đỉnh mới t sau các thao tác co-mở.

Như Hình 4.1.1, đây là một đồ thị bốn đỉnh. Hiện đã thêm đỉnh t , và $\alpha = 4$, tức mọi đỉnh đều có một cạnh trọng số 4 tới t (không vẽ trong hình).

Trước hết thực hiện thao tác co-mở cơ bản với đỉnh 3, như Hình 4.1.2. Sau đó thực hiện thao tác co-mở cơ bản với đỉnh 2, như Hình 4.1.3. Lúc này, chi phí mở rộng của cây hướng vào có gốc là đỉnh 1 bằng 3, nhỏ hơn α , nhưng kết quả sau khi thực hiện thao tác co-mở cơ bản với nó lại là trở tới t . Nguyên nhân là trong cây hướng vào này tồn tại một đỉnh có cạnh đi ra trên cây hướng vào rất lớn, dẫn tới khả năng trước hết co nó vào gốc rồi để đỉnh đó trở tới t .

Điều này buộc ta đưa vào một khái niệm: **giá trị trừ trước** của đỉnh, ký hiệu là dec.

Chú ý rằng bất kể trọng số cạnh trên đồ thị thay đổi thế nào, lượng thay đổi của các cạnh đi ra từ cùng một đỉnh ban đầu đều giống nhau. Những đỉnh ban đầu bị co không biến mất vô cớ; sau khi đồng thời thay đổi mọi cạnh đi ra, ta gộp các cạnh đi ra của chúng với các đỉnh bị co khác. Do đó, định nghĩa giá trị trừ trước của một đỉnh là lượng giảm của các cạnh đi ra từ mỗi đỉnh ban đầu, trong trạng thái hiện tại, sau khi biến cạnh đi ra nhỏ nhất thành 0.

Quan sát biểu hiện thực tế của giá trị trừ trước trong thao tác co-mở cơ bản: ta gộp vài tập hợp, rồi đồng thời tăng mọi giá trị trừ trước trong tập hợp sau khi gộp. Với đỉnh mới sau khi co, ta định nghĩa giá trị trừ trước của đỉnh mới là giá trị trừ trước lớn nhất trong các đỉnh của nó. Khi đó tại mọi thời điểm, một đỉnh u đều có một cạnh tới t với trọng số $\alpha - \text{dec}(u)$.

Bổ đề 5.1. Trong bài toán rừng hướng vào nhỏ nhất, cho một đồ thị có hướng có trọng số $G = (V, E, w)$. Nếu chỉ thực hiện các thao tác co-mở cơ bản và tạo thành một cây hướng vào H , đồng thời giá trị trừ trước của gốc H là lớn nhất trong cây hướng vào đó, thì H là cây sinh hướng vào có gốc chưa xác định nhỏ nhất của đồ thị con sinh bởi các đỉnh của nó.

Chứng minh. Chỉ cần chứng minh rằng nếu thêm đỉnh mới t và nối mọi đỉnh tới t bằng một cạnh trọng số đủ lớn α , rồi liên tục thực hiện thao tác co-mở cơ bản với gốc của H , thì cuối cùng ta sẽ chọn để gốc trở thẳng tới t . Giả sử gốc là r , và tập đỉnh của gốc sau khi co là R . Bản chất của thao tác co vào chỉ là: liên tục thêm vào R các phần tử có giá trị trừ trước nhỏ hơn rồi cộng toàn cục. Lặp như vậy, phần tử có giá trị trừ trước lớn nhất trong R luôn là r . $\square$

Bổ đề 5.2. Trong bài toán cây phân nhánh nhỏ nhất, nếu gốc t chỉ có các cạnh từ đỉnh khác trở tới nó, và các cạnh đó đều có trọng số α . Nếu chỉ thực hiện các thao tác co-mở cơ bản, thì với một cây hướng vào H mà gốc của nó là đỉnh có dec lớn nhất trên cây, thao tác co-mở cơ bản với nó sẽ trở thẳng tới t khi và chỉ khi, sau khi bỏ qua t , chi phí mở rộng của nó lớn hơn α .

Chứng minh. Vì H là cây sinh hướng vào có gốc chưa xác định nhỏ nhất trong đồ thị con sinh bởi các đỉnh của nó, nên H hoặc mở rộng tới một đỉnh không phải t , hoặc gốc của H trở thẳng tới t . $\square$

Vậy nếu cưỡng ép giá trị trừ trước của gốc là lớn nhất, ý nghĩa của t chính là một ràng buộc đối với chi phí mở rộng.

Sau khi hiểu vai trò của t , nếu ta có thể bảo đảm tại mọi thời điểm, gốc của mỗi cây hướng vào đều là đỉnh có giá trị trừ trước lớn nhất, thì có thể biểu diễn tường minh ràng buộc của t . Để làm được điều đó, trước hết thao tác co vào không ảnh hưởng tới vị thế lớn nhất của gốc, nên chỉ cần thay đổi hợp lý thứ tự các thao tác mở rộng. Với hai cây hướng vào A, B , có gốc lần lượt là t_A, t_B , và $\text{dec}(t_A) \leq \text{dec}(t_B)$, việc cho A trở tới B có thể giữ cho giá trị trừ trước của gốc vẫn là lớn nhất.

Sửa nhẹ thao tác co-mở cơ bản bằng cách thêm ưu tiên theo dec: ưu tiên chọn đỉnh có dec nhỏ nhất. Gọi thao tác như vậy là **thao tác co-mở theo ưu tiên**.

Bổ đề 5.3. Trong bài toán cây phân nhánh nhỏ nhất, nếu gốc t chỉ có các cạnh từ đỉnh khác trở tới nó, và các cạnh đó đều có trọng số α . Nếu chỉ thực hiện thao tác co-mở theo ưu tiên, thì một cây hướng vào sẽ trở thẳng tới t trong nghiệm tối ưu khi và chỉ khi, sau khi bỏ qua t , chi phí mở rộng của nó lớn hơn α .

Chứng minh. Với điều kiện chỉ thực hiện thao tác co-mở theo ưu tiên, gốc của mỗi cây hướng vào đều có dec lớn nhất trong cây. $\square$

Bây giờ có thể mô tả hình thức thuật toán không thêm đỉnh t .

1. Nhị phân hệ số góc α .
2. Từ đây trở đi, trong bước 3 và bước 4, dùng phép cộng trừ đồng thời trọng số các cạnh đi ra từ một đỉnh để cạnh đi ra nhỏ nhất của mỗi đỉnh luôn giữ trọng số 0, đồng thời duy trì giá trị trừ trước.
3. Chọn đỉnh u chưa chọn cạnh đi ra và có giá trị trừ trước nhỏ nhất. Nếu chi phí mở rộng của nó lớn hơn α , thì trực tiếp cộng $\alpha - \text{dec}(u)$ vào đáp án, xóa cây hướng vào này và bỏ qua bước 4.
4. Chọn cạnh đi ra nhỏ nhất của u làm cạnh đi ra của đỉnh này. Nếu các cạnh đi ra tạo thành chu trình, co chu trình thành một đỉnh.
5. Nếu vẫn còn đỉnh chưa xác định cạnh đi ra thì quay lại bước 3.

11.5.2 Tránh tối ưu lỗi

Xét quan hệ giữa chi phí mở rộng và giá trị trừ trước. Ta thấy chi phí mở rộng chính là giá trị trừ trước của gốc tại thời điểm mở rộng. Một kết luận khác dễ thấy là: nếu nối cây hướng vào A vào cây hướng vào B , thì các thao tác co vào của cây hướng vào B không đổi. Vì vậy với cây hướng vào có chi phí mở rộng nhỏ nhất hiện tại, ta có thể thực hiện trước các thao tác của cây đó cho tới khi nó mở rộng. Do đó, rừng hướng vào thu được bằng cách chọn co-mở theo ưu tiên tương đương với cách chọn sau:

- Chọn cây hướng vào có chi phí mở rộng nhỏ nhất, rồi chọn gốc u của nó.

Cũng vì nối cây hướng vào A vào cây hướng vào B không làm thay đổi các thao tác co vào của cây hướng vào B , nên chi phí mở rộng của cây hướng vào B không giảm. Khi đó trong một dãy các thao tác co-mở theo ưu tiên, chi phí mở rộng của các cây hướng vào được chọn chỉ tăng không giảm. Với một dãy thao tác tăng như vậy, việc nhị phân α mất ý nghĩa.

Đồng thời khi bỏ nhị phân, cũng không khó nhận ra: vì vị trí nhị phân trước đó là nghiệm tối ưu với kích thước tập cạnh hiện tại, nên tại mọi thời điểm trong quá trình này, ta đều đang có nghiệm tối ưu với kích thước tập cạnh hiện tại. Đã như vậy, tốt hơn là tiện thể tính đáp án cho mọi kích thước tập cạnh.

11.5.3 Thuật toán cụ thể và phân tích độ phức tạp

Trước khi mô tả hình thức, hãy xem một ví dụ để nhớ lại cách thuật toán vận hành.

Hình 4.2.1–4.2.5: ví dụ năm đỉnh và năm cạnh minh họa quá trình lần lượt mở rộng các cây hướng vào có chi phí mở rộng nhỏ nhất.

Ví dụ này như Hình 4.2.1, có 5 đỉnh và 5 cạnh. Ban đầu mỗi đỉnh là một cây hướng vào độc lập; trong đó đỉnh 1 có chi phí mở rộng nhỏ nhất, sau khi mở rộng thu được Hình 4.2.2. Tương tự, sau khi đỉnh 2 mở rộng, ta thu được Hình 4.2.3.

Lúc này có các cây hướng vào $\{1, 2, 3\}$, $\{4\}$ và $\{5\}$, chi phí mở rộng của chúng lần lượt là 6, $+\infty$ và 5, nên chọn đỉnh 5 để mở rộng, thu được Hình 4.2.4. Cuối cùng, chọn cây hướng vào $\{1, 2, 3\}$ để mở rộng, thu được Hình 4.2.5.

Mô tả thuật toán.

1. Ban đầu có $|V|$ cây hướng vào, mỗi cây hướng vào là một đỉnh.
2. Gọi tập cạnh hiện tại là E_{now} , ghi nhận $\text{MDF}(|E_{\text{now}}|) = w(E_{\text{now}})$.
3. Tính chi phí mở rộng hiện tại của mỗi cây hướng vào.

4. Nếu không còn cây hướng vào nào có thể dẫn một cạnh ra ngoài thì dừng.
5. Chọn cây hướng vào có chi phí mở rộng nhỏ nhất, thực hiện một số thao tác co-mở cho tới khi nó mở rộng, rồi quay lại bước 2.

Phân tích độ phức tạp. Nút thắt của cách cài đặt trực tiếp nằm ở việc tính chi phí mở rộng của mỗi cây hướng vào. Vì nối cây hướng vào A vào cây hướng vào B không làm thay đổi các thao tác co vào của cây hướng vào B , ta có thể xử lý trước mọi thao tác co vào của mỗi cây hướng vào. Như vậy với một cây hướng vào, ta tính được chi phí mở rộng của nó trong $O(1)$.

Có thể dùng heap gộp được để thực hiện thao tác co vào và duy trì cạnh đi ra; phần này có độ phức tạp thời gian $O(E \log E)$. Để tiện tìm cây hướng vào có chi phí mở rộng nhỏ nhất, cần thêm một heap duy trì chi phí mở rộng của các cây hướng vào; phần này có độ phức tạp thời gian $O(V \log V)$.

Vì vậy tổng độ phức tạp thời gian là $O(E \log E)$.

11.5.4 Một chứng minh đúng đắn khác

Ở mục 2, ta có thể mô tả bài toán rừng hướng vào nhỏ nhất thành bài toán giao matroid có trọng số.

Định lý 5.1. Với một bài toán giao matroid có trọng số bất kỳ, nếu bài toán có nghiệm ở kích thước k , thì với mọi nghiệm tối ưu B kích thước $k - 1$, tồn tại ít nhất một nghiệm tối ưu A kích thước k sao cho:

1. Nếu với mọi (u, v) thỏa $u \in B - A, v \in A - B, B - \{u\} + \{v\} \in \mathcal{T}_1$ nhưng $B + \{v\} \notin \mathcal{T}_1$, ta nối một cạnh giữa u và v , thì tồn tại $x \in A - B$ thỏa $B + \{x\} \in \mathcal{T}_1$, sao cho $A - B - \{x\}$ có ghép hoàn hảo tới $B - A$.
2. Nếu với mọi (u, v) thỏa $u \in B - A, v \in A - B, B - \{u\} + \{v\} \in \mathcal{T}_2$ nhưng $B + \{v\} \notin \mathcal{T}_2$, ta nối một cạnh giữa u và v , thì tồn tại $y \in A - B$ thỏa $B + \{y\} \in \mathcal{T}_2$, sao cho $A - B - \{y\}$ có ghép hoàn hảo tới $B - A$.

Định lý trên là một phần của thuật toán giao matroid có trọng số, nên ở đây không chứng minh.

Hãy nhớ lại hai matroid dùng để mô tả bài toán rừng hướng vào nhỏ nhất: $\mathcal{T}_1$ là các đồ thị không có chu trình nếu xem cạnh là vô hướng, còn $\mathcal{T}_2$ là các đồ thị mà mỗi đỉnh có bậc ra 0 hoặc 1.

Quan sát matroid thứ hai. Nếu hiện đã biết một nghiệm tối ưu B kích thước $k - 1$, theo định lý ta biết tồn tại một nghiệm tối ưu A kích thước k . Với một cạnh từ u tới v , từ $B + v \notin \mathcal{T}_2$ suy ra đỉnh đầu của v là đỉnh đầu của một cạnh trong B . Tương tự, đỉnh đầu của mọi cạnh trong $A - B - y$ đều là đỉnh đầu của một cạnh nào đó trong B . Vì vậy so với B , trong A , xét về số cạnh đi ra, đúng một đỉnh chuyển từ 0 thành 1.

Bổ đề 5.4. Trong bài toán rừng hướng vào nhỏ nhất, nếu biết trong nghiệm tối ưu, tập các đỉnh có cạnh đi ra là S , thì lần lượt thực hiện thao tác co-mở cơ bản với mỗi phần tử của S cho tới khi mở rộng sẽ thu được nghiệm tối ưu.

Chứng minh. Vì mỗi đỉnh trong $V - S$ đều không có cạnh đi ra, ta có thể gộp tập này thành một đỉnh t . Khi đó bài toán gốc chuyển thành bài toán cây phân nhánh nhỏ nhất. $\square$

Vậy nếu đã biết B và muốn tìm A , ta có thể liệt kê đỉnh nào có bậc ra chuyển từ 0 thành 1, tính chi phí để thực hiện thao tác co-mở cơ bản với nó cho tới khi mở rộng, rồi cuối cùng lấy đỉnh có chi phí nhỏ nhất. Chi phí này tương đương với chi phí mở rộng của một cây hướng vào. Do đó quá trình này tương đương với thuật toán mở rộng cây hướng vào theo ưu tiên.

11.5.5 Tiểu kết

Tới đây, ta đã giải quyết khá trọn vẹn bài toán rừng hướng vào nhỏ nhất. Hơn nữa, thuật toán này có thể đồng thời tính đáp án cho mọi k .

Ngoài ra, thuật toán này có thể vẫn còn không gian tối ưu tiếp, chẳng hạn dùng các cấu trúc dữ liệu ít gặp trong thi thuật toán như soft heap.

11.6 Ứng dụng

Bài J của 2020 Multi-University Training Contest 4 là một ứng dụng rất tốt của thuật toán này. Là người ra bài, tôi hy vọng thông qua bài này có thể phổ biến cách giải bài toán rừng hướng vào nhỏ nhất.

11.6.1 Ý chính bài toán

Cho n câu đố, đánh số từ 1 tới n . Cần chia các câu đố này thành k dãy không rỗng. Mỗi câu đố có một trọng số A , ban đầu $\forall i, A_i = 0$.

Tiếp theo có m lần phán định, mỗi lần có dạng (x, l, r, c) , nghĩa là nếu trong dãy chứa x , tồn tại $y \in [l, r]$ xuất hiện trước x , thì

$$A_x = \max(A_x, c).$$

Cần lập phương án chia thành k dãy không rỗng và sắp thứ tự trong mỗi dãy, sao cho tối đa hóa

$$\sum_{i \in [1, n]} A_i.$$

11.6.2 Dựng đồ thị

Dựng đồ thị: với mỗi câu đố tạo một đỉnh, và nối một cạnh trọng số 0 giữa hai đỉnh bất kỳ. Với mỗi lần phán định, từ x nối một cạnh có hướng trọng số c tới mọi đỉnh trong $[l, r]$. Đáp án chính là rừng hướng vào lớn nhất có số thành phần liên thông yếu bằng k .

Tiếp theo chứng minh tính đúng đắn của cách dựng đồ thị, gồm hai phần.

Phần thứ nhất: với một rừng hướng vào bất kỳ có số thành phần liên thông yếu bằng k , ta đều tìm được phương án chia thành k dãy tương ứng. Thật ra chỉ cần với mỗi thành phần liên thông yếu, lấy một thứ tự topo ngược tùy ý làm dãy.

Phần thứ hai: với một phương án chia thành k dãy bất kỳ, ta đều tìm được rừng hướng vào tương ứng có số thành phần liên thông yếu bằng k . Chỉ cần với mỗi câu đố i , giữ lại một cạnh bất kỳ làm cho A_i lớn nhất.

Sau khi đổi dấu trọng số cạnh, ta đã chuyển bài toán gốc thành bài toán rừng hướng vào nhỏ nhất.

11.6.3 Phân tích

Quan sát thuật toán mở rộng cây hướng vào theo ưu tiên: nếu không duyệt qua mọi vòng tự khép, thì sau khi duyệt nhiều nhất $O(V)$ cạnh, thuật toán chắc chắn dừng. Vì với mỗi cạnh không phải vòng tự khép, nếu co vào thì số đỉnh giảm; nếu không thì mở rộng, số thành phần liên thông yếu giảm. Vì vậy hướng tối ưu của ta là tránh liệt kê quá nhiều vòng tự khép trong thời gian hữu hạn.

Chú ý rằng trên đồ thị này, tuy số cạnh rất lớn, nhưng nếu dùng (x, l, r, c) để mô tả một nhóm cạnh thì số nhóm cạnh trên đồ thị rất nhỏ. Do đó với mỗi đỉnh đã co, dùng cấu trúc dữ liệu duy trì tập đỉnh ban đầu của nó. Khi truy vấn cạnh đi ra của đỉnh này, ta có thể tìm rất nhanh đỉnh nhỏ nhất trong $[l, r]$

không thuộc tập đỉnh đó, nhằm tránh liệt kê vòng tự khép. Khi đó, ngoài việc mỗi nhóm cạnh ít nhất bị liệt kê một lần để kiểm tra liệu toàn bộ có phải vòng tự khép hay không, thuật toán chỉ duyệt $O(V)$ cạnh.

Độ phức tạp thời gian của thuật toán là

$$O((n + m) \log(n + m)).$$

11.7 Tổng kết

Bài viết xoay quanh bài toán rừng hướng vào nhỏ nhất, lần lượt giới thiệu matroid, bài toán cây phân nhánh nhỏ nhất và kiến thức liên quan tới tối ưu lồi, đồng thời đề xuất thuật toán mở rộng cây hướng vào theo ưu tiên, giải quyết khá tốt bài toán rừng hướng vào nhỏ nhất. Thuật toán này có mã nguồn ngắn gọn, ít chi tiết cài đặt, có thể dùng để giải loại bài này trong thi thuật toán. Trong lĩnh vực lý thuyết đồ thị, bài toán rừng hướng vào nhỏ nhất, với tư cách một mở rộng của bài toán cây phân nhánh nhỏ nhất, cũng có giá trị lý thuyết quan trọng.

Tài liệu tham khảo

1. Yang Qianlan, *Bàn về một số mở rộng của matroid và ứng dụng của chúng*. Tập luận văn đội tuyển quốc gia Trung Quốc IOI2018, 2018.
2. Michel X. Goemans. Massachusetts Institute of Technology 18.433, *Combinatorial Optimization: Lecture notes on matroid intersection*. MIT Mathematics, <http://math.mit.edu/~goemans/18433S11/matroid-intersect-notes.pdf>, p. 7.
3. Wikipedia contributors. *Matroid*. Wikipedia, <https://en.wikipedia.org/wiki/Matroid>.
4. Wikipedia contributors. *Edmonds' Algorithm*. Wikipedia, https://en.wikipedia.org/wiki/Edmonds%27_algorithm.

12 Bàn về suffix automaton nén

Xu Yixuan, Trường Trung học cao cấp Thường Châu, Giang Tô

Tóm tắt

Bài viết xuất phát từ một loạt cấu trúc dữ liệu hậu tố truyền thống trong thi tin học, rồi dẫn ra một cấu trúc dữ liệu hậu tố mới: suffix automaton nén, cùng một biến thể của nó là suffix automaton nén đối xứng. Với cả hai cấu trúc, bài viết phân tích chi tiết các tính chất, đưa ra một thuật toán xây dựng có độ phức tạp thời gian và không gian tuyến tính khi được phép xử lý offline, đồng thời giới thiệu các ứng dụng thực tế.

Trong cộng đồng thi tin học hiện nay, sự quan tâm của học sinh đối với suffix automaton vẫn rất lớn. Hy vọng suffix automaton nén được giới thiệu trong bài viết này, cùng cách suy nghĩ phía sau nó, có thể gợi mở thêm nhiều suy nghĩ và tìm tòi về cấu trúc dữ liệu hậu tố, cũng như về các bài toán rộng hơn.

12.1 Lời nói đầu

Cấu trúc dữ liệu hậu tố là một hệ thống đang được dùng rộng rãi trong thi thuật toán. Nó bao gồm những nội dung quen thuộc như suffix array, trie hậu tố, suffix automaton, suffix tree, v.v. Các cấu trúc dữ liệu hậu tố này có thể xử lý hiệu quả nhiều bài toán về xâu.

Nếu hiểu quan hệ giữa các cấu trúc dữ liệu hậu tố này từ góc nhìn lý thuyết automaton, ta có thể thấy: tối tiểu hóa trie hậu tố sẽ thu được suffix automaton, còn co nó lại sẽ thu được suffix tree. Nếu đồng thời tối tiểu hóa và co trie hậu tố, ta sẽ thu được suffix automaton nén mà bài viết này giới thiệu.

Từ đó, tác giả bắt đầu nghiên cứu suffix automaton nén và viết bài này.

Cấu trúc bài viết như sau:

- Mục 2 giới thiệu một số định nghĩa cơ bản về xâu và automaton.
- Mục 3 ôn lại các cấu trúc dữ liệu hậu tố truyền thống trong OI, đồng thời giới thiệu quan hệ giữa chúng.
- Mục 4 giới thiệu chi tiết định nghĩa, cách xây dựng và ứng dụng của suffix automaton nén.
- Mục 5 giới thiệu biến thể của nó: định nghĩa, cách xây dựng và ứng dụng của suffix automaton nén đối xứng.
- Mục 6 phân tích và tổng kết các nội dung trong bài.

12.2 Định nghĩa cơ bản

12.2.1 Cơ sở về xâu

Gọi S là một xâu. Dùng $|S|$ để biểu thị số ký tự trong S , tức độ dài của S .

Nếu $|S| = 0$, ta gọi S là xâu rỗng, ký hiệu $S = \emptyset$.

Ký hiệu $S[i]$ là ký tự thứ i trong S , trong đó $1 \leq i \leq |S|$.

Ký hiệu $S[l, r]$ là xâu gồm các ký tự từ thứ l tới thứ r trong S , gọi là một xâu con của S .

Nếu $1 \leq l \leq r \leq |S|$, thì $S[l, r]$ là một xâu con không rỗng; ngược lại, $S[l, r] = \emptyset$.

Ký hiệu $\text{Pre}[i] = S[1, i]$, tức phần gồm ký tự thứ i và các ký tự đứng trước nó trong S ; đây là một tiền tố của S .

Ký hiệu $\text{Suf}[i] = S[i, |S|]$, tức phần gồm ký tự thứ i và các ký tự đứng sau nó trong S ; đây là một hậu tố của S .

12.2.2 Ngữ cảnh

Trong bài viết này, mọi bài toán được xét đều hướng tới một xâu mẹ duy nhất S . Vì vậy, các ký hiệu được quy ước dưới đây thường lược bỏ S , và chỉ ghi xâu con nào đó của S trong ký hiệu.

Định nghĩa 2.2.1. Với xâu con $S[l, r]$ của S , định nghĩa ngữ cảnh trái $\text{LeftContext}(S[l, r])$ của nó là

$$S[\min\{x \mid \forall S[l, r] = S[y - (r - l), y], S[x, r] = S[y - (r - x), y]\}, r].$$

Tương tự, định nghĩa ngữ cảnh phải $\text{RightContext}(S[l, r])$ của nó là

$$S[l, \max\{x \mid \forall S[l, r] = S[y, y + (r - l)], S[l, x] = S[y, y + (x - l)]\}].$$

Nói một cách thông tục, ngữ cảnh trái của một xâu con $S[l, r]$ là xâu dài nhất thu được bằng cách ghép phần chắc chắn xuất hiện ngay trước mọi lần xuất hiện của $S[l, r]$ trong S với $S[l, r]$. Ngữ cảnh phải của $S[l, r]$ là xâu dài nhất thu được bằng cách ghép $S[l, r]$ với phần chắc chắn xuất hiện ngay sau mọi lần xuất hiện của $S[l, r]$ trong S .

Theo cách mô tả này, không khó nhận ra

$$\text{LeftContext}(\text{RightContext}(S[l, r])) = \text{RightContext}(\text{LeftContext}(S[l, r])).$$

Định nghĩa 2.2.2. Với xâu con $S[l, r]$, định nghĩa ngữ cảnh $\text{Context}(S[l, r])$ của nó là

$$\text{LeftContext}(\text{RightContext}(S[l, r])).$$

Như vậy, ngữ cảnh của một xâu con là xâu dài nhất chắc chắn xuất hiện mỗi khi xâu con đó xuất hiện trong S .

Định nghĩa 2.2.3. Với xâu con $S[l, r]$ của S , định nghĩa tập trái $\text{Left}(S[l, r])$ của nó là

$$\{x \mid S[x, x + (r - l)] = S[l, r]\}.$$

Tương tự, định nghĩa tập phải $\text{Right}(S[l, r])$ của nó là

$$\{x \mid S[x - (r - l), x] = S[l, r]\}.$$

Có thể thấy tập trái của một xâu con chính là tập các đầu trái của mọi lần xuất hiện của nó trong S , còn tập phải là tập các đầu phải của mọi lần xuất hiện của nó trong S .

Ví dụ 2.2. Xét xâu $S = \text{aabaaba}$. Khi đó

$$\begin{aligned} \text{LeftContext}(a) &= a, & \text{LeftContext}(aa) &= aa, & \text{LeftContext}(b) &= aab, \\ \text{RightContext}(a) &= a, & \text{RightContext}(aa) &= aaba, & \text{RightContext}(b) &= ba, \\ \text{Context}(a) &= a, & \text{Context}(aa) &= aaba, & \text{Context}(b) &= aaba. \end{aligned}$$

Với xâu con ab của S , các vị trí xuất hiện của nó là $\{[2, 3], [5, 6]\}$, do đó

$$\text{Left}(ab) = \{2, 5\}, \quad \text{Right}(ab) = \{3, 6\}.$$

12.2.3 Automaton

Định nghĩa 2.3. Một automaton hữu hạn xác định (DFA) A gồm:

- một tập trạng thái hữu hạn không rỗng Q ;
- một bảng chữ cái đầu vào Σ , tức một tập ký tự hữu hạn không rỗng;
- một hàm chuyển $\delta : Q \times \Sigma \rightarrow Q$, chẳng hạn $\delta(q, \sigma) = p$, với $p, q \in Q, \sigma \in \Sigma$;
- một trạng thái bắt đầu $s \in Q$;
- một tập trạng thái chấp nhận $F \subseteq Q$.

Vì vậy, một DFA có thể viết dưới dạng $A = (Q, \Sigma, \delta, s, F)$.

Nói đơn giản, có thể xem một DFA là một đồ thị có hướng hữu hạn, với tập đỉnh là Q , và trên mỗi cạnh ghi một ký tự thuộc Σ . Có một đỉnh đặc biệt s gọi là trạng thái bắt đầu, ngoài ra còn có một loạt đỉnh đặc biệt F gọi là trạng thái chấp nhận. Trong bài viết này, ta nghiên cứu các automaton có thể chấp nhận mọi xâu con của một xâu S , vì vậy mọi DFA được nhắc tới đều thỏa $F = Q$.

12.3 Các cấu trúc dữ liệu hậu tố truyền thống trong OI

12.3.1 Trie hậu tố

Để có một DFA có thể chấp nhận mọi xâu con của S , và không chấp nhận xâu nào khác, ta dễ nghĩ tới việc xây một trie cho tất cả hậu tố của S . Vì trie có thể chấp nhận các tiền tố của một xâu nào đó trong tập, còn các tiền tố của mọi hậu tố của S đúng bằng tập các xâu con của S , trie này chấp nhận đúng và đủ mọi xâu con của S .

Định nghĩa 3.1. Xây trie cho $\{\text{Suf}[i] \mid i \in \{1, 2, \dots, |S|\}\}$. DFA thu được gọi là trie hậu tố.

Ví dụ 3.1. Trie hậu tố của xâu $S = \text{abaab}$ được minh họa trong bản gốc như sau:

Hình ví dụ 3.1: trie hậu tố của $S = \text{abaab}$; các nút màu xám là những nút ứng với một hậu tố của S .

Có thể thấy trie hậu tố chấp nhận đúng và đủ mọi xâu con của S .

12.3.2 Suffix automaton

Trong trường hợp xấu nhất, số nút và số cạnh của trie hậu tố đều có thể đạt cấp $O(|S|^2)$, thường khó chấp nhận trong ứng dụng thực tế. Vì vậy, ta cần tìm một cấu trúc dữ liệu hậu tố hiệu quả hơn.

Như đã nói ở trên, trie hậu tố là một DFA, còn DFA có khái niệm tối thiểu hóa.

Chẳng hạn trong Ví dụ 3.1, có thể thấy các nút aab, baab, abaab đều không có cạnh đi ra. Vì vậy, có thể xem chúng là tương đương và gộp thành cùng một nút. Tương tự, các nút baa và abaa đều chỉ có một cạnh đi ra, cùng ký tự, trở tới các nút tương đương. Do đó các chuyển tiếp phía sau ở hai nút này là giống nhau; ta cũng có thể xem chúng là tương đương và gộp thành một nút.

Định nghĩa 3.2. Gộp các nút có cùng tập Right trên trie hậu tố. DFA thu được gọi là suffix automaton.

Ví dụ 3.2.1. Suffix automaton của xâu $S = \text{abaab}$ được minh họa trong bản gốc như sau:

Hình ví dụ 3.2.1: suffix automaton của $S = \text{abaab}$, các nhãn nút là tập Right; ngoài trạng thái bắt đầu, các nút có bậc ra khác 1 được tô xám.

Có thể thấy nếu DFS theo các cạnh của suffix automaton, ta khôi phục được trie hậu tố ban đầu.

Trong Định nghĩa 3.2, ta không trực tiếp nhắc tới việc tối thiểu hóa trie hậu tố. Nhưng xét ý nghĩa của việc tối thiểu hóa trie hậu tố, ta muốn giảm số nút của DFA nhiều nhất có thể mà không ảnh hưởng tới tập xâu mỗi nút có thể chấp nhận. Trong bài toán khớp xâu con, quá trình này gần như chính là gộp các nút có cùng tập Right của xâu tương ứng. Điều này tương tự định nghĩa suffix automaton ở Định nghĩa 3.2.

Dù vậy, cần chỉ ra rằng suffix automaton không hoàn toàn đồng nhất với trie hậu tố đã tối thiểu hóa.

Để phân biệt hai khái niệm này, hãy xét một ví dụ cụ thể.

Ví dụ 3.2.2. Bản gốc lần lượt minh họa trie hậu tố, suffix automaton và trie hậu tố tối thiểu hóa của xâu $S = \text{baa}$:

Hình ví dụ 3.2.2: ba cấu trúc của $S = \text{baa}$: trie hậu tố, suffix automaton, và trie hậu tố tối thiểu hóa.

Có thể thấy hai nút có tập Right lần lượt là $\{2, 3\}$ và $\{2\}$ được gộp trong trie hậu tố tối thiểu hóa. Thật vậy, vì một lần xuất hiện có đầu phải là $|S|$ không còn ký tự phía sau, nếu hai tập Right chỉ khác nhau ở phần tử $|S|$, thì các chuyển tiếp phía sau của chúng là như nhau. Ngược lại, nếu hiệu đối xứng của hai tập Right chứa một phần tử khác $|S|$, thì các chuyển tiếp phía sau chắc chắn khác nhau, nên hai tập này cũng là hai nút khác nhau trong trie hậu tố tối thiểu hóa.

Từ đó có thể thấy khác biệt duy nhất giữa suffix automaton và trie hậu tố tối thiểu hóa nằm ở việc có gộp các nút mà hiệu đối xứng của hai tập Right tương ứng bằng $\{|S|\}$ hay không. Nếu khi tối thiểu hóa, ta xem các nút trên trie hậu tố đại diện cho một hậu tố, tức các nút màu xám trong hình, là loại nút khác với nút thông thường, thì suffix automaton và trie hậu tố tối thiểu hóa có cùng định nghĩa.

Trong phần sau, khi nói “suffix automaton là kết quả tối thiểu hóa trie hậu tố”, chữ “tối thiểu hóa” đều mang nghĩa: tối thiểu hóa trong điều kiện xem các nút trên trie hậu tố đại diện cho một hậu tố là khác với nút thông thường.

Bổ đề 3.2. Với hai xâu con bất kỳ x, y của S , ít nhất một trong hai điều sau đúng:

$$\begin{aligned}\text{Right}(x) \cap \text{Right}(y) &= \emptyset, \\ \text{Right}(x) &\subseteq \text{Right}(y) \quad \text{hoặc} \quad \text{Right}(y) \subseteq \text{Right}(x).\end{aligned}$$

Chứng minh. Nếu $\text{Right}(x) \cap \text{Right}(y) \neq \emptyset$, chắc chắn tồn tại $s \in \text{Right}(x) \cap \text{Right}(y)$. Từ đó suy ra x, y đều là hậu tố của $\text{Pre}[s]$, nên ít nhất một trong x, y là hậu tố của xâu còn lại. Không mất tính tổng quát, giả sử x là hậu tố của y , khi đó $\text{Right}(y) \subseteq \text{Right}(x)$. $\square$

Sự tồn tại của Bổ đề 3.2 đảm bảo số nút của suffix automaton ở cấp $O(|S|)$. Thật vậy, một suffix automaton có nhiều nhất $2|S| - 1$ nút và $3|S| - 4$ cạnh chuyển. Chính nhờ độ phức tạp thời gian và không gian tốt này mà suffix automaton được dùng rộng rãi trong thi OI.

12.3.3 Suffix tree

Muốn giảm độ phức tạp thời gian và không gian của trie hậu tố còn có một cách khác. Chú ý rằng trie hậu tố thực chất là hợp của nhiều nhất $|S|$ đường đi bắt đầu từ gốc, nên số lá của nó không vượt quá $|S|$. Khi đó cây ảo do các lá này tạo thành có số nút không quá $2|S| - 1$.

Định nghĩa 3.3.1. Xây cây ảo trên các lá của trie hậu tố. Kết quả thu được gọi là suffix tree.

Ví dụ 3.3.1. Suffix tree của xâu $S = \text{abaab}$ được minh họa trong bản gốc:

Hình ví dụ 3.3.1: suffix tree của $S = \text{abaab}$.

Có thể thấy quá trình xây suffix tree thực chất là co tất cả các nút có bậc ra bằng 1 trên trie hậu tố, từ đó giảm số nút của suffix tree. Ta gọi quá trình này là phép co trên trie hậu tố.

Trong Ví dụ 3.3.1, một số nút màu xám vốn đại diện cho một hậu tố của S cũng bị co và giản lược. Nhưng khi xử lý bài toán thực tế, ta thường cần xét riêng các nút đại diện cho một hậu tố này. Vì vậy, có thể định nghĩa suffix tree theo một cách khác.

Định nghĩa 3.3.2. Xây cây ảo trên các nút đại diện cho hậu tố của xâu gốc trong trie hậu tố. Kết quả thu được gọi là suffix tree đầy đủ.

Ví dụ 3.3.2. Suffix tree đầy đủ của xâu $S = \text{abaab}$ được minh họa trong bản gốc:

Hình ví dụ 3.3.2: suffix tree đầy đủ của $S = \text{abaab}$, trong đó các nút đại diện cho hậu tố của xâu gốc được giữ lại.

So với suffix tree, suffix tree đầy đủ giữ lại toàn bộ các nút đại diện cho hậu tố của xâu gốc.

Cần chú ý rằng vì một số đường đi đã được nén lại, một cạnh có thể chứa nhiều ký tự, nên suffix tree thực ra không được xem là một DFA. Dù vậy, suffix tree vẫn có độ phức tạp thời gian và không gian tuyến tính rất tốt, đồng thời có hình thức trực quan hơn suffix automaton. Chính vì thế, suffix tree cũng là một công cụ mạnh để xử lý xâu trong thi OI.

12.4 Suffix automaton nén

12.4.1 Định nghĩa

Thực hiện phép co trên trie hậu tố sẽ thu được suffix tree.

Thực hiện phép tối thiểu hóa trên trie hậu tố sẽ thu được suffix automaton.

Vậy nếu đồng thời thực hiện phép co và phép tối thiểu hóa trên trie hậu tố, ta sẽ thu được gì?

Định nghĩa 4.1.1. Đồng thời thực hiện phép co và phép tối thiểu hóa trên trie hậu tố. Kết quả thu được gọi là suffix automaton nén.

Ví dụ 4.1.1. Suffix automaton nén của xâu $S = \text{abaab}$ được minh họa trong bản gốc:

Hình ví dụ 4.1.1: suffix automaton nén của $S = \text{abaab}$.

So sánh Ví dụ 3.2.1 và Ví dụ 3.3.1, có thể thấy suffix automaton nén của xâu S đúng là kết quả co tất cả các nút có bậc ra bằng 1 trên suffix automaton. Đồng thời, nếu DFS trên suffix automaton nén, ta có thể khôi phục suffix tree của xâu S .

Điều này cũng phù hợp với định nghĩa suffix automaton nén là đồng thời co và tối thiểu hóa trie hậu tố.

Ta cũng có thể định nghĩa một suffix automaton nén tương ứng với suffix tree đầy đủ.

Định nghĩa 4.1.2. Tối thiểu hóa suffix tree đầy đủ. Kết quả thu được gọi là suffix automaton nén đầy đủ.

Ví dụ 4.1.2. Suffix automaton nén đầy đủ của xâu $S = \text{abaab}$ được minh họa trong bản gốc:

Hình ví dụ 4.1.2: suffix automaton nén đầy đủ của $S = \text{abaab}$.

12.4.2 Cách xây dựng

Với một DFA không có tính chất đặc biệt, độ phức tạp thời gian của tối thiểu hóa thường khó chấp nhận, nhưng phép co thì rất dễ. Vì vậy, có thể co suffix automaton đã biết để thu được suffix automaton nén.

Ví dụ 4.2.1. Xét cách xây suffix automaton nén của xâu $S = \text{abaab}$.

Như Ví dụ 3.2.1, trước hết xây suffix automaton của S , rồi đánh dấu các nút có bậc ra khác 1 bằng màu xám.

Xử lý các nút chưa được đánh dấu và có bậc ra bằng 1 theo thứ tự topo của suffix automaton, tính nút màu xám đầu tiên đạt tới nếu đi theo cạnh đi ra từ các nút này. Sau đó, lần lượt xử lý trạng thái bắt đầu và từng cạnh đi ra của các nút màu xám, rút gọn cạnh đó thành cạnh nối tới nút màu xám đầu tiên đạt tới khi đi theo cạnh ấy.

Kết quả cuối cùng đúng như Ví dụ 4.1.1.

Vì độ phức tạp thời gian và không gian để xây suffix automaton đều là $O(|S|)$, và quá trình co cũng có độ phức tạp thời gian và không gian $O(|S|)$, nên độ phức tạp thời gian và không gian để xây suffix automaton nén đều là $O(|S|)$.

Nhưng trong định nghĩa suffix automaton nén đầy đủ, không có suffix automaton tương ứng trực tiếp để đem co. Vậy làm sao xây suffix automaton nén đầy đủ của xâu mà tránh thao tác tối thiểu hóa?

Ví dụ 4.2.2. Xét cách xây suffix automaton nén đầy đủ của xâu $S = abaab$.

Theo định nghĩa, DFS trên suffix automaton nén đầy đủ phải khôi phục được suffix tree đầy đủ. Nghĩa là ngoài các nút có bậc ra khác 1 trên suffix automaton, ta còn cần đánh dấu thêm một số nút màu xám, sao cho các nút đại diện cho hậu tố của xâu gốc trên suffix tree được giữ lại.

Hình ví dụ 4.2.2: trên suffix automaton, đánh dấu thêm các nút có tập Right chứa $|S|$.

Như hình trên, xét việc đánh dấu thêm các nút trên suffix automaton có tập Right chứa $|S|$. Những nút như vậy trên trie hậu tố chắc chắn đại diện cho một hậu tố của xâu gốc. Vì vậy, sau khi đánh dấu chúng bằng màu xám rồi thực hiện phép co như Ví dụ 4.2.1, ta thu được kết quả tối thiểu hóa suffix tree đầy đủ, tức suffix automaton nén đầy đủ. Kết quả cuối cùng đúng như Ví dụ 4.1.2.

Có thể thấy độ phức tạp thời gian và không gian để xây suffix automaton nén đầy đủ cũng đều là $O(|S|)$.

12.4.3 Tính chất và ứng dụng

Từ định nghĩa và cách xây dựng suffix automaton nén, ta thu được các tính chất cơ bản sau:

Tính chất 4.3.1. Suffix automaton nén là kết quả co suffix automaton.

Mọi xâu trên các cạnh đi vào cùng một nút của suffix automaton nén đều là hậu tố của xâu dài nhất trong số đó.

Tính chất 4.3.2. Suffix automaton nén là kết quả tối thiểu hóa suffix tree.

DFS trên suffix automaton nén có thể khôi phục suffix tree.

Để phát huy tốt hơn ưu thế của suffix automaton nén, ta còn cần chứng minh một tính chất quan trọng của nó. Khi dùng suffix automaton nén để giải bài toán, tính chất này thường đặc biệt then chốt.

Định lý 4.3. Gọi L_i là độ dài cạnh dài nhất trong tất cả các cạnh đi vào nút thứ i của suffix automaton nén. Khi đó

$$\sum_i L_i = O(|S|).$$

Chứng minh. Xét quá trình co suffix automaton.

Với một cạnh vốn có trên suffix automaton, có hai khả năng:

1. Cạnh đó xuất hiện trên cạnh đi vào dài nhất của một nút màu xám. Khi đó, nó đóng góp 1 vào $\sum_i L_i$.
2. Cạnh đó không xuất hiện trên cạnh đi vào dài nhất của một nút màu xám. Khi đó, nó đóng góp 0 vào $\sum_i L_i$.

Vì vậy, một cạnh vốn có trên suffix automaton đóng góp nhiều nhất $O(1)$ vào $\sum_i L_i$. Theo Bổ đề 3.2, số cạnh của suffix automaton ở cấp $O(|S|)$, do đó $\sum_i L_i = O(|S|)$. $\square$

Tiếp theo, hãy xét một bài toán cụ thể.

Ví dụ 4.3. Sasha and Swag Strings. Cho một chuỗi S chỉ gồm chữ cái thường, $1 \leq |S| \leq 10^5$. Hãy tính tổng, trên mọi cạnh của suffix tree, số chuỗi con phân biệt về bản chất của chuỗi ghi trên cạnh đó.

Xét một cách giải truyền thống cho bài này. Chú ý rằng chuỗi ghi trên mỗi cạnh của suffix tree đều là một chuỗi con của S , nên bài toán có thể chuyển thành: với $O(|S|)$ chuỗi con của S , lần lượt tính số chuỗi con phân biệt về bản chất của chúng.

Với bài toán sau chuyển đổi, tài liệu tham khảo [5] giới thiệu một lời giải offline dùng Link-Cut Tree để duy trì suffix tree và dùng cây đoạn để duy trì đáp án. Độ phức tạp thời gian là $O(|S| \log^2 |S|)$.

Nhưng lời giải này cần nhiều cấu trúc dữ liệu nâng cao, khiến mã cài đặt thực tế khá khó. Đồng thời, độ phức tạp thời gian cũng cao, chỉ vừa đủ qua dữ liệu cấp 10^5 . Tiếp theo, ta giới thiệu một lời giải tốt hơn cho bài này bằng suffix automaton nén.

Theo Tính chất 4.3.2, ta biết suffix automaton nén là kết quả tối thiểu hóa suffix tree. Vì vậy, chỉ cần tính số chuỗi con phân biệt về bản chất của chuỗi ghi trên mỗi cạnh của suffix automaton nén, ta sẽ thu được số chuỗi con phân biệt về bản chất của chuỗi ghi trên mỗi cạnh của suffix tree.

Xét từng nút i , ta lần lượt tính số chuỗi con phân biệt về bản chất của mọi chuỗi ghi trên cạnh đi vào nút đó. Theo Tính chất 4.3.1, các chuỗi này đều là hậu tố của chuỗi dài nhất trong số đó.

Cách truyền thống để tính số chuỗi con phân biệt về bản chất của một chuỗi thường dựa vào suffix tree hoặc suffix automaton, và có thể hỗ trợ thêm ký tự theo một hướng của chuỗi, đồng thời duy trì động số chuỗi con phân biệt về bản chất. Do đó, ta có thể tính số chuỗi con phân biệt về bản chất của mọi chuỗi trên các cạnh đi vào nút i trong thời gian $O(L_i)$.

Với toàn bộ các nút, tổng thời gian để tính số chuỗi con phân biệt về bản chất của các chuỗi trên cạnh đi vào là

$$O\left(\sum_i L_i\right).$$

Theo Định lý 4.3,

$$O\left(\sum_i L_i\right) = O(|S|).$$

Như vậy, ta thu được một lời giải có độ phức tạp thời gian và không gian đều là $O(|S|)$. Đồng thời, so với thuật toán đã nhắc ở trên, thuật toán dùng suffix automaton nén dễ cài đặt hơn.

Có thể thấy suffix automaton nén không chỉ dễ cài đặt mà còn rất mạnh.

12.5 Suffix automaton nén đối xứng

12.5.1 Định nghĩa

Trong Ví dụ 3.2.1, ta dùng tập Right của chuỗi đầu vào T để mô tả các nút trên suffix automaton.

Xét tập các chuỗi T có thể được cùng một nút chấp nhận, ký hiệu $\{T_i\}$. Gọi chuỗi dài nhất trong đó là $T_{\max}$. Khi đó, với mọi $T \in \{T_i\}$, T phải là hậu tố của $T_{\max}$.

Ngoài ra, xét ý nghĩa của tập Right, ta có thể suy thêm:

$$\forall T \in \{T_i\}, \quad \text{LeftContext}(T) = T_{\max}.$$

Điều này nghĩa là ta có thể dùng một chuỗi cụ thể $T_{\max}$ để mô tả nút trên suffix automaton.

Ví dụ 5.1.1. Suffix automaton của chuỗi $S = abaab$ được minh họa trong bản gốc:

Hình ví dụ 5.1.1: suffix automaton của $S = abaab$, mô tả nút bằng chuỗi dài nhất tương ứng.

Từ đây ta cũng thấy, so với trie hậu tố, suffix automaton giữ lại các nút tương ứng với những chuỗi T thỏa

$$\text{LeftContext}(T) = T.$$

Một nút trên suffix automaton có bậc ra bằng 1 và tập Right tương ứng không chứa $|S|$ có đúng một cạnh đi ra, đồng thời không đại diện cho một hậu tố của chuỗi gốc. Điều này nghĩa là chuỗi T tương ứng với nó thỏa

$$\text{RightContext}(T) \neq T.$$

Trong quá trình xây suffix automaton nén đầy đủ, các nút này đều bị co.

Điều đó nghĩa là mỗi nút trên suffix automaton nén đầy đủ tương ứng với một chuỗi T đồng thời thỏa

$$\text{LeftContext}(T) = T, \quad \text{RightContext}(T) = T,$$

tức

$$\text{Context}(T) = T.$$

Vì vậy, ta cũng có thể dùng các chuỗi T thỏa $\text{Context}(T) = T$ để mô tả một nút.

Ví dụ 5.1.2. Suffix automaton nén đầy đủ của chuỗi $S = abaab$ được minh họa trong bản gốc:

Hình ví dụ 5.1.2: suffix automaton nén đầy đủ của $S = abaab$, mô tả nút bằng chuỗi có $\text{Context}(T) = T$.

Chú ý rằng các chuỗi T thỏa $\text{Context}(T) = T$ trong S và các chuỗi T' thỏa $\text{Context}(T') = T'$ trong chuỗi đảo S' có tương ứng một-một với nhau. Ta có thể đồng thời xây suffix automaton nén đầy đủ cho chuỗi thuận và chuỗi đảo của S , rồi gộp các nút tương ứng.

Định nghĩa 5.1. Đồng thời xây suffix automaton nén đầy đủ cho chuỗi thuận và chuỗi đảo, gộp các nút tương ứng, và giữ lại cả hai nhóm cạnh chuyển. Kết quả thu được gọi là suffix automaton nén đối xứng.

Ví dụ 5.1.3. Suffix automaton nén đối xứng của chuỗi $S = abaab$ được minh họa trong bản gốc:

Hình ví dụ 5.1.3: suffix automaton nén đối xứng của $S = abaab$; cạnh xanh là chuyển của chuỗi gốc, cạnh đỏ là chuyển của chuỗi đảo.

Trong hình, các cạnh chuyển màu xanh là cạnh của chuỗi gốc, biểu thị việc thêm ký tự vào phía sau chuỗi hiện tại; các cạnh chuyển màu đỏ là cạnh của chuỗi đảo, biểu thị việc thêm ký tự vào phía trước chuỗi hiện tại.

12.5.2 Cách xây dựng

Theo Định nghĩa 5.1, ta có thể xây suffix automaton nén đối xứng bằng cách xây suffix automaton nén đầy đủ cho chuỗi thuận và chuỗi đảo, rồi gộp các nút tương ứng.

Khi đó, ta cần một cấu trúc có thể nhanh chóng tìm các chuỗi con là đảo của nhau.

Có thể dùng hash chuỗi hoặc suffix array làm cấu trúc truy vấn chuỗi con cần thiết.

Tùy theo cấu trúc truy vấn chuỗi con được chọn, độ phức tạp thời gian và không gian để xây suffix automaton nén đối xứng là $O(|S|)$ hoặc $O(|S| \log |S|)$.

12.5.3 Tính chất và ứng dụng

Ngoài các tính chất của suffix automaton nén đã nhắc trong Mục 4.3, suffix automaton nén đối xứng còn có khả năng rất mạnh: có thể đồng thời thêm ký tự ở cả hai phía. Tiếp theo, ta dùng các ví dụ cụ thể để thấy sức mạnh của suffix automaton nén đối xứng.

Ví dụ 5.3.1. Tìm chuỗi con. Cho một chuỗi S độ dài N , $1 \leq N \leq 10^6$, chỉ gồm chữ cái thường. Chuỗi T ban đầu rỗng. Cần hỗ trợ online tổng cộng Q , $1 \leq Q \leq 10^6$, thao tác thuộc bốn loại sau:

1. Cho ký tự c , đặt $T = T + c$.
2. Cho ký tự c , đặt $T = c + T$.
3. Cho số nguyên i , gán T bằng chuỗi thu được sau thao tác thứ i .
4. Kiểm tra T có phải chuỗi con của S hay không; nếu có, tìm một vị trí xuất hiện bất kỳ của nó.

Nếu bài toán không có thao tác (2), rõ ràng ta có thể dùng suffix automaton truyền thống để giải.

Bài này đồng thời có thao tác thêm ký tự vào hai đầu của chuỗi đầu vào, điều mà suffix automaton truyền thống không giải quyết được. Vì vậy, xét xây suffix automaton nén đối xứng cho chuỗi S . Dùng nút tương ứng với $\text{Context}(T)$ để biểu diễn chuỗi T . Đồng thời, duy trì vị trí xuất hiện của T trong $\text{Context}(T)$; chú ý rằng do tính chất của ngữ cảnh, vị trí xuất hiện của T trong $\text{Context}(T)$ là duy nhất.

Với thao tác (1):

Nếu $\text{RightContext}(T) \neq T$, thì mỗi khi T xuất hiện trong S , phía sau nó chắc chắn xuất hiện một ký tự cụ thể. Ta kiểm tra c có phải ký tự đó hay không. Nếu đúng, thì $\text{Context}(T + c) = \text{Context}(T)$; nếu không, $T + c$ không phải chuỗi con của S .

Nếu $\text{RightContext}(T) = T$, thì $\text{Context}(T + c) \neq \text{Context}(T)$. Ta tìm cạnh đi ra bắt đầu bằng ký tự c trong suffix automaton nén theo chiều thuận. Nếu tồn tại, $\text{Context}(T + c)$ chính là nút đạt tới; nếu không, $T + c$ không phải chuỗi con của S .

Tương tự, với thao tác (2):

Nếu $\text{LeftContext}(T) \neq T$, thì mỗi khi T xuất hiện trong S , phía trước nó chắc chắn xuất hiện một ký tự cụ thể. Ta kiểm tra c có phải ký tự đó hay không. Nếu đúng, thì $\text{Context}(c + T) = \text{Context}(T)$; nếu không, $c + T$ không phải chuỗi con của S .

Nếu $\text{LeftContext}(T) = T$, thì $\text{Context}(c + T) \neq \text{Context}(T)$. Ta tìm cạnh đi ra bắt đầu bằng ký tự c trong suffix automaton nén theo chiều đảo. Nếu tồn tại, $\text{Context}(c + T)$ chính là nút đạt tới; nếu không, $c + T$ không phải chuỗi con của S .

Xét thao tác (3). Với một trạng thái, ta chỉ cần duy trì nút tương ứng với $\text{Context}(T)$, và vị trí xuất hiện của T trong $\text{Context}(T)$. Vì vậy, chỉ cần dùng mảng ghi lại các thông tin này sau mỗi thao tác là có thể hỗ trợ thao tác (3).

Cuối cùng, với thao tác (4), ta đã duy trì được $\text{Context}(T)$ và vị trí xuất hiện của T trong $\text{Context}(T)$. Vì vậy, việc kiểm tra T có phải xâu con của S hay không chính là kiểm tra có tồn tại nút tương ứng với $\text{Context}(T)$ hay không; còn tìm một vị trí xuất hiện bất kỳ của $\text{Context}(T)$ sẽ suy ra được một vị trí xuất hiện của T .

Với mỗi nút trên suffix automaton nén đối xứng, khi xây dựng ta có thể xử lý trước một vị trí xuất hiện của nó trong S , từ đó hỗ trợ thao tác (4). Lời giải này có độ phức tạp thời gian $O(N + Q)$ và độ phức tạp không gian $O(N)$.

Điều này cho thấy ưu thế của suffix automaton nén đối xứng khi xử lý các bài toán thêm ký tự hai chiều.

Ví dụ 5.3.2. Alice and Bob and A String. Cho một xâu S chỉ gồm chữ cái thường, $1 \leq |S| \leq 10^5$.

Alice và Bob đang chơi một trò chơi. Ban đầu, họ có một xâu con T của S . Alice đi trước, hai người luân phiên thực hiện thao tác sau: thêm một ký tự vào trước và một ký tự vào sau T , sao cho kết quả vẫn là một xâu con của S . Người không thể thao tác thua.

Giả sử Alice và Bob đều chơi tối ưu để mình thắng. Với toàn bộ xâu con của S , hãy giúp Alice tính xâu T nhỏ thứ k , $1 \leq k \leq 10^{10}$, theo thứ tự từ điển trong số các xâu khiến cô ấy thắng chắc; hoặc kết luận không tồn tại đủ k xâu khiến Alice thắng chắc.

Trước hết xét cách xác định, với một xâu T cho trước, Alice có thắng được hay không.

Cách nghĩ trực quan nhất là xét mọi xâu con T của S từ dài tới ngắn. Gọi dp_T cho biết xâu con T có phải trạng thái thắng của người đi trước hay không. Nếu T có thể chuyển tới một trạng thái thua của người đi trước, thì $dp_T = \text{true}$; ngược lại, tức mọi trạng thái T có thể chuyển tới đều là trạng thái thắng của người đi trước, thì $dp_T = \text{false}$.

Tuy nhiên, cách này cần xét mọi xâu con của S , không thể chấp nhận khi $|S|$ đạt cấp 10^5 .

Xét một xâu T đồng thời thỏa

$$\text{LeftContext}(T) \neq T, \quad \text{RightContext}(T) \neq T.$$

Mỗi khi T xuất hiện trong S , trước và sau nó chắc chắn xuất hiện các ký tự cố định. Vì vậy, với trạng thái có xâu hiện tại là T , người đi trước chỉ có đúng một lựa chọn. Khi đó, ta có thể đẩy trò chơi theo lựa chọn duy nhất này cho tới khi ít nhất một trong hai điều sau đúng:

$$\text{LeftContext}(T) = T, \quad \text{RightContext}(T) = T.$$

Theo Định lý 4.3, số xâu thỏa ít nhất một điều kiện trên ở cấp $O(|S|)$. Vì vậy, sau khi xây suffix automaton nén đối xứng, ta chỉ cần thực hiện DP trên những xâu như vậy.

Tiếp theo, xét cách tìm trạng thái thắng nhỏ thứ k theo thứ tự từ điển.

Theo Tính chất 4.3.2, nếu DFS trên suffix automaton nén, ta có thể khôi phục suffix tree của xâu gốc. Mà suffix tree vừa hay sắp xếp toàn bộ xâu con của S theo thứ tự từ điển. Vì vậy, nếu ta có thể nhanh chóng tính tổng số trạng thái thắng trên một cạnh, thì chỉ cần DFS trên suffix automaton nén là xác định được đáp án nằm trên cạnh nào, từ đó xác định đáp án.

Theo quy tắc chuyển, có thể thấy tổng số trạng thái thắng trên một cạnh có thể được tính nhanh bằng cách, với từng nút X , duy trì tổng tiền tố theo từng chặn lẻ độ dài của trạng thái thắng/thua của các xâu T thỏa

$$\text{Context}(T) = \text{LeftContext}(T) = X$$

hoặc

$$\text{Context}(T) = \text{RightContext}(T) = X.$$

Như vậy, ta thu được một lời giải có độ phức tạp thời gian $O(\sigma|S|)$, trong đó $\sigma = 26$.

12.6 Tổng kết

Bài viết xuất phát từ một loạt cấu trúc dữ liệu hậu tố truyền thống trong thi tin học, rồi dẫn ra một cấu trúc dữ liệu hậu tố mới: suffix automaton nén, cùng một biến thể của nó là suffix automaton nén đối xứng. Với cả hai cấu trúc, bài viết phân tích chi tiết các tính chất, đưa ra một thuật toán xây dựng có độ phức tạp thời gian và không gian tuyến tính khi được phép xử lý offline, đồng thời giới thiệu các ứng dụng thực tế.

Trong bài viết này, tác giả dùng nhiều hình vẽ để hỗ trợ giải thích, ngôn ngữ cũng dễ hiểu. Tác giả cho rằng ngay cả các thí sinh có nền tảng sâu còn yếu, sau khi đọc bài viết này cũng có thể thu được điều gì đó.

Đáng nói là suffix automaton nén được giới thiệu trong bài thực ra là kết quả tự nhiên sau khi tiếp tục suy xét về suffix tree và suffix automaton quen thuộc với mọi người. Điều này cho thấy nhiều khi cách giải quyết bài toán mới được ẩn trong các công cụ đã có; chỉ cần dám đổi mới và tích cực suy nghĩ, ta có thể phát hiện ra chúng.

Tác giả chú ý rằng trong cộng đồng thi tin học hiện nay, sự quan tâm của học sinh đối với suffix automaton vẫn rất lớn. Hy vọng suffix automaton nén được giới thiệu trong bài viết này, cùng cách suy nghĩ phía sau nó, có thể gợi mở thêm nhiều suy nghĩ và tìm tòi về cấu trúc dữ liệu hậu tố, cũng như về các bài toán rộng hơn.

Lời cảm ơn

Cảm ơn China Computer Federation đã cung cấp nền tảng học tập và giao lưu.

Cảm ơn thầy Cao Wen của Trường Trung học cao cấp Thường Châu, Giang Tô, và thầy Wu Tao đã quan tâm, chỉ dẫn tôi trong nhiều năm.

Cảm ơn anh Guo Xiaoxu đã giúp đỡ và hướng dẫn cho bài viết này.

Cảm ơn huấn luyện viên đội tuyển Gao Wenyan đã giúp đỡ và hướng dẫn tôi.

Cảm ơn bạn Wang Xiuhuan, bạn Yang Junzhao và bạn Du Weihua cùng trường đã đọc duyệt bài viết này.

Cảm ơn gia đình và bạn bè đã ủng hộ, động viên tôi.

Cảm ơn các thầy cô và các bạn đã từng giúp đỡ tôi.

Tài liệu tham khảo

1. Wikipedia contributors. *Deterministic finite automaton*.
2. Wikipedia contributors. *DFA minimization*.
3. Chen Lijie, *Suffix automaton*. Slide bài giảng NOI WC 2012.
4. Liu Yanyi, *Mở rộng suffix automaton trên trie*. Tập luận văn đội tuyển quốc gia Trung Quốc, 2015.
5. Chen Jianglun, *Báo cáo ra đề Số nút của suffix tree và tối ưu một lớp bài toán đoạn*. Tập luận văn đội tuyển quốc gia Trung Quốc, 2018.
6. Luo Suiqian, *Suffix array: công cụ mạnh để xử lý sâu*. Tập luận văn đội tuyển quốc gia Trung Quốc, 2009.
7. Southeast University Press, *Cấu trúc dữ liệu nâng cao*.
8. Li Yudong, *Hướng dẫn nâng cao thi thuật toán*.
9. OI Wiki, *Virtual tree*, <https://oi-wiki.org/graph/virtual-tree/>.

10. OI Wiki, *String hashing*, <https://oi-wiki.org/string/hash/>.